ABSTRACT

Title of Thesis: A FRAMEWORK FOR BENCHMARKING GRAPH-BASED ARTIFICIAL INTELLIGENCE

Kent Daniel O'Sullivan
Master of Science, 2024

Thesis Directed by: Professor William Regli
Department of Computer Science

Graph-based Artificial Intelligence (GraphAI) encompasses AI problems formulated using graphs, operating on graphs, or relying on graph structures for learning. Contemporary Artificial Intelligence (AI) research explores how structured knowledge from graphs can enhance existing approaches to meet the real world's demands for transparency, explainability, and performance.

Characterizing GraphAI performance is challenging because different combinations of graph abstractions, representations, algorithms, and hardware acceleration techniques can trigger unpredictable changes in efficiency. Although benchmarks enable testing different GraphAI implementations, most cannot currently capture the complex interaction between effectiveness and efficiency, especially across dynamic and structured knowledge graphs.

This work proposes an empirical 'grey-box' approach to GraphAI benchmarking, providing a method that enables experimentally trading between effectiveness and efficiency across different combinations of graph abstractions, representations, algorithms, and hardware accelerators. A systematic literature review yields a taxonomy of GraphAI tasks and a collection of intelligence

and security problems that interact with GraphAI . The taxonomy and problem survey guide the development of a framework that fuses empirical computer science with constraint theory in an approach to benchmarking that does not require invasive workload analyses or code instrumentation.

We formalize a methodology for developing problem-centric GraphAI benchmarks and develop a tool to create graphs from OpenStreetMaps data to fill a gap in real-world mesh graph datasets required for benchmark inputs. Finally, this work provides a completed benchmark for the Population Segmentation Intelligence and Security problem developed using the GraphAI benchmark problem development methodology. It provides experimental results that validate the utility of the GraphAI benchmark framework for evaluating if, how, and when GraphAI acceleration should be applied to the population segmentation problem.

A FRAMEWORK FOR BENCHMARKING GRAPH-BASED ARTIFICIAL INTELLIGENCE

by

Kent Daniel O'Sullivan

Thesis submitted to the Faculty of the Graduate School of the
University of Maryland, College Park in partial fulfillment
of the requirements for the degree of
Master of Science
2024

Advisory Committee:
William Regli, Chair/Advisor
Mohammad Hajiaghayi
Brian Pierce
Laxman Dhulipala

© Copyright by
Kent O'Sullivan
2024

Acknowledgments

Dr. Bill Regli, for his guidance as my advisor and support throughout my time at the University of Maryland.

My Thesis Committee: Dr. Mohammad Hajiaghayi, Dr. Brian Pierce, and Dr. Laxman Dhulipala for their time and constructive feedback.

Nicole, for her extensive editing support, unwavering willingness to collaborate on research, and insistence on pushing me to publish my work over the last two years.

Nate, Sam, and Taylor, for the morale our study group provided.

The Australian-American Fulbright Commission, the Kinghorn Foundation, and the University of Maryland for their scholarships which gave me the means to undertake this course of study.

The Australian Army, for supporting my desire to study computer science for two years.

The Applied Research Laboratory for Intelligence and Security for supporting my research.

Most importantly, to my partner Mary for supporting me in moving to the other side of the world to study, pausing her goals while I pursued mine, and making sure that I saw more of the United States than just the inside of the Computer Science Department.

Table of Contents

Acknowledgements ii

Table of Contents iii

List of Tables vi

List of Figures vii

List of Abbreviations ix

Chapter 1: Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Problem and Background . . . 2
- 1.3 Developing the GraphAI Benchmark Framework and GraphAI Benchmark Methodology. . . . 5
- 1.4 Applying Our GraphAI Benchmark Framework . . . 8
- 1.5 Applications and Impact of the GraphAI Benchmarking Framework . . . 9
- 1.6 Thesis Outline . . . 10

Chapter 2: Preliminaries 12
- 2.1 Graph-Based Artificial Intelligence . . . 12
  - 2.1.1 A Taxonomy of GraphAI . . . 13
  - 2.1.2 Real-World Uses of GraphAI . . . 15
- 2.2 Graph Definitions . . . 15
  - 2.2.1 Graph Components . . . 16
  - 2.2.2 Graph Characteristics . . . 17
  - 2.2.3 Graph Types . . . 19
- 2.3 Graph Topology . . . 20
  - 2.3.1 Real-World Graphs . . . 20
  - 2.3.2 Synthetic Graphs . . . 21
  - 2.3.3 Summary . . . 22
- 2.4 Graph Representations . . . 23
  - 2.4.1 Graph Storage . . . 23
  - 2.4.2 Graph Abstractions . . . 27
- 2.5 Graph Primitive Operations . . . 29
- 2.6 Architectural Constraints for Graph Processing: Locality . . . 30

2.6.1 Locality . . . 31
2.7 Summary: Optimizing and Evaluating GraphAI is a Hard Problem . . . 34

Chapter 3: Literature Review **35**
3.1 Benchmarking . . . 35
3.1.1 Competitive 'Black-Box' Benchmarks . . . 36
3.1.2 'White-Box' Benchmarks . . . 37
3.1.3 Constrained Benchmarks . . . 38
3.1.4 Summary . . . 38
3.2 Benchmarking GraphAI . . . 40
3.2.1 Task Coverage. . . . 42
3.2.2 Dataset Coverage. . . . 46
3.2.3 Metric Coverage. . . . 49
3.3 GraphAI Bottlenecks . . . 51
3.3.1 Memory Bottlenecks . . . 51
3.3.2 Computation Bottlenecks . . . 52
3.3.3 Communication Bottlenecks . . . 53
3.4 Summary: GraphAI benchmarking requires a 'grey-box' approach . . . 53

Chapter 4: The Graph Artificial Intelligence Benchmarking Framework **55**
4.1 Motivation . . . 55
4.2 Benchmark Mechanics . . . 56
4.2.1 Benchmark Specification . . . 58
4.2.2 System Under Test . . . 58
4.2.3 Grey-Box Evaluation . . . 59
4.3 GraphAI Benchmark Design . . . 60
4.3.1 Empirical Benchmark Design . . . 61
4.3.2 Design using the Theory of Constraints . . . 63
4.4 Our GraphAI Benchmark Framework . . . 63
4.5 Summary: A 'Grey-Box' GraphAI Benchmark Framework . . . 67

Chapter 5: The Population Segmentation Problem **69**
5.1 Identify and Characterize the Population Segmentation Problem . . . 69
5.1.1 Definition . . . 69
5.1.2 Tasks . . . 70
5.1.3 Datasets . . . 71
5.1.4 Outputs . . . 72
5.2 Identify and Characterize Phenomena of Interest . . . 73
5.2.1 Efficiency . . . 73
5.2.2 Effectiveness . . . 73
5.2.3 Cost . . . 74
5.2.4 Outputs . . . 75
5.3 Conduct Exploratory Experimentation . . . 75
5.3.1 Datasets Observations . . . 77
5.3.2 Implementation Observations . . . 77

5.3.3 Outputs . . . 79
5.4 Develop Hypotheses . . . 80
5.4.1 Stimulating Hardware . . . 81
5.4.2 Stimulating Representations . . . 82
5.4.3 Stimulating Implementations . . . 83
5.4.4 Selecting Hypotheses . . . 84
5.5 Construct the Investigation Apparatus . . . 85
5.5.1 Observation Apparatus . . . 85
5.5.2 Datasets . . . 86
5.5.3 Tasks . . . 91
5.5.4 Metrics . . . 92
5.5.5 Limitations . . . 92
5.5.6 Outputs . . . 93
5.6 Analyze Results, Develop Theoretic Explanation and Iterate . . . 94
5.6.1 Experiment Setup . . . 94
5.6.2 Experiment Results . . . 96
5.7 Discussion . . . 101

Chapter 6: Conclusion **104**
6.1 Future Work . . . 104
6.2 Conclusion . . . 106

Bibliography **109**

List of Tables

2.1 The GraphAI taxonomy groups GraphAI tasks into GraphAI problems and broader GraphAI problem areas. 14
2.2 A survey of GraphAI problems in the Intelligence and Security (I&S) domain. . . . 16

3.1 Coverage of GraphAI tasks by existing benchmark suites and workload analyses. . 41
3.2 Coverage of GraphAI datasets by existing benchmark suites and workload analyses. 45
3.3 Coverage of GraphAI metrics by existing benchmark suites and workload analyses. 48

4.1 Summary of dependencies between benchmark components and evaluation metrics. 60
4.2 Summary of expected metric behaviors when a benchmark component is constrained. 66

5.1 Example problem-centric tasks for the population segmentation benchmark. . . . 71
5.2 Survey of computational approaches to population segmentation, showing efficiency and effectiveness bottlenecks. 76
5.3 Summary of datasets for the population segmentation problem showing size, summary statistics and domain. 90
5.4 Summary of experiments for the population segmentation benchmark. Each SUT completes the same tasks on the same data. 96

List of Figures

1.1 The GraphAI taxonomy consists of six GraphAI problem areas. Indicative tasks for each problem area are in grey. . . . 2
1.2 GraphAI challenges are a hierarchy of dependencies foundationally limited by graph processing. . . . 5
1.3 The GraphAI benchmark framework takes a grey-box approach to benchmarking, designing inputs to stimulate observable changes to outputs to infer system and implementation detail. . . . 6
1.4 The GraphAI benchmark design methodology for creating problem-centric benchmarks, showing how methodology outputs map to benchmark components. . . . 6
1.5 The high-level view of the GraphAI benchmarking framework. SUTs interact with the benchmark through an *observation apparatus* API. . . . 9

2.1 Example graph with five nodes and five edges. . . . 23
2.2 Adjacency List for graph in Figure 2.1. Each line represents a vertex, with the first in the list being the source and each subsequent vertex a destination of an edge. 24
2.3 Adjacency Matrix for graph in Figure 2.1 is a $|V| \times |V|$ matrix where a value of 1 indicates an edge and 0 is no edge. . . . 25
2.4 Incidence Matrix for the graph in Figure 2.1 is a $|E| \times |V|$ matrix where a -1 is the source node and 1 is the destination of a given edge. . . . 26
2.5 The Compressed Sparse Row (CSR) format uses three lists to represent the graph in Figure 2.1. . . . 27
2.6 A simplified representation of a directed graph in vertex order where each vertex is added to memory creation order (here, alphabetically). Edges are not shown. . 32
2.7 A simplified representation of a directed graph in vertex order where each vertex is added to memory creation order with attributes stored separately. Pointers from topology to attributes are not shown for clarity but correspond to the first letter of the attribute. . . . 33

3.1 The basic components of a generic benchmark system. . . . 39

4.1 The components of the GraphAI benchmarking framework. . . . 57
4.2 Hooker's methodology for an empirical science of algorithms [1, 2]. . . . 62
4.3 The components of the GraphAI benchmarking framework. . . . 68

5.1 The coverage of static Community Detection (CD) datasets showing the expected difficulty of effectiveness versus efficiency. Red colors are more 'difficult' interactions. . . . 91
5.2 The observation apparatus for the population segmentation benchmark. . . . 93

5.3 The experiments compare three SUT, each with a static and iterative implementation of the Louvain Algorithm [3]. 95
5.4 The GPU implementation is drastically faster than the Baseline and Xeon SUT for the same tasks. 97
5.5 The GPU SUT experiences a dip in NMI for Scale-Free datasets while effectiveness remains constant across both CPU-based SUTs. 97
5.6 Porportion of a total experiment shows GPU spend more time on ETL relative to total Execution Time (Ex Time). 99
5.7 The underlying hardware impacts the performance of iterative algorithms, with only the Xeon CPU SUT showing monotonic improvement that allows trading effectiveness for efficiency. 100

List of Abbreviations

A*	A-Star Search [Algorithm]
AI	Artificial Intelligence
ARLIS	Applied Research Laboratory for Intelligence and Security
BC	Betweenness Centrality
BFS	Breadth First Search [Algorithm]
BLAS	Basic Linear Algebra Subprograms
BSP	Bulk-Synchronous Parallel [programming model]
CC	Connected Components
CRUD	Create, Read, Update, Delete (database operations)
CSC	Compressed Sparse Column
CSP	Constraint Satisfaction Problem
CSR	Compressed Sparse Row
DAG	Directed Acyclic Graph
DARPA	Defense Advanced Research Projects Agency
DCM	Data-Centric Model for programming
DoD	United States Department of Defense
DFS	Depth First Search [Algorithm]
DRAM	Dynamic Random Access Memory
DSL	Domain Specific Languages
FLOPs	Floating Point Operations
FLOPS	Floating Point Operations per Second
FLOPS/W	Floating Point Operations per Second per Watt
GA	Graph Abstraction [AI problem category]
GAP	Berkeley Graph Algorithm Platform
GAS	Gather-Apply-Scatter [programming model]
GBBS	Graph-Based Benchmark Suite
GCC	Global Clustering Coefficient [Algorithm]
GCN	Graph Convolution Network
GDB	Graph Database
GNSS	Global Navigation Satellite System
GNN	Graph Neural Network
GP	Graph Prediction [AI problem category]
GR	Graph Reasoning [AI problem category]
HIVE	Hierarchical Identify Verify Exploit
HVT	High Value Target
IIoT	Industrial Internet of Things

I/O	Input/Output
IoT	Internet of Things
IPC	Instructions per Cycle
I&S	Intelligence and Security
KG	Knowledge Graph
LCC	Local Clustering Coefficient [Algorithm]
LLC	Last Level Cache
MLCC	Mean Local Clustering Coefficient [Algorithm]
MIT	Massachusetts Institute of Technology
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
MST	Minimum Spanning Tree [Algorithm]
NMI	Normalized Mutual Information
NUMA	Non-Uniform Memory Access
OE	Operating Environment
OPSEC	Operational Security
OSINT	Open Source Intelligence
OSM	Open Street Maps
POI	Point of Interest
PR	PageRank [Algorithm]
RAM	Random Access Memory
SA	(Graph) Structural Analysis [AI problem category]
SCC	Strongly Connected Components [Algorithm]
SGM	(isomorphic) Sub Graph Matching [Algorithm]
SNAP	Stanford Network Analysis Project
SOTA	State of the Art
SPEC	Standard Performance Evaluation Corporation
SPM	Spatial Pattern Matching
SPS	Searching, Planning and Scheduling [AI problem category]
SSSP	Single Source Shortest Path [Algorithm]
STGNN	Spatio-Temporal Graph Neural Network
SWaP	Size Weight and Power
TAI	Trustworthy AI [AI problem category]
TB	Terabytes
TC	Triangle Counting [Algorithm]
TEPS	Traversed Edges per Second [Metric]
TEPS/W	Traversed Edges per Second per Watt
UCS	Uniform Cost Search [Algorithm]
WCC	Weakly Connected Components [Algorithm]

Chapter 1

Introduction

1.1 Motivation

Graphs are expressive data structures that encode evidence as vertices and contextual relationships between evidence as edges. As powerful abstractions of the real world, graphs lend themselves well to learning and reasoning tasks, which encompass an emerging area of work we term Graph-Based Artificial Intelligence (GraphAI). We taxonomize GraphAI within the existing AI landscape, covering the well-known areas of Graph Prediction (GP) [4–6] and Graph Mining (GM) [7] and extending those to include Graph Problem Solving (GPS), Graph Reasoning (GR), Graph Abstraction (GA), and Graph Transparency (GT), which are detailed in Figure 1.1. Across these areas, GraphAI includes tasks like structural analysis, inference over graphs, and applications of graphs to AI problems like model transparency, explainability, and interpretability.

Although graphs are expressive and broadly applicable to various AI problems, they impose a significant computational burden on systems that operate GraphAI . Intelligent algorithms that operate on graphs frequently have quadratic or worse complexity, which limits their usability on real-world graphs containing billions or trillions of vertices. Modern general-purpose processors use caching and prefetching to speed up computation over large non-graph inputs. The challenge with graphs is that most GraphAI algorithms exploit the graph's structure in processing, which requires access to the data in topological order. That requirement conflicts with graph storage, which is typically chronological in the order that vertices are created, meaning topologically close

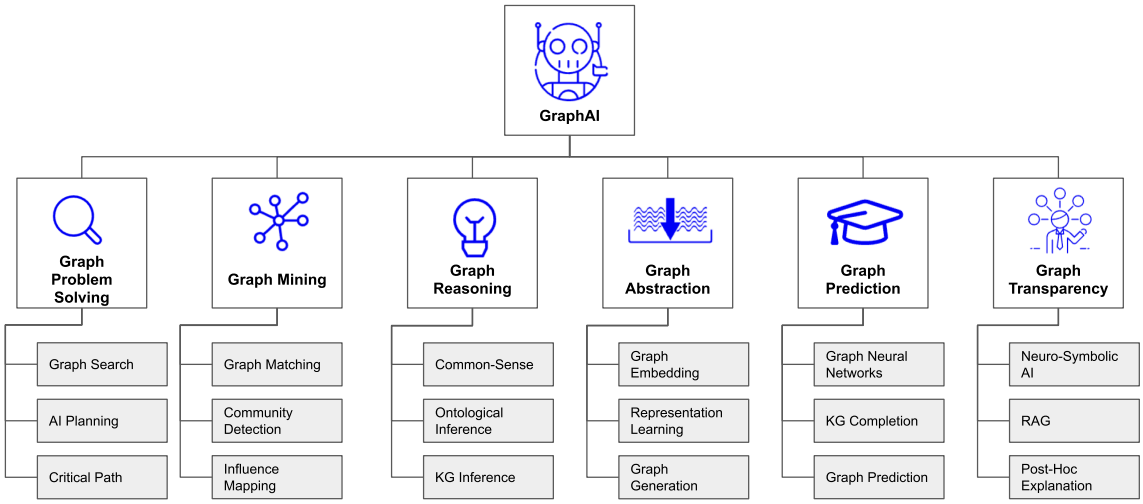


Figure 1.1: The GraphAI taxonomy consists of six GraphAI problem areas. Indicative tasks for each problem area are in grey.

vertices are not necessarily proximal in memory. Without caching benefits, GraphAI algorithms rely on data retrieved from main memory, which causes memory latency and bandwidth bottlenecks that limit scalability and prevent GraphAI from being deployed on real-world problems that might otherwise benefit from it.

1.2 Problem and Background

Common approaches to speeding up GraphAI processing typically use hardware, like specialized accelerators or distributed processing; representation, like custom data structures or abstraction models; or algorithms, like parallel or approximate methods. These approaches are interdependent, meaning decisions about optimizing hardware can limit choices about parallelism or programming models, which makes it difficult to determine the optimal configuration for a

given collection of problems.

This thesis aims to determine *if*, *how*, and *when* to speed up problem-solving on GraphAI given a set of operating *constraints*. Specifically, we consider how to effectively combine representations, hardware accelerators, and algorithms to speed up GraphAI and under what conditions a specific combination is appropriate for a use case given some constraints on a solution's efficiency, effectiveness, and cost. Prior approaches to identifying if, how, and when to accelerate a GraphAI system rely on generic competitive benchmarks or invasive workload analysis, both of which provide incomplete views of GraphAI performance across multiple possible system configurations.

Benchmarking is a process by which systems or components are compared according to some criteria, which is used to determine their relative performance [8]. Most benchmarks for GraphAI take a 'black-box' approach to competing systems based on *efficiency* or *effectiveness* metrics but rarely both, and never their interactions. Benchmarks typically focus on evaluating isolated *kernel*-level tasks rather than more complicated real-world problems and typically use a small selection of datasets that do not cover the breadth of expected input sizes and shapes. Existing benchmarks also lack support for dynamic graph processing approaches, especially comparing dynamic and static approaches under the same evaluation framework.

Alternatively, *workload analyses* are 'white-box' approaches that analyze the internal state of a system while completing a workload to identify bottlenecks. Although workload analyses yield detailed information about how to accelerate the System Under Test (SUT), they require access to the system for profiling and access to the code for instrumentation. The invasive configuration requirements mean workload analyses cannot be scaled for general-purpose comparisons, making them unsuitable for broad application to GraphAI .

In addition to the choice in the analytic method, a hierarchy of other challenges underlay the broader problem of deciding if, how, and when to improve GraphAI under some constraints, which we outline in Figure 1.2. Taking a user's top-down perspective of the challenge hierarchy, the user must first decide the degree to which effectiveness should be traded for efficiency or cost. The need to make this trade-off stems from the real-world constraints imposed on GraphAI systems, such as a requirement to generate a recurring solution at a pace more frequent than the algorithm's runtime allows. Further, most graph hardware accelerators are specialized Application Specific Integrated Circuit (ASIC) and are optimized to speed up a specific algorithm, representation, abstraction, and programming model. The problem with ASICs is that a given GraphAI system may need to support many graph tasks, so an accelerator that optimizes the performance of a singe GraphAI task may not garner a broad enough improvement in all GraphAI tasks to justify its cost. These real-world challenges and trade-offs are particularly pronounced for GraphAI because of the inherent nature of graph processing that underlies GraphAI problems. Graphs of different shapes and sizes unpredictably influence algorithmic complexity, including exacerbating irregular memory accesses and computation patterns that undermine spatial and temporal locality. When combined with the data dependence, the lack of standard graph representation methods, abstraction models, or a set of primitive operations further prevents a unity of effort toward accelerating graph processing across all graph tasks.

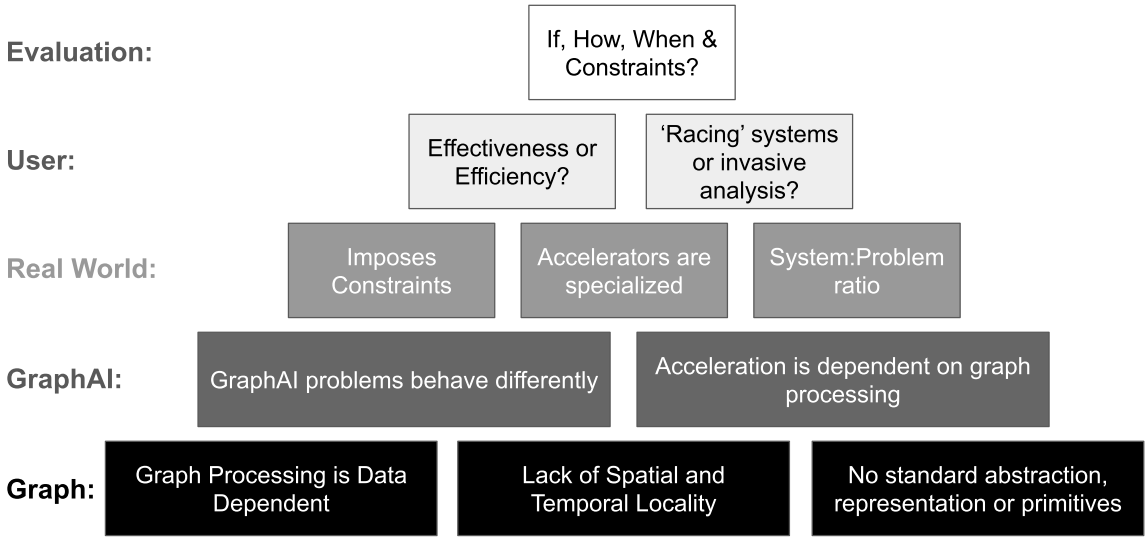


Figure 1.2: GraphAI challenges are a hierarchy of dependencies foundationally limited by graph processing.

1.3 Developing the GraphAI Benchmark Framework and GraphAI Benchmark Methodology.

We address these GraphAI issues by contributing our GraphAI benchmark framework, shown in Figure 1.5. The benchmark evaluates the performance of a SUT using an Input/Output observation apparatus to provide the SUT with problem-centric tasks and data, record the time to solution, and verify the correctness of an answer. Each GraphAI benchmark in the framework is independent, and we contribute an empirical methodology for constructing a problem-centric, accelerator-agnostic, 'grey-box' GraphAI benchmark that can evaluate both the *efficiency* and the *effectiveness* of dynamic and static GraphAI implementations and systems.

By 'grey-box', we mean an approach to benchmarking between the fully opaque black-box

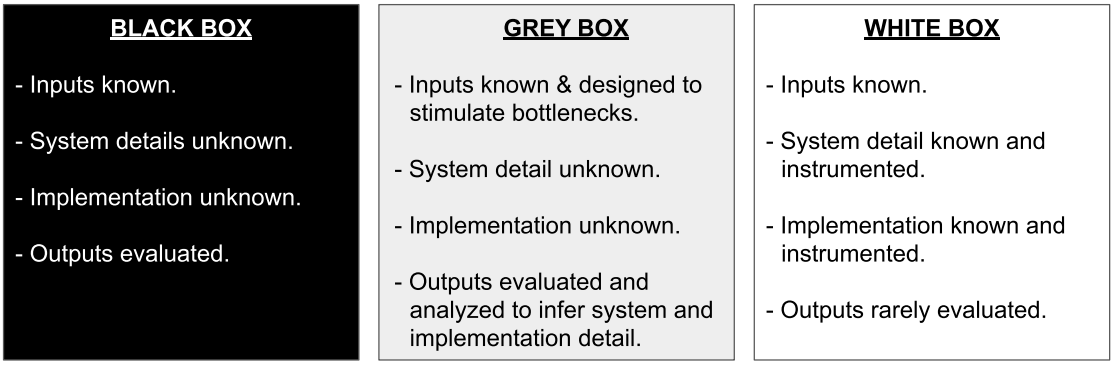


Figure 1.3: The GraphAI benchmark framework takes a grey-box approach to benchmarking, designing inputs to stimulate observable changes to outputs to infer system and implementation detail.

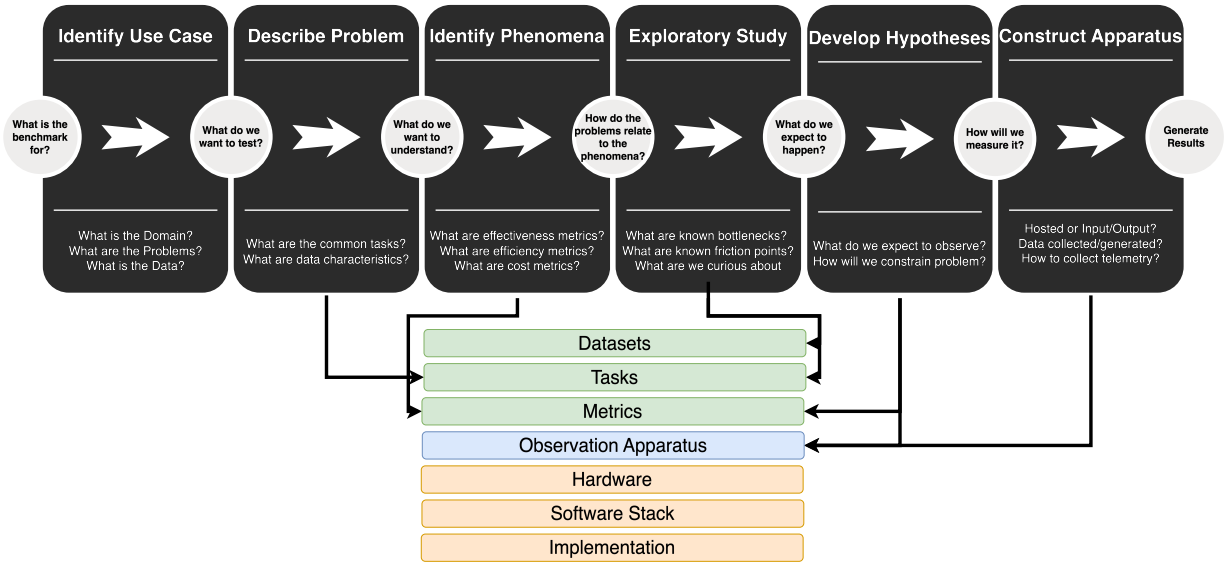


Figure 1.4: The GraphAI benchmark design methodology for creating problem-centric benchmarks, showing how methodology outputs map to benchmark components.

view that naively races systems against each other and the invasive white-box view that painstakingly monitors the internal state of a system (Figure 1.3). In implementing a grey-box view, we use our GraphAI benchmark design methodology to identify expected system and implementation bottlenecks and design adversarial inputs that we expect to stimulate those bottlenecks. By evaluating the output and analyzing the efficiency and effectiveness across datasets, we can infer which expected bottleneck will likely cause an observed output without invasive instrumentation. This approach of manipulating inputs to trigger bottlenecks observable in the outputs allows us to turn one of the most significant problems in graph processing (data dependence) into a valuable asset.

We employ a formal method (Figure 1.4) that emphasizes the transparency, verifiability, and reproducibility of benchmark problems. At its core, the method constructs a detailed mapping of the characteristics of the input graphs to the outputs of a GraphAI system and details how to interpret those outputs in the context of known and suspected bottlenecks in hardware, representation, and implementation decisions. The method comprises six steps that generate the components of the benchmark specification and the observation apparatus.

The process begins by identifying a use case to ground the problem in real-world datasets and current approaches. A more detailed analysis of the problem yields information about the size and shape of the data and formally specifies the everyday tasks associated with that problem in natural language. Formalizing the study's phenomena includes identifying what specific interactions of dataset and task are likely to yield interesting results, providing the first significant contribution to the problem's 'grey-box' approach. Considering the phenomena, exploratory analysis of the domain gathers the knowledge to make initial hypotheses mapping the input characteristics to observable outputs. The hypotheses drive the data collection to study the phenom-

ena of interest and any observed deviation from those hypotheses indicates interesting behavior warranting further study. The construction of the apparatus involves collecting and generating the datasets that will be used to trigger the conditions the hypotheses predict will stimulate the phenomena of interest.

Our proposed methodology for designing benchmark problems yields a collection of datasets, tasks, and metrics that combine to provide a 'grey-box' view of the SUT. The repeatable process we propose delivers an extensible framework that can be continually improved to evaluate new GraphAI problems as they emerge.

1.4 Applying Our GraphAI Benchmark Framework

To validate the utility of our proposed methodology, we construct a GraphAI benchmark for the Population Segmentation use case in the Intelligence and Security (I&S) domain. The *problem-centric* population segmentation tasks we select for our benchmark are real-world problem statements rather than simplified kernel-tasks. For example, we include the task '*What are the population segments, and who is in each?*' instead of the common kernel-task '*run Graph Clustering (GC)*'.

To achieve our 'grey-box' effect, we identify that the primary contributor to degraded *efficiency* in population segmentation is increasing dataset size and that the primary contributor to degraded *effectiveness* is increasing 'fuzziness' in community structure. We select real-world scale-free datasets with ground-truth community labels from the Stanford Large Network Dataset Collection (SLNDC) and generate real-world mesh datasets using a tool we create for converting Open StreetMaps (OSM) road network data to directed, weighted, attributed graphs with ground-

truth communities. To address the gaps in real-world data for datasets of large size and middling 'fuzziness,' we use the Lancichinetti, Fortunato, Radicci (LFR) [9] and *GenCAT* [10] community data generators.

We select the effectiveness metric of Normalized Mutual Information (NMI) to measure the similarity of predicted communities to ground truth and the efficiency metrics of Execution Time (Ex Time) and Batch Latency (BL), allowing us to compare static and dynamic approaches and trade-off between efficiency and effectiveness all under the same framework.

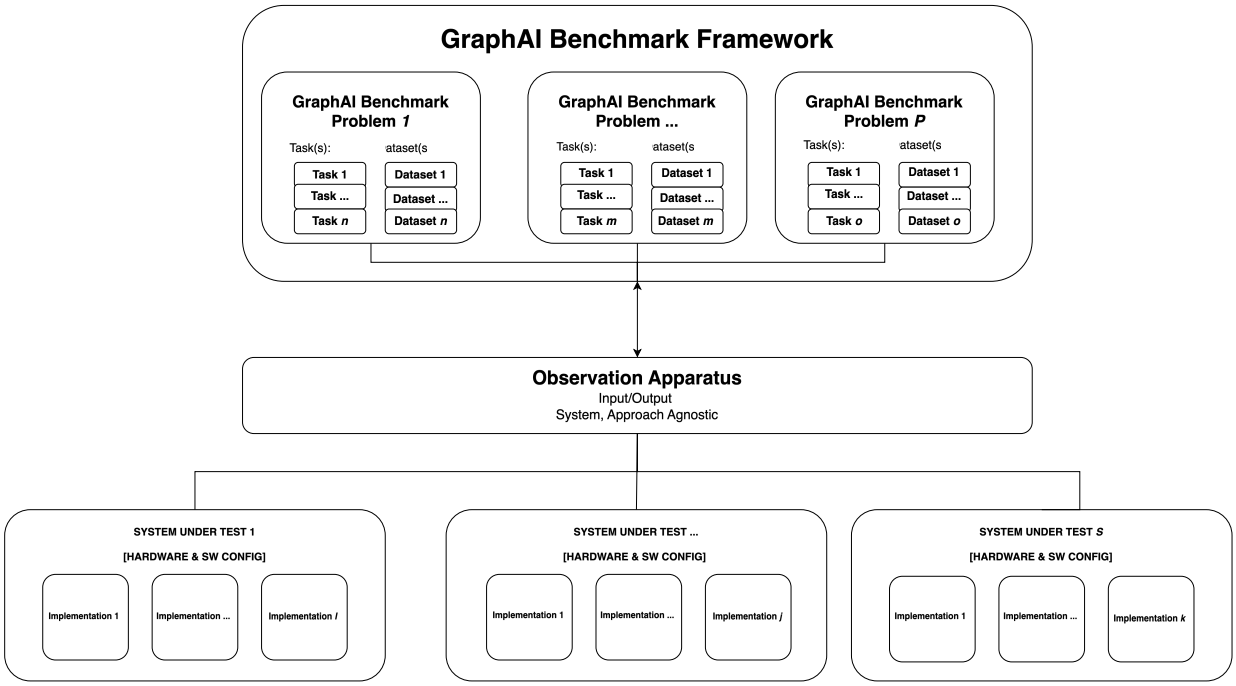


Figure 1.5: The high-level view of the GraphAI benchmarking framework. SUTs interact with the benchmark through an *observation apparatus* API.

1.5 Applications and Impact of the GraphAI Benchmarking Framework

Our GraphAI benchmark framework is a flexible experimentation tool for analysts, engineers, and researchers. Analysts who routinely work with graph data and apply GraphAI to solve

problems can use the GraphAI benchmark to better understand the sizes and shapes of inputs that will impact their work performance and use prior results to build estimates for GraphAI execution time. Engineers can use the GraphAI benchmark to evaluate which systems they should procure and to make architectural decisions about how their system implements GraphAI to maximize the efficiency and effectiveness across the breadth of tasks the system must support. Researchers building new GraphAI algorithms can use the GraphAI benchmark framework to experimentally vary a GraphAI system's hardware and implementation details to find bottlenecks as a function of input datasets and problems. Because the benchmark specification is independent of the SUT, an analyst, engineer, or researcher can iterate quickly, modifying their hardware, software stack, and implementation while easily measuring the corresponding performance changes that inform further work.

1.6 Thesis Outline

The thesis continues with an introduction to the GraphAI taxonomy and key graph and computer architecture concepts in Chapter 2 to identify what makes optimizing GraphAI approaches hard and why measuring relative performance is especially difficult. Then, Chapter 3 presents a detailed review of current GraphAI benchmarking and workload analysis efforts to characterize the current gaps in benchmark coverage of GraphAI tasks, datasets, and metrics and identify what causes performance to degrade in GraphAI systems. Chapter 4 introduces the GraphAI benchmark framework and details our methodology for developing GraphAI problems to constitute the benchmark framework. We use the population segmentation problem from the Intelligence and Security domain to demonstrate how our development methodology is applied,

describe the implementation of our GraphAI benchmark prototype, and outline the findings of our initial experiments in Chapter 5. Finally, Chapter 6 concludes after identifying specific directions for future work.

Chapter 2

Preliminaries

This chapter introduces our GraphAI taxonomy and demonstrates its utility in real-world settings by surveying and mapping I&S use cases to the taxonomy. GraphAI operates on graphs and runs on systems built on computer hardware, so the remainder of the chapter introduces foundational concepts in graphs, graph processing, and computer architectures that interact to affect the efficiency, effectiveness, and cost of GraphAI applications.

2.1 Graph-Based Artificial Intelligence

GraphAI is the collection of intelligent algorithms and machine learning models that learn and reason over information represented in graph data structures. Surveying graph processing, Artificial Intelligence (AI) and Machine Learning (ML) literature shows that many algorithms and models share goals and characteristics. Our GraphAI taxonomy extends prior GraphAI taxonomies [4–6, 11] to include the GPS and GT problem families, and group 'like' tasks together to refine prior, less-structured taxonomies [12][1]. We use our GraphAI taxonomy as a lens to contextualize and extend a survey of I&S problems to show how GM is the most in-demand GraphAI task, informing our choice of case-study in Chapter 5.

[1] https://openbiolink.github.io/ITOExplorer/

2.1.1 A Taxonomy of GraphAI

Our GraphAI taxonomy results from a systematic literature review following an abridged version of Okoli's method [13]. Our survey targeted specific GraphAI tasks by searching textbooks, survey papers, and scientific papers containing the term 'graph' and at least one of the terms 'artificial intelligence' or 'machine learning'. After collecting the tasks, we grouped them thematically by goal into GraphAI problems and then by characteristics into a broader AI category.

The resulting taxonomy divides GraphAI into six overarching "AI Problem" categories (*Graph Problem Solving; Graph Mining; Reasoning; Graph Abstraction; Graph Prediction* and *Graph Transparency*), summarized in Table 2.1. Our survey concludes that a GraphAI system must account for various possible computation patterns, dataset shapes, dataset sizes, and static and dynamic graphs. Chapter 4 explores the challenges of designing a benchmark to measure performance across those variables on the collection of GraphAI tasks surveyed.

AI Problem	GraphAI Problem	Example Tasks
Graph Problem Solving (GPS)	Search	BFS, DFS, UCS, A*, MinMax
Formulate as graph problem [14]	Constraint Satisfaction	Forward Checking, Backtracking
	Planning	GraphPlan, SSSP, MST
	Scheduling	CriticalPath
	Inference	Variable Elimination (Bayes Net)
	Reinforcement Learning	Q-Learning
Graph Mining (GM)	Pattern Matching	Subgraph Matching, Spatial Pattern Matching
Operate on graph topology [12, 15–22]	Characterization	WCC, SCC, LCC, GCC, TC
	Graph Clustering	Community Detection
	Graph Partitioning	Modularity Optimization
	Influence Mapping	PageRank, Belief Propogation, Max Flow
Graph Reasoning (GR)	Knowledge Acquisition	Knowledge Graph Completion, Ontology Learning
Operate on graph knowledge [12, 23–27]	Logic-Based Reasoning	Logical Inference, Ontological Inference
	Representation Reasoning	Tensor Factorization, Semantic Matching
	Neural Reasoning	Relation Inference, Entity Resolution
	Natural Language Processing	Question Answering, Textual Entailment
Graph Abstraction (GR)	Topological Representation Learning	Node2Vec, DeepWalk
Transform for downstream tasks [10, 12, 23, 28–34]	Knowledge Representation Learning	Complex Vector Embeddings
	Graph Generation	Privacy Preservation; Imitative Synthetic Graphs
Graph Prediction (GP)	Graph Classification	Node, Edge & Graph Classification
Learn directly on graphs [12, 35–37]	Graph Completion	Link & Property Prediction, Knowledge Acquisition
	Dynamic Graph Prediction	Traffic Flow Prediction
Graph Transparency (GT)	Grounding	NeuroSymbolic Systems
Explainability & Interpretability [38–47]	Knowledge Structuring	Retrieval Augmented Generation
	Output Interpretation	Post-Hoc Explanation

Table 2.1: The GraphAI taxonomy groups GraphAI tasks into GraphAI problems and broader GraphAI problem areas.

Where: Breadth First Search (BFS), Depth First Search (DFS), Uniform Cost Search (UCS), A-Star Search (A*), Single Source Shortest Path (SSSP), acMST, Weakly Connected Components (WCC), Strongly Connected Components (SCC), Local Clustering Coefficient (LCC), Global Clustering Coefficient (GCC), Triangle Counting (TC)

2.1.2 Real-World Uses of GraphAI

GraphAI is used heavily in real-world problem-solving. To contextualize the GraphAI taxonomy, we extend a recent survey of the problems, datasets, and challenges faced by I&S professionals [48]. Our survey aims to identify specific examples of GraphAI tasks being applied to problems with I&S characteristics. Our survey also followed an abridged systemic literature review process [13], using a set of search terms about the I&S problems ('⟨I&S Problem⟩ ' AND ('Artificial Intelligence' OR 'Machine Learning')) and another set of terms about each task in our GraphAI taxonomy ('⟨GraphAI Task⟩' AND 'Case Study' AND ('Intelligence' OR 'Security' OR 'Defense')). For each paper identified as relevant, we extracted the GraphAI tasks and the typical input data and constraints present in the case study. We grouped them by the specific I&S subdomain they supported.

The survey results are summarized in Table 2.2. The key takeaways from the survey are (i) many of the real-world I&S problems are network problems well suited for the application of, (ii) large dynamic knowledge graphs are a standard input format, and (iii) many use cases have temporal constraints that preclude GraphAI approaches that cannot produce solutions in minutes to hours.

2.2 Graph Definitions

Graphs are the fundamental data structure across GraphAI problems. The underlying graphs drive the efficiency, effectiveness, and cost problems that impact GraphAI tasks. This section defines the components, characteristics, and graph classes for explaining and analyzing GraphAI problems.

AI Problem	I&S Problem	Input Data	GraphAI Approach	Constraints
GPS	Link Analysis [49]	Knowledge Graph	Reachability	Heterogeneity
	Navigation	Spatial graph	Path Planning	SWaP; Complexity
GM	HVT Identification	Multi-Source Graphs	Influence Mapping	Heterogeneity
	Co-Traveller Detection [48, 50–53]	Spatio-Temporal Data	Clustering	Temporal
	POI Identification [16, 54]	Location Data	Spatial Pattern Matching	Complexity
	Demographic Segmentation [55–57]	Communication Logs, Social Media	Community Detection	Temporal
	Counter-Threat Finance [58–62]	Transaction Records, Blockchain	Subgraph Matching	Temporal
GR	Kill-Chain Analysis [63–66]	Event Logs	Logical Inference	Completeness
	All-Source Fusion [48, 67, 68]	Streaming Knowledge Graph	Entity Resoultion	Temporal
GP	Anomaly Detection [69, 70]	Streaming Knowledge Graph	Classification	Temporal
	Link Prediction [71–74]	Streaming Knowledge Graph	Completion	Temporal

Table 2.2: A survey of GraphAI problems in the I&S domain.

2.2.1 Graph Components

Graph components are the building blocks of graphs that persist across representations, abstractions, and programming models.

Definition 0 [Graph (G)]: The data structure comprising of the related vertices and edges such that $G = \{V, E\}$. It is also called a 'network'. $|G|$ represents the size of the graph, and when used in this general sense, it means the sum of the number of vertices and edges.

Definition 1 [Vertex (V)]: Represents an 'entity' or 'thing.' Also called a node. $|V|$ is the number of vertices in the graph (size of the vertex set).

Definition 2 [Edge (E)]: Represents a relationship between two vertices. Also called an arc or a link. $|E|$ represents the number of edges in the graph (size of the edge set).

Definition 3 [Attribute]: A label, property, or other information encoded in a vertex or edge.

For example, a vertex might have a ‘name’ attribute. Graphs with attributes are called attributed graphs.

Definition 4 [Weight]: A numerical value assigned to an edge to describe some real-world value it represents, like the number of citations, length of a road, or cost of an action. Graphs with weights are called weighted graphs.

Definition 5 [Direction]: The flow of information across an edge from the source node to the destination node. Graphs having edges with direction are called directed graphs.

Definition 6 [Multi-Edge]: A special case of a directed edge where more than one edge may exist between a given pair of vertices. Graphs with multi-edges are called multi-graphs and are typical of heterogeneous graphs like knowledge graphs.

Definition 7 [Hyper-Edge]: An edge that connects an arbitrary number of vertices. Graphs with hyper-edges are called hyper-graphs.

2.2.2 Graph Characteristics

Graphs have the following characteristics that define their size and shape, including their topology, degree distribution, and diameter.

Definition 8 [Topology]: The structural characteristics of a graph that emerge when edges link vertices together, regardless of attributes or edge weights. Common topologies include mesh, scale-free, and uniform-random.

Definition 9 [Degree]: The count of edges at a given vertex. Frequently separated into degree$_{out}$ and degree$_{in}$ in directed graphs.

Definition 10 [Average Degree]: The mean degree across all vertices in the graph. A higher mean degree indicates a denser graph.

Definition 11 [Degree Distribution]: The distribution of vertex degrees in a given graph. It can follow any statistical distribution and correlates strongly with known graph processing bottlenecks.

Definition 12 [Neighborhood]: The collection of vertices with edges directly connecting them to a given vertex. The K-hop neighborhood is the collection of vertices within K edges of a given vertex, such as a 'friend of a friend.'

Definition 13 [Diameter]: The shortest path length between the two most distant vertices in a graph, where the number of hops between vertices measures distance. Sparser graphs tend to have larger diameters because there are fewer high-degree nodes to bridge gaps between otherwise distant nodes.

2.2.3 Graph Types

Although graphs can have a variety of characteristics, they can be broadly categorized as simple or heterogeneous and static or dynamic.

Definition 14 [Simple Graph]: The most basic form of a graph. It can be directed, weighted, and attributed, but the attributes are of a consistent type (e.g., all vertex labels are of type 'name').

Definition 15 [Heterogeneous Graph]: A graph with multiple types of vertex, such as a social graph with vertices to represent 'person' and vertices to represent 'organization.' Heterogeneous graphs are more complicated to process than simple graphs because they require type-checking before acting on a given vertex or edge.

Definition 16 [Static Graph]: A graph whose vertices, edges, weights, and attributes are known a priori and do not change. Static graphs are generally simpler to process than dynamic graphs.

Definition 17 [Dynamic Graph]: A graph that evolves with its vertices, edges, attributes, and weights being created, updated, and deleted over time. Also called a streaming graph. Dynamic graphs introduce temporal constraints on graph processing, making many well-researched static approaches unviable.

2.3 Graph Topology

Graphs fit into core classes (scale-free, random, mesh) based on size, shape, topological structure, and construction method. We begin by addressing the classes of graphs that occur in the real world and then discuss synthetically generated graphs, which can be constructed randomly or using a prior distribution.

2.3.1 Real-World Graphs

This section will first identify the 'natural' graph classes derived from real-world phenomena like email records, chat logs, network logs, road maps, protein structures, or any extensive collection of problems that can be represented as a graph.

2.3.1.1 Scale-Free Graphs

Definition 18 [Scale-Free Graph]: A naturally occurring graph class characterized by a distribution of vertex degrees that follow a power-law distribution.

Scale-free graphs are common in social networks because as the number of vertices increases over time, new vertices preferentially attach to vertices that already have high degrees [75]. Scale-free graphs tend to have a small diameter, a significant variance in neighborhood size, and a 'global-neighborhood' characteristic where all vertices are in the same neighborhood within a few hops. These characteristics are often amalgamated and termed the 'small world' property.

2.3.1.2 Mesh Graphs

Definition 19 [Mesh Graph]: A naturally occurring graph class characterized by a flat, regular degree distribution.

Mesh graphs are often derived from physical phenomena like road networks, and so new vertices tend to extend the graph outwards [76]. Mesh graphs tend to have a large diameter, a slight variance in neighborhood size, and a 'local-neighborhood' characteristic where only a small subset of vertices are reachable within a few hops of any given vertex.

2.3.2 Synthetic Graphs

Real-world graphs offer realistic test inputs for graph algorithms. However, some domains need more data to support robust testing, especially regarding performance on irregular degree distributions. As a solution, synthetic graphs can be mathematically or imitatively generated to provide simple graphs for testing and evaluation.

Definition 20 [Synthetic Graph]: A graph that is generated by using a mathematical function or to imitate an existing distribution.

2.3.2.1 Mathematically Generated Synthetic Graphs (Random Graphs)

Multiple mathematical functions can generate synthetic graphs, typically by adding vertices and edges following some set of heuristics to reflect a given probability distribution. Notable generators include the Erdős-Rényi generator to create Uniform Random Graphs [77], and

the Random Matrix (RMAT) generator [78], and Kronecker [79] generator to generate scale-free graphs. Mathematically generated random graphs can approximate real-world graphs but, because of the generation process's structural simplifications provide a sub-optimal substitute for actual graphs' irregularities.

2.3.2.2 Imitatively Generated Synthetic Graphs (Imitative Graphs)

For privacy and confidentiality reasons, many graph datasets are inaccessible for research and development. Active research in privacy preservation and synthetic graph generation seeks to summarize a real-world graph statistically and use summary as the input to a synthetic graph generator so that the synthetic graph will have the same topological properties as the original without the confidential information. Imitative graph generation methods use input features like *Triangle Histograms, K-Hop Neighbor Histograms, Random Walk Histograms*, and *Number of Edges* to produce topologically similar graphs [34]. Their process is inherently lossy and has only been tested on tiny graphs (tens of thousands of edges), so it is unreliable for production use.

2.3.3 Summary

The core graph topology classes discussed above (scale-free, mesh, mathematically generated, imitatively generated) each have inherent benefits and drawbacks for graph processing. We view these classes as representatives of the kinds of datasets relevant to GraphAI . As such, a good GraphAI benchmark should cover each class to enable a longitudinal view of expected performance.

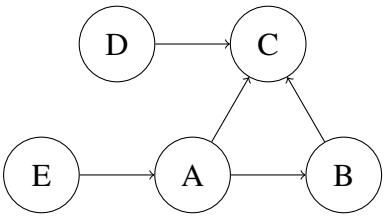


Figure 2.1: Example graph with five nodes and five edges.

2.4 Graph Representations

A graph's representation and abstraction model fundamentally affect how it is processed. The representation format of the input graph is a crucial contributor to the cost, effectiveness, and efficiency of GraphAI tasks operating on graphs. The tradeoff between memory efficiency (how much space a graph consumes) and lookup efficiency (how fast information can be accessed) is a core tension in graph processing. This section introduces fundamental concepts of graph representation, including storage and abstractions as the primary levers used to balance memory consumption and computational efficiency.

2.4.1 Graph Storage

A graph's storage format directly impacts the space it consumes in memory, the operations that can be performed on it, and the speed of those operations. This section discusses the common ways a graph's topological information can be stored and how compression and abstraction models can impact performance. The following examples show the common representations used to store the graph in Figure 2.1.

2.4.1.1 Adjacency Lists

The adjacency list representation (Figure 2.2) uses a linked list or array. The first entry in each line is the source vertex, and each subsequent entry indicates an edge between the source and the listed vertex. Adjacency lists can represent directed or undirected graphs and can represent weights using lists of tuples. Adjacency lists have a small memory footprint ($\mathcal{O}(|V| + |E|)$), but finding a particular vertex is expensive – linear in the number of vertices ($\mathcal{O}(|V|)$) in the worst case.

$A, \quad B, \quad C$

$B, \quad C,$

$C,$

$D, \quad C,$

$E, \quad A,$

Figure 2.2: Adjacency List for graph in Figure 2.1. Each line represents a vertex, with the first in the list being the source and each subsequent vertex a destination of an edge.

2.4.1.2 Adjacency Matrices

An adjacency matrix (Figure 2.3) uses a $|V| \times |V|$ matrix (M) to represent the graph. A 1 at location $M_{i,j}$ indicates the existence of an edge from vertex i to vertex j. Adjacency matrices can represent directed or undirected graphs, and the value at position $M_{i,j}$ can be changed to represent a weighted edge. Adjacency matrices have a memory footprint of $\mathcal{O}(|V|^2)$ but a constant lookup time ($\mathcal{O}(1)$). One of their major drawbacks is the space-inefficient requirement to store all of the

0s where edges do not exist.

$$\begin{bmatrix} & A & B & C & D & E \\ A & 0 & 1 & 1 & 0 & 0 \\ B & 0 & 0 & 1 & 0 & 0 \\ C & 0 & 0 & 0 & 0 & 0 \\ D & 0 & 0 & 1 & 0 & 0 \\ E & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Figure 2.3: Adjacency Matrix for graph in Figure 2.1 is a $|V| \times |V|$ matrix where a value of 1 indicates an edge and 0 is no edge.

2.4.1.3 Incidence Matrices

Incidence Matrices (Figure 2.4) use a $|E| \times |V|$ matrix (M) to represent the graph. An edge is represented with two values: a -1 at position $M_{i,j}$ to indicate the source and a 1 at position $M_{i,k}$ to indicate the destination. Here, i represents the edge's index, and j and k represent the vertices that the edge connects. Incidence matrices are the most expressive representation of those listed here since they can encode multi-graphs and hyper-graphs. Weights can be expressed by modifying the numbers in the matrix. A multi-graph encoding requires inserting source values at $M_{a,j}$ and $M_{a,k}$ and destination values at $M_{b,j}$ and $M_{b,k}$, describing two directed edges going from vertex j to k. A hyper-edge encoding requires placing a source value at $M_{a,j}$ and destination values at $M_{a,k}$, $M_{a,l}$, and $M_{a,m}$, describing a single edge that connects the vertex j to k, l and m. The space complexity of an incidence matrix is $\mathcal{O}(|V| \times |E|)$, and so for sparse graphs, the

incidence matrix format may be more space-efficient than an adjacency matrix, but in general, it will consume significantly more memory where $|E| \gg |V|$. Edge lookup using an incidence matrix is $\mathcal{O}(|E|)$ since it requires scanning the entire edge list for each vertex of interest, which is more expensive than the $\mathcal{O}(|V|)$ edge lookup using adjacency lists.

$$\begin{bmatrix} & e_0 & e_1 & e_2 & e_3 & e_4 \\ A & -1 & 0 & 0 & 1 & -1 \\ B & 1 & -1 & 0 & 0 & 0 \\ C & 0 & 1 & 1 & 0 & 1 \\ D & 0 & 0 & -1 & 0 & 0 \\ E & 0 & 0 & 0 & -1 & 1 \end{bmatrix}$$

Figure 2.4: Incidence Matrix for the graph in Figure 2.1 is a $|E| \times |V|$ matrix where a -1 is the source node and 1 is the destination of a given edge.

2.4.1.4 Compressed representations

Compressed representations aim to balance expressiveness, memory consumption, and computability. The dominant choices are the Compressed Sparse Row (CSR) and Compressed Sparse Column (CSC) formats (Figure 2.5). The CSR format uses three lists to compress the rows of a sparse (adjacency) matrix [80]. The CSC format reverses the CSR format so that the columns are compressed. The space efficiency for CSR and CSC formats is $\mathcal{O}(|V|\log(|M_{nonzero}|) + |M_{nonzero}|\log(|V|))$ for an adjacency matrix M, encoding $|V|$ vertices [80]. CSR and CSC formats take $\mathcal{O}(\log(|M_{nonzero}|))$ time to check the existence of an edge where $|M_{nonzero}|$ is the count

of non zero entries in the matrix. More complicated representations exist that can compress the graph representations even further but they require more computational overhead for the encoding and decoding process [81], precluding them from being used outside of optimized parallel implementations that 'hide' the encode/decode time.

$$\begin{array}{rl} \text{Value List..} & \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix} \\ \text{Row Index.....} & \begin{bmatrix} 0 & 2 & 2 & 3 & 4 & 5 \end{bmatrix} \\ \text{Col Index...} & \begin{bmatrix} 1 & 2 & 2 & 2 & 0 \end{bmatrix} \end{array}$$

Figure 2.5: The CSR format uses three lists to represent the graph in Figure 2.1.

2.4.2 Graph Abstractions

The abstract model of a graph refers to how its representation is simplified for storage and operations. Two dominant graph abstraction paradigms exist: the *Canonical* and the *Linear Algebraic*. The choice of abstraction model directly influences the primitives used, the storage requirements, hardware compatibility, and the effectiveness of acceleration techniques.

2.4.2.1 Canonical

The canonical representation explicitly represents vertices and edges in memory. Operations on canonical graphs need to explicitly 'follow' the edges of a graph, traversing them in turn until target vertices are found. The canonical paradigm is more easily interpretable by humans and is generally used when explaining representations and operations.

To enable more powerful parallel processing methods and hardware accelerators, *vertex-centric*, *edge-centric*, *graph-centric*, and *data-centric* abstractions have been developed.

Vertex-Centric. The vertex is the central unit of data in this abstraction. The abstraction uses iterative operations on vertices, updating their states based on the properties of neighboring vertices [82]. Google's Pregel System [83] and Apache Giraph [84] follow the vertex-centric approach.

Edge-Centric. The edge is the central unit of data in this abstraction. Edges are updated based on the properties of the incident vertices [85]. The X-Stream [86] and GraphX [87] libraries implement the edge-centric approach.

Graph-Centric. The graph itself is the central unit of data in this abstraction. This paradigm explicitly partitions the input graphs into subgraphs for processing [88]. ApacheGiraph++ is a noted adopter of the graph-centric abstraction [89].

Data-Centric. The graph is the central unit of data in this abstraction. It is distinct from the graph-centric model in that rather than explicitly partitioning the graph, it implicitly segments it into 'active' and 'inactive' parts. There is a single graph, but most vertices are ignored at any given time, reducing the computation load and stress on memory. 'Active' nodes are added to a '*frontier*' that allows for more efficient computing by limiting the number of edges or vertices processed at any time. The *Gunrock* framework introduced and continues to champion the data-centric frontier abstraction [90].

These canonical abstractions all have tradeoffs. Vertex and edge-centric abstractions tend to inflict high communication overheads as each vertex and edge are treated individually. Graph-centric abstractions can struggle with optimally partitioning the graphs and load imbalance in the partitions but can reduce communication overheads by abstracting them to subgraphs. Data-

centric abstractions using the frontier have shown significant improvements in execution time, but many implementations cannot support certain classes of algorithms [91].

2.4.2.2 Linear-Algebraic

Linear-algebraic abstractions exploit the ability to represent graphs as sparse matrices using their incidence and adjacency matrices. The Combinatorial Basic Linear Algebra Subprograms (CombBLAS) [92, 93] and Graph Basic Linear Algebra Subprograms (GraphBLAS) [94, 95] are the two most popular linear algebraic graph abstractions. The linear algebraic abstraction exploits the latent coarse-grained parallelism within graphs and builds on advances in high-performance linear algebra processing to improve graph processing [92]. Both abstractions similarly identify that many graph operations can be composed of general and sparse matrix-vector multiplications, so actions like a single step in a Breadth First Search (BFS) can be represented as a single linear-algebraic operation [96]. Linear algebraic representations are robust and offer a promising direction for GraphAI but cannot support certain classes of algorithms, like those leveraging priority queues [92]. They can also be memory-intensive [97] and are less performant than implementations optimized for the hardware they run on [98]. As a result, they have not been widely adopted because of the implicit performance penalty their use incurs.

2.5 Graph Primitive Operations

Primitives are the abstract building blocks that compose more complicated programs. For example, the *Select, Project* and *Join* primitives from relational database processing abstraction are combined in various orders to compose complicated queries and operations on databases.

Having a set of common primitive operations is extremely helpful in building hardware-agnostic applications and gives hardware designers a set of operations to optimize to improve performance. However, there has yet to be an agreement on exactly what the building blocks of graph processing should be [99] since graph operations tend to depend on the abstraction model and implementation used.

The complexity of parallel operations on graphs has driven the development of several primitive families, including the Gather Apply Scatter (GAS) family derived from Map-Reduce [83] and the hierarchical collections of graph primitives built on work-efficient atomic and parallel processing primitives [81, 91, 100, 101]. Graph manipulation primitives have inherited the Create Read Update Delete (CRUD) primitives because of their proximity to databases [102], but are not used in most graph processing approaches. One of the most promising primitive families is matrix operations. Suites like the CombBLAS and the GraphBLAS use a linear-algebraic abstraction of the graph meaning the primitive operations are matrix manipulations that carry out graph operations [92–96].

2.6 Architectural Constraints for Graph Processing: Locality

Modern general-purpose architectures optimize for general-purpose computing. Most computing tasks involve working on a few critical pieces of data for a long time or accessing data close to each other because we tend to store things together that we will need to retrieve together. Graph processing does not resemble general-purpose computing, so the architectural optimizations to capitalize on accessing close-together data and reusing important data have a diminished impact because graphs lack locality.

2.6.1 Locality

Locality is a term in computer architecture referring to the properties of data that enable it to be pre-emptively cached because it is nearby and more likely to be needed than data that is far away (spatial locality) or is necessary and likely to be reused soon (temporal locality). Figure 2.6 is a simplified example of the divergence between topological proximity and memory proximity in graphs at the root of the locality issues.

Spatial Locality. We store related data together, and the idea of optimizing for spatial locality is that when data is needed for processing, data stored near it in memory is also likely to be needed for processing soon so proximal memory blocks can be pre-emptively fetched to the cache for quick access [103]. In Figure 2.6, the vertices are stored in memory in *vertex-order*, that is, written to memory in the order they are created (irrespective of edges).

Storing graphs in vertex-order typically leads to vertices that are topologically close in the graph not being close in memory. In other words, they lack spatial locality. The lack of spatial locality also means that only a tiny percentage of each cache line is used in each step, as irrelevant information is retrieved just because it happens to be nearby. The irregular memory access problem is exacerbated in larger graphs, where the vertex list is more likely to not fit in the cache. In these cases the processor has to access main memory to retrieve the data [104], which is reflected empirically in a decreasing *instructions per miss* rate on larger graphs [76].

Temporal Locality. We tend to transition between related tasks, so data important for what we are doing now is likely necessary for what we do next. Temporal locality is the idea that when a piece of data is needed for processing, it will likely be needed for processing again soon, so it can be retained in the cache for quick access. To exploit *temporal locality*, systems can cache

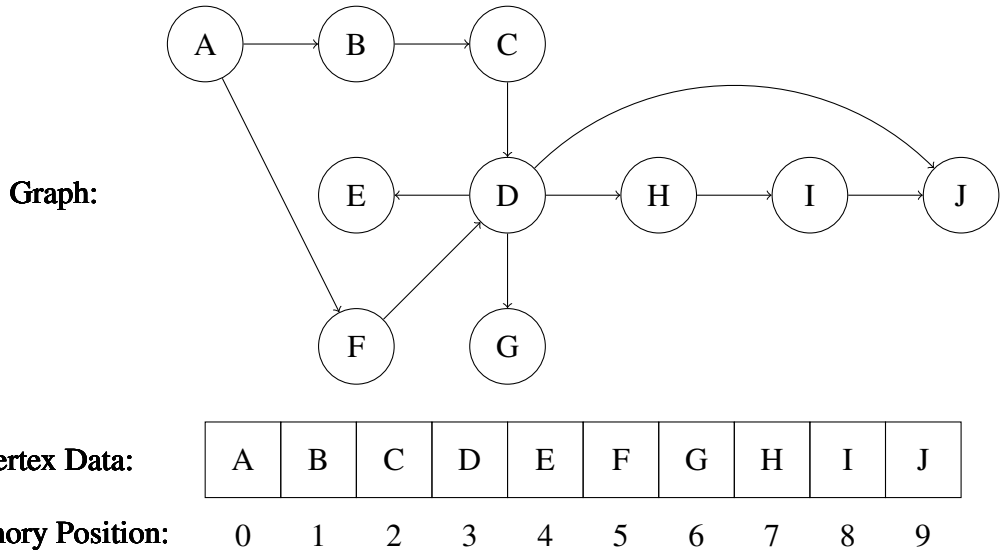


Figure 2.6: A simplified representation of a directed graph in vertex order where each vertex is added to memory creation order (here, alphabetically). Edges are not shown.

high-degree vertices since they are likely to be needed often [76] as is frequently the case for scale-free graphs.

Attributed Graphs and Locality. Attributed graphs exacerbate the latency problem because graph attributes are often stored separately in memory [105]. Figure 2.7 expands on the example from Figure 2.6 by adding vertex attributes. The attributes are stored at separate memory locations, with pointers from the topological vertex (memory positions $1-9$) to their respective attributes (memory positions $10-29$). Attributes consume more memory than topological data, so more bandwidth is required to load them. The heterogeneous data types being loaded in attributed graphs (like knowledge graphs) have been shown to increase the number of Dynamic Random Access Memory (DRAM) accesses, increasing latency and memory bandwidth pressure [106]. To illustrate why, consider a BFS searching for the '**Grape**' vertex. Each time a vertex is dequeued, its attribute must be retrieved from memory to see if it is the target vertex. In the worst case, this requires a memory access each time a vertex is checked, or, if pre-fetching

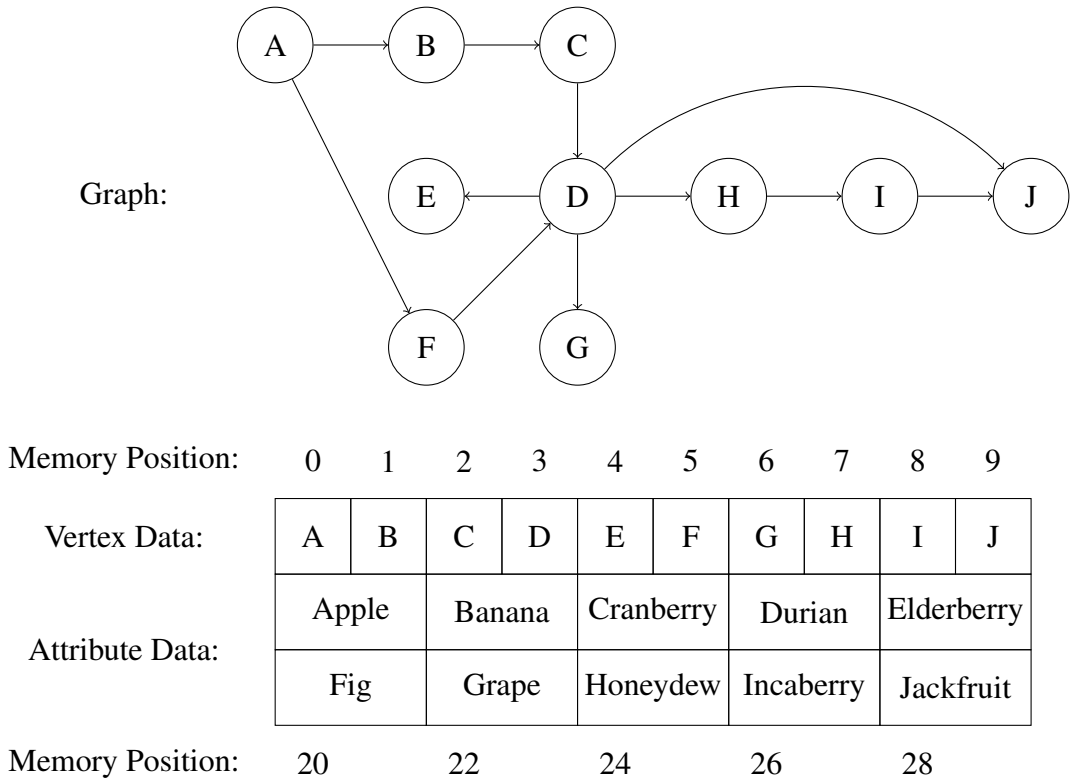


Figure 2.7: A simplified representation of a directed graph in vertex order where each vertex is added to memory creation order with attributes stored separately. Pointers from topology to attributes are not shown for clarity but correspond to the first letter of the attribute.

the attribute along with the vertex, the cache line quickly becomes filled with the vertex plus its attribute, exacerbating the spatial locality problem.

2.7 Summary: Optimizing and Evaluating GraphAI is a Hard Problem

This chapter outlines the breadth of GraphAI problems and identifies the real-world factors that impact their performance. Further, it explains how the size, shape, representation, storage, abstraction, and choice of primitive operations can impact the efficiency and effectiveness of graph operations. These factors combine into the hierarchy of challenges introduced in Figure 1.2 to create an interdependent web of considerations that complicate both the optimization of GraphAI and the evaluation of how different combinations of representations, abstractions, and primitives will interact with algorithmic and hardware acceleration techniques.

Chapter 3

Literature Review

In this chapter, we motivate our methodological approach by reviewing benchmark design from an empirical perspective. First, we discuss the benefits and drawbacks of the two main types of benchmarks. Then, we review the GraphAI benchmark landscape to demonstrate critical gaps in the coverage of GraphAI tasks, datasets, and metrics. Finally, we survey workload analyses and summarize known GraphAI processing bottlenecks to emphasize that current benchmark approaches lack support for evaluating improvements on these bottlenecks. The result of the literature review is our benchmark's design, described in Chapter 4.

3.1 Benchmarking

Benchmarking is a process by which systems or components are compared according to some criteria, determining their relative performance [8]. Benchmarks like SPEC [103], LINPACK [107], and SATLIB [108] have successfully been used to advance research in High Performance Computing (HPC), dynamic programming, and satisfiability.

Benchmarks can be *specification-based*, meaning only input and output requirements are defined, *kit-based*, meaning substantive components are provided, or hybrid, meaning somewhere in between. Specification-based benchmarks provide formal definitions of tasks, and users report their results to them. In contrast, kit-based approaches often have some instrumentation to measure a system's performance. Because of their flexibility, specification-based approaches like

Graph500 [109] are more popular. Whether specification or kit-based, the two main approaches to benchmarking are *competitive* and *white-box* benchmarks.

3.1.1 Competitive 'Black-Box' Benchmarks

Competitive benchmarks are extremely popular in the AI community. They involve selecting a metric or metrics and 'racing' systems as black boxes to identify which system maximizes that metric. They are typically specification-based, and the only factors considered are the results or the time to generate them. For example, in Computer Vision (CV), the Common Objects in COntext (COCO) challenge for object detection uses average precision[1] [110]. VQAv2 uses Visual Question Answering (VQA) accuracy[2] [111], and the Large Language Model (LLM) challenge ChatBot Arena[3] [112] uses 'user preference' as the metric. A recent study of AI benchmarks for CV and Natural Language Processing (NLP) finds that models overfit to the benchmark problems, similar problem areas are over-saturated, and they cluster on an increasingly centralized collection of problems, calling for future benchmarks to focus on versatility, breadth of coverage, and real-world applicability [113].

Competitive benchmarks help get a quick overview of the current State Of The Art (SOTA) but have three significant drawbacks. First, they provide a 'black box' view of the systems, making it difficult to tell why one system performed better than another or how to improve them [1, 114]. Second, the 'leaderboard' style of challenge encourages optimizing for the benchmark rather than progressing the knowledge of the field [2]. The third drawback is that competitive benchmarks emphasize effectiveness measures, often ignoring efficiency altogether. The

[1] https://cocodataset.org/#detection-leaderboard
[2] https://visualqa.org/roe.html
[3] https://huggingface.co/spaces/lmsys/chatbot-arena-leaderboard

problem with over-emphasizing effectiveness is the possibility of missing the broader picture and rewarding an inefficient algorithm that that uses more resources to brute-force a slightly better result than an efficient algorithm that uses fewer resources.

3.1.2 'White-Box' Benchmarks

'White-Box' benchmarks take the opposite approach to competitive benchmarks, instead focusing on identifying bottlenecks and providing a platform for further research [1, 2]. 'White-Box' benchmarks follow either the workload analysis or the hosted benchmark paradigm.

Workload Analyses. Workload Analyses are detailed white-box studies of the internal states of a system that is undertaking a series of tasks on a collection of datasets. They focus on more detailed measurements like memory usage, network bandwidth, and processor measurements. Workload analyses are invasive and require complete access to the SUT to instrument the implementation and profile the environment during execution. Reports are typically generated manually, focusing on a single system or comparing a few systems. Although workload analyses provide a detailed understanding of a system's behavior, they are work-intensive and do not scale well.

Hosted Benchmarks. Hosted benchmarks provide a shared environment where implementations can be tested under fixed conditions. Third-party entities typically maintain hosted benchmarks that accept submissions, run them, and report the results. The hosted approach relies on shared problem formats and common primitive operations. These limitations constrain developers to implementations that can run on the provided environment. For example, the StarExec project for logic solvers [115] allows users to submit implementations and publish the results to

public leaderboards, but only if their approach meets the requirements to run on the StarExec cluster. The salient limitation of a hosted benchmark is that it prevents comparisons of different hardware configurations; it only allows the comparison of implementations that must be compatible with the host platform. For graph and GraphAI problems where many of the available acceleration techniques use hardware and representation changes, a hosted benchmark is rarely suitable for an evaluation that reflects real-world conditions.

3.1.3 Constrained Benchmarks

Constrained benchmarks are a promising alternative to competitive and hosted benchmarks. A constrained benchmark limits the time or resources available to competitors while requiring them to maximize an effectiveness metric. For example, the *efficientQA* Efficient Open Domain Question Answering Competition required competitors to maximize Question Answering Accuracy while adhering to strict memory budgets, forcing trade-offs between the size of the model and the size of the reference corpora [116]. A constrained benchmarking approach shows how, even without explicit access to the internal state of a system required for an 'white-box' benchmark, we can still achieve a more thorough comparison than a simple competitive 'race' for effectiveness.

3.1.4 Summary

Current benchmarking approaches give us three paradigms. Competitive benchmarks are flexible, accessible computational competitions that tell us which system is the fastest or the most accurate but not why. 'White-box' benchmarks are labor-intensive and impose constraints

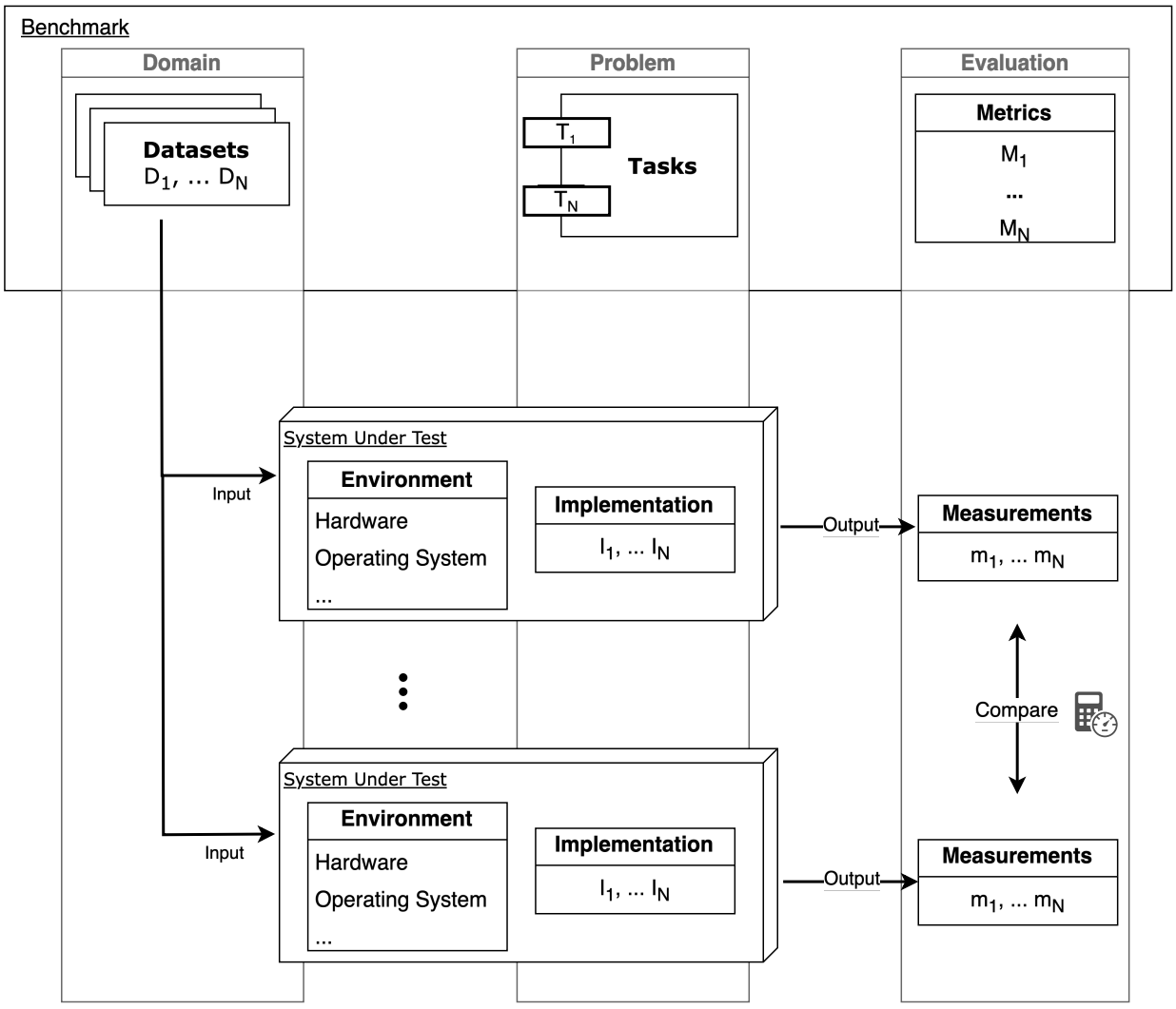


Figure 3.1: The basic components of a generic benchmark system.

on the systems being studied. While constrained benchmarks take a alternative approach, they focus on only one dimension during evaluation. Constrained benchmarks show how careful manipulation of the input data and operating environment can stimulate innovation by researchers and interesting behavior in the SUT. The approach we present in Chapter 4 proposes combining the accessibility of competitive benchmarks with the informativeness of a formal 'white-box' benchmark by leveraging the ideas from constrained based benchmarks about how to stimulate behavior in the SUT in a 'grey-box' approach to the benchmarking problem.

Regardless of the specific benchmarking approach, all benchmarks feature a specification for the *datasets, tasks* and *metrics*, and are expected to evaluate a SUT that comprises of some hardware and software stack and the specific implementation that will solve the specified tasks on the provided datasets and be monitored to generate the target metrics. When the benchmark is hosted, the hardware and software stack are frozen, and only the implementation is measured. These basic components in Figure 3.1 are explained in more detail in Section 4.2. The following section examines GraphAI -specific approaches to benchmarking, identifying how these paradigms are used and their limitations in practice.

3.2 Benchmarking GraphAI

Most of the influential GraphAI benchmarks are specification-based competitive benchmarks. However, providing specific real-world datasets and reference implementations typical of kit-based benchmarks is standard practice. From the conclusions in Section 3.1 and the benchmarks discussed in the present section, we identify three core abstract components of a benchmark.

The three abstract components that form the benchmark specification are *datasests*, *tasks* and *metrics*. We use these components as the focal points for the literature review of 23 prior benchmarks in graph analytics, Graph Database (GDB), and graph neural networks Graph Neural Network (GNN) to identify gaps that a GraphAI benchmark should cover.

Focus Area	Benchmark	Number of Tasks	Extract Transform Load (ETL)	GPS	GM	GR	GA	GP	GT
Graphs	Graph500 [109]	3	✓	✓					
	$PBBS_{1.0}$ [117]	7		✓					
	SNAP [118]	19	✓	✓	✓				
	$Graphalytics_{0.1}$ [119]	5	✓	✓	✓				
	$Graphalytics_{1.0}$ [120]	6		✓	✓				
	GAP [121]	5		✓	✓				
	SAGA-Bench [122]	5	✓	✓					
	GBBS [101, 123]	26		✓	✓				
	$PBBS_{2.0}$ [124]	6		✓					
	$Graphalytics_{1.2}$ [125]	6		✓	✓				
GDBs	SRBench [126]	3	✓	✓		✓			
	SocialGraph [127]	5	✓	✓					
	SocialBench [128]	6	✓	✓					
	TEKG [129]	4	✓	✓					
	BenchGraph [130, 131]	5	✓	✓					
GNNs	OGB [132]	4					✓	✓	
	PAGNNF† [133]	2						✓	
	GNNMark [36]	5						✓	
	Bridging-GNN† [134]	2‡						✓	
	LRGB [135]	5						✓	
	BGNN [136]	4						✓	
	CogDL [35]	8					✓	✓	
	OpenGSL [137]	1						✓	

Table 3.1: Coverage of GraphAI tasks by existing benchmark suites and workload analyses.

†=Workload analysis. ‡=Inferred, not specified.

3.2.1 Task Coverage.

A benchmark task is a specific action that needs to be taken or a problem that needs to be solved; it specifies what the system must do for the benchmark to run. We identify 23 graph benchmarks that satisfy at least one category from our GraphAI taxonomy. The reported tasks cluster around traditional graph tasks, GDB-specific tasks, or GNN prediction tasks, depending on the focus area of the benchmark.

In Table 3.1, benchmarks are shown grouped according to their focus area – traditional graph analytics (Graphs), GDB, or GNNs. For each benchmark surveyed, we list the total number of tasks and whether at least one task covers each problem area in our GraphAI taxonomy. In addition to those categorizations, we add a column called ETL to reflect whether the benchmark measures the critical ETL operations for GDBs and dynamic graph processing. The coverage of the ETL column suggests graph manipulations are often scoped out of traditional graph and GNN benchmarks because of their focus on static graph processing and the associated assumption of ETL cost amortization. We identify two key takeaways about the task coverage of current GraphAI benchmarks based on the information collected in Table 3.1.

Tasks focus on specific operations rather than general problems. In total, we identified 62 distinct GraphAI tasks by reviewing benchmark specifications, published results, and public code repositories, suggesting that a wide variety of task coverage exists in the literature, just not all in one benchmark. The benchmark with the broadest task coverage, the Graph-Based Benchmark Suite (GBBS) [101], has 26 tasks, but the median coverage across all benchmarks is 5 tasks, meaning most of the benchmarks tend to focus on a specific sub-area within a given GraphAI problem. Few benchmarks cover more than two of the seven GraphAI problem areas,

and the areas cluster around 'like' problems. The focus on comparing specific tasks drives the development of those tasks, which in turn drives the development of benchmarks to compare those tasks. Reviewing hardware accelerator designs emphasizes that systems are being built to focus on accelerating these 'narrow' benchmark kernel-tasks[4], rather than providing a more holistic real-world view of the problem.

In contrast, we propose a system-agnostic, problem-centric approach so that systems completing the benchmark must solve the problem rather than complete a narrowly specified kernel-task in a shallow facsimile of the real world [103]. Part of the motivation for this choice is to compose more complicated problems out of a collection of simple problems to reflect better the composite processing requirements expected in real-world settings [48].

ETL is under-represented in benchmarks given their importance in real-world GraphAI processing. Only nine of the surveyed benchmarks explicitly include tasks that require ETL, reflecting the problematic assumption that graphs are static, meaning the construction cost can be amortized over the life of the graph. As our overview of GraphAI problems in the real world from Section 2.1 emphasizes, many problems are temporally constrained by the dynamic nature of real-world data. As a result, the limited coverage of graph manipulations makes it difficult to generalize their results to dynamic graph processing. Furthermore, only three benchmarks (SAGA-Bench [122], SRBench [126], and PAGNFF [133]) provide explicit support to dynamic graph tasks. The benchmarking framework we propose in Chapter 4 explicitly includes ETL time in every benchmark problem to permit comparison between dynamic and static approaches to the same problem and to better reflect the requirements of the real world where shared infrastructure

[4]In benchmarking, the most minor measurable 'thing' is an *atomic operation* (e.g., traversing an edge). Multiple atomic operations compose into a *kernel* (e.g., BFS), which can compose into *toy, synthetic* or *real* programs, which implement kernels to solve some higher level problem at varying levels of scale and realism [114]

may need to load different graphs at different times for different GraphAI tasks. We create a benchmark that can accommodate static or dynamic graph processing approaches by tracking the BL, the time between when ETL begins on a datum and when it shows up in the results of a query of operation.

Focus Area	Benchmark	Distinct Datasets	Max Edges	Attributed	Directed	Weighted	Graph Types	Data Origin	Input Format
Graphs	Graph500 [109]	6	70T			(O)	SF	S_{UG}	AL
	$PBBS_{1.0}$ [117]	6	112M		✓	(O)	R,SF	S_{UG}	AL
	SNAP [118]	126	1.8B	✓	✓	✓	R,SF,M,KG	R	AL
	$Graphalytics_{0.1}$ [119]	6	5B		✓	✓	R,SF	S_{UG}	AL
	$Graphalytics_{1.0}$ [120]	37	17B		✓	✓	R,SF	S_{PB}	AL
	GAP [121]	5	2.1B		✓	✓	R,SF,M	RW,S_{PB}	AL
	SAGA-Bench [122]	5	500M		✓		R,SF,D	RW,S_{PB}	AL,HM
	GBBS [101, 123]	0	N/A	N/A	N/A	N/A	N/A	N/A	N/A
	$PBBS_{2.0}$ [124]	6	112M		✓	(O)	R,SF	S_{UG}	AL
	$Graphalytics_{1.2}$ [125]	37	17B		✓	✓	R,SF	RW,S_{PB}	AL
GDBs	SRBench [126]	3	1.76B	✓	✓		SF,KG, Dyn	RW	RDF
	SocialGraph [127]	5	161M		✓		SF,KG	S_{UG}	RDF
	SocialBench [128]	4	20B		✓		SF,KG	S_{UG}	RDF
	TEKG [129]	1	1T		✓		SF,KG	S_{UG}	RDF
	BenchGraph [130, 131]	9	52M		✓		SF,KG	S_{PB}	Cypher
GNNs	OGB [132]	15	1.6B	✓	✓		SF,KG	RW	SFF
	PAGNNF† [133]	5	44.4K	✓	✓		SF,M,D	RW	SFF
	GNNMark [36]	12	1.9M	✓	✓		SF,M	RW	SFF
	Bridging-GNN† [134]	8	21K	✓	✓		SF,M	RW	SFF
	LRGB [135]	8	2.3M	✓	✓		SF,M	RW,S_{PB}	SFF
	BGNN [136]	5	2693K	✓	✓		SF,M	RW,S_{PB}	SFF
	CogDL [35]	81	132M	✓	✓		SF,M	RW	SFF
	OpenGSL [137]	10	2.24M	✓	✓		SF,M	RW	✓

Table 3.2: Coverage of GraphAI datasets by existing benchmark suites and workload analyses.

O= Optional, **SF**=Scale-Free, **R**=Random, **M**=Mesh, **KG**=Knowledge Graph, **D**= Dynamic, **RW**= Real-World, $\mathbf{S}_{PB}$= Pre-Built Synthetic, $\mathbf{S}_{UG}$= User-Generated Synthetic, †=Workload Analysis.

3.2.2 Dataset Coverage.

Datasets are a core component of benchmarks. They are the 'thing' or 'problem' on which a system must conduct a task. A collection of standard datasets makes the results between different systems comparable. Since GraphAI performance is dependent on the size and shape of the input graph, good performance measurements require broad coverage of graph shapes from the domain of interest. We surveyed the datasets used in 23 graph benchmarks, finding that the median number of input graphs is 6 per benchmark and that there is a general focus on real-world scale-free graphs. The benchmarks are grouped by their focus area (graph analytics, GDB, GNNs) in Table 3.2 and the column entries indicate whether each benchmark satisfies a given graph property with at least one of the datasets it contains. We identify three areas of concern in dataset coverage for GraphAI benchmarks.

Heterogeneous graphs are underrepresented. Real-world GraphAI problems require processing heterogeneous graphs such as knowledge graphs [48]. In our survey, heterogeneous graphs were sparsely covered in all focus areas except GDBs. Traditional graph analysis and GNNs both focus on topological features, leading to a scarcity of complex attributed graphs in benchmark datasets.

Benchmark datasets vary in domain coverage rather than graph topology. The median number of datasets benchmarks use is 6, with most selections being loosely justified and not explicitly tied to any domain. The benchmarks we surveyed tend to capture publicly available graphs about graph-native problems like mapping citation networks, analyzing friend networks on social media, and email events. However, numerous graphs are available between Stanford Network Analysis Project (SNAP)'s SLNDC [21] and the Network Data Repository (NDR) [138]

in a variety of other domains like health, physical sciences, geospatial sciences that could be included. Only the Graph Algorithm Platform (GAP) [121] benchmark explicitly chooses its datasets to represent mesh, scale-free, and uniform random topologies to enable performance measurements across varied input shapes. For our approach, we will build on the GAP methodology and aim for datasets from at least the scale-free, random, and mesh structures for every problem in the framework to stress SUTs across all input shapes.

Dynamic graph datasets are very uncommon. Dynamic graphs have spatio-temporal characteristics that are not common in static graphs. Edges and vertices can be added or removed at any point, and the graph must conduct ETL concurrently with completing GraphAI tasks. These dynamic approaches focus attention on providing results within a limited timeframe. The lack of dynamic graph datasets means there is no clear drive to build and evaluate dynamic GraphAI systems, so the development of time and resource-constrained GraphAI approaches is slow. Because of the lack of available datasets, research on dynamic graphs needs to transform static graphs into dynamic ones through random sampling. However, many of the same drawbacks of synthetic graph generation apply to an artificial sampling of static graphs. For example, random sampling does not replicate the natural pattern of vertex and edge creation, modification, and deletion inherent to dynamic graphs.

Focus Area	Benchmark	*Effectiveness*	*ETL-Time*	*Exec-Time*	*Throughput*	*Compute*	*Memory*	*Network*	*Power*
Graphs	Graph500 [109]		✓	✓	✓				
	$PBBS_{1.0}$ [117]			✓					
	SNAP [118]			✓					
	$Graphalytics_{0.1}$ [119]			✓	✓				
	$Graphalytics_{1.0}$ [120]			✓	✓				
	GAP [121]		✓	✓					
	SAGA-Bench [122]		✓	✓	✓		✓		
	GBBS [101, 123]			✓					
	$PBBS_{2.0}$ [124]			✓					
	$Graphalytics_{1.2}$ [125]			✓	✓				
GDBs	SRBench [126]	✓	✓	✓	✓				
	SocialGraph [127]			✓					
	SocialBench [128]			✓					
	TEKG [129]			✓					
	BenchGraph [130, 131]			✓			✓		
GNNs	OGB [132]	✓					✓		
	PAGNNF† [133]		✓	✓		✓	✓		
	GNNMark [36]			✓			✓		
	Bridging-GNN† [134]			✓			✓		
	LRGB [135]	✓							
	BGNN [136]	✓		✓	✓				
	CogDL [35]	✓		✓					
	OpenGSL [137]	✓		✓			✓		

Table 3.3: Coverage of GraphAI metrics by existing benchmark suites and workload analyses.

†=Workload Analysis.

3.2.3 Metric Coverage.

Metrics are the summaries of a system's efficiency, effectiveness, and cost, which we generate by observing how a system solves a task on a given dataset. The metrics generated by GraphAI benchmarks enable comparison of the relative effectiveness, efficiency, and cost of GraphAI systems. We survey the coverage metrics of 23 graph benchmarks from published results and public leaderboards in Table 3.3. We group benchmarks by their focus areas (graph analytics, GDBs and GNNs) and indicate whether each benchmark reports a given metric in at least one of its workloads. We identify four areas of concern in the coverage of metrics by existing benchmarks.

Collecting detailed metrics is invasive and impractical. To collect interesting and useful metrics about the internal state of the GraphAI system (compute, memory, network, and power), the SUT needs to be under the control of the benchmark owner so that the code can be correctly instrumented and the underlying system sampled throughout execution. As a result, these 'internal' metrics are only observed in detailed workload analysis studies where the researcher has access to the system. In the remainder of cases, execution time is typically the only viable metric. The Input/Output model for benchmarking is flexible but does not have access to the system's internal state, necessitating creative methods to infer the system's internal state. In the GraphAI benchmark framework, we fuse an empirical approach to algorithm analysis with constraint theory to balance the benefit of a detailed workload analysis with the flexibility offered by a simple Input/Output design.

Throughput measures can be misleading. Throughput metrics are generally estimates of Traversed Edges per Second (TEPS), which are computed by counting the edges in the input

graph, estimating the traversals required by a single iteration of the algorithm, and multiplying that by the number of iterations to get the total edge traversals, which is then divided by the execution time to measure TEPS [139, 140]. The drawback of this estimation strategy is that it performs poorly when algorithms aggregate or otherwise change the graph's structure at each iteration. In the GraphAI benchmarking framework, we focus on execution time as a downstream indicator of throughput and the interaction of throughput with the other aspects of the system.

Effectiveness is neglected. Benchmarks measure performance by efficiency metrics or effectiveness metrics, but not both. Few of the surveyed benchmarks reported effectiveness metrics, likely because many 'traditional' graph problems are deterministic with a single 'correct' solution. On the other hand, benchmarks for newer problems like Long Range Graph Learning [135] often only include effectiveness metrics, ignoring time and space efficiency until the problem can be effectively solved. The GraphAI framework takes a more nuanced view of correctness, linked to defining problems generally rather than as specific tasks and using the principles of constraint-based benchmarking. We intend to develop problems with degrees of correctness to encourage work on iterative, incremental, and approximate solutions that may offer a trade-off between effectiveness and efficiency. The ability to perform this trade-off gives a benchmark user option to experiment with their constraints to find the best solution.

Cost measurement requires system access. Cost measurements estimate a particular solution's cost and form the third dimension of metrics. However, none of the surveyed benchmarks estimate cost, and few acknowledge its proxies like power consumption. A complete system constraints model must balance efficiency, effectiveness, and cost. For now, our GraphAI benchmark framework focuses on effectiveness and efficiency because the lack of methods for non-invasive cost estimation prevents accurate cost calculations. Adding support for cost calculations is a

high-priority area of future work.

3.3 GraphAI Bottlenecks

A *bottleneck*, or *system constraint* is any resource whose capacity is less than or equal to the demand placed on it [141]. Designing tasks that will discriminate GraphAI system performance to create a 'grey-box' understanding of the system state requires understanding the common bottlenecks in GraphAI systems. This section summarizes the primary GraphAI bottlenecks commonly identified in GraphAI workload analyses– *memory, compute* and *communication* bottlenecks.

3.3.1 Memory Bottlenecks

Memory bottlenecks are when the maximum capacity of a system is limited by the speed of reading from and writing to memory. Memory bottlenecks occur when a system cannot keep a processor fully occupied because the data required for computation cannot be moved from memory to the processor quickly enough [103]. They occur in graph processing because of the lack of locality and consequent underutilization of the cache hierarchy described in Section 2.6. Subproblems contributing to memory bottlenecks are memory bandwidth utilization and memory latency. Memory bandwidth defines how much can concurrently be moved to and from memory. It becomes the bottleneck when there is poor cache utilization [142], competing parallel processes [143, 144] or irregular memory access patterns [145–147] causing the processor to stall because not enough data is getting to it [99]. Memory latency defines how long it takes to retrieve required data from memory and exacerbates the under-utilization of memory bandwidth [133, 148]. Latency is problematic in GraphAI applications because the irregular data

accesses coming from the inherent lack of spatial and temporal locality [149, 150] cause cache misses [76, 146, 147] and inefficient cache usage [145, 150, 151]. These caching issues result in slow main memory accesses that even optimized pipelined architectures cannot 'hide' [152]. The GraphAI benchmark framework will use input data to trigger memory bottlenecks. Graphs with high vertex counts and high (but regularly distributed) degrees will induce the pointer-chasing behavior that forces memory bottlenecks to manifest.

3.3.2 Computation Bottlenecks

Computation bottlenecks occur when the system can serve data to the processor faster than operations can be performed on them. Optimizations for general-purpose architectures fail to deliver many benefits because of the irregular data-dependent executions in graph processing [153]. For example, branch misprediction is common [145], and distributed workloads frequently become unbalanced [145, 154].

Looking at GNN, they have hybrid computing patterns that combine irregular graph processing and regular dense neural network processing. As a result, they suffer from both memory and computation bottlenecks and are challenging to implement efficiently on general-purpose architectures [155]. To mitigate the impact of hybrid computing patterns, GNNs break algorithms into collections of the GAS primitives to take advantage of distributed and accelerated computing built for that abstraction model. For the GraphAI benchmark, we implement an Input/Output benchmark design so that a benchmark user can bring any arbitrary system configuration to test, trialing different hardware accelerators and system configurations to try and elevate the computation bottleneck.

3.3.3 Communication Bottlenecks

Communication bottlenecks occur when a system lacks the bandwidth to transfer data between workers, or communication latency makes frequent communications impractical. They are caused by transferring large amounts of data and communication-intensive algorithms that require a lot of network traffic to stay synchronized. For graph processing, communication bottlenecks occur in parallel computing settings or when an external hardware accelerator is used. These are caused by separation, movement, and synchronization overheads. Parallel processing requires the data to be divided up between workers. Partitioning graphs is a known NP-complete problem, and the time taken to partition graphs for processing significantly impacts overall efficiency [82]. When partitioned, the graph must be copied to a distributed worker node (or an accelerator), and the time taken to copy the data for processing degrades efficiency [156, 157]. Synchronizing parallel worker nodes is challenging when the algorithm needs to access all vertices each iteration [99] or when the graph partitioning between workers is suboptimal [158]. Distributed parallel approaches incur both penalties, resulting in severe communication bottlenecks [76, 101].

3.4 Summary: GraphAI benchmarking requires a 'grey-box' approach

This chapter has described prior work in benchmarking GraphAI systems. It identifies the requirement to take a 'grey-box' approach to estimate the internal state of a system so that the general utility of a black-box competitive benchmark balances the in-depth analytical power of the invasive white-box workload analysis. Prior efforts to evaluate GraphAI approaches focus on subsets of abstract kernel-level tasks, do not have adequate coverage of the varieties of graph

dataset sizes and shapes, and neglect the trade-offs between efficiency, effectiveness, and cost. Of the three types of bottlenecks, memory bottlenecks are the most impactful for GraphAI , with communication bottlenecks appearing for distributed and dynamic GraphAI approaches. To implement the 'grey-box' approach, the GraphAI benchmark framework adopts an Input/Output paradigm that combines empirical algorithmic analysis with constraint theory to design the combinations of tasks and inputs that we expect to trigger system bottlenecks while giving users the flexibility to modify their SUT without the need for burdensome reinstrumentation or adherence to environmental restrictions.

Chapter 4

The Graph Artificial Intelligence Benchmarking Framework

In this chapter, we present our GraphAI benchmarking methodology that addresses the lack of domain-specific task, dataset, and metric coverage in prior work.

4.1 Motivation

In reviewing prior work in competitive benchmarking and workload analysis (section 3.1), we conclude that purely competitive 'black-box' benchmarks from the Input/Output paradigm lack descriptive power and that workload analyses from the 'white-box' paradigm are too invasive for general-purpose use. The review of GraphAI benchmarking (section 3.2) also concludes that existing GraphAI task, dataset, and metric coverage is narrow, neglects dynamic processing of heterogeneous graphs and fails to evaluate approximate and incremental solutions appropriately. Our 'grey-box' approach addresses these issues with an Input/Output model that is agnostic to the configuration of the SUT. It fuses the empirical study of algorithms with constraint theory to design problem-centric tasks and has a broad coverage of datasets that stimulate known system bottlenecks. Our approach uses a nuanced view of efficiency and effectiveness to compare static and dynamic approaches in the same framework.

Problem-centric tasks and broad dataset coverage are essential components of the GraphAI benchmark framework design. We follow a problem-centric design to specify tasks by the inputs and outputs rather than the typical system-centric abstract kernel approach that specifies which pro-

cess should be applied to generate a solution. The problem-centric focus mimics real-world problems and builds in ETL and multi-step composite problems beyond prior work's focus on simple kernel-tasks. The GraphAI framework offers a powerful tool to infer the system's internal state without needing 'white-box' access by combining the problem-centric focus with datasets designed to stimulate known system bottlenecks and algorithmic weaknesses. Furthermore, by monitoring the effectiveness of results produced by the SUT over time and designing problems to have answers with degrees of correctness, our benchmark framework can compare deterministic, non-deterministic, and approximate approaches of both static and dynamic graph processing methodologies in a single framework. The breadth of coverage allows a benchmark user to experiment with many different approaches, explore trading effectiveness for efficiency, and experiment with working under efficiency, effectiveness, and cost constraints.

4.2 Benchmark Mechanics

This section briefly extends the literature review from Chapter 3 by reviewing specific work in benchmark design and construction to identify guiding principles for constructing our benchmark and conducting experimentation.

A benchmark designer must consider the purpose of the benchmark, general principles of benchmark design, and the granularity of the problem [159]. The purpose of the benchmark defines how a designer scopes the benchmark and applies the four core principles of benchmark design (*relevance, fairness, reproduciblity* and *verifiability*) [8, 160]. The purpose of the benchmark also drives its level of granularity. From most to least granular, benchmarks of *atomic operations, kernels, toy programs, synthetic programs* and *real programs* provide different information about

the systems under test and the metrics that describe them [103]. The GraphAI benchmark design follows the principles of benchmark design. It extends beyond the popular kernel-only benchmarks to balance understanding the performance of individual components with the system's performance as a whole.

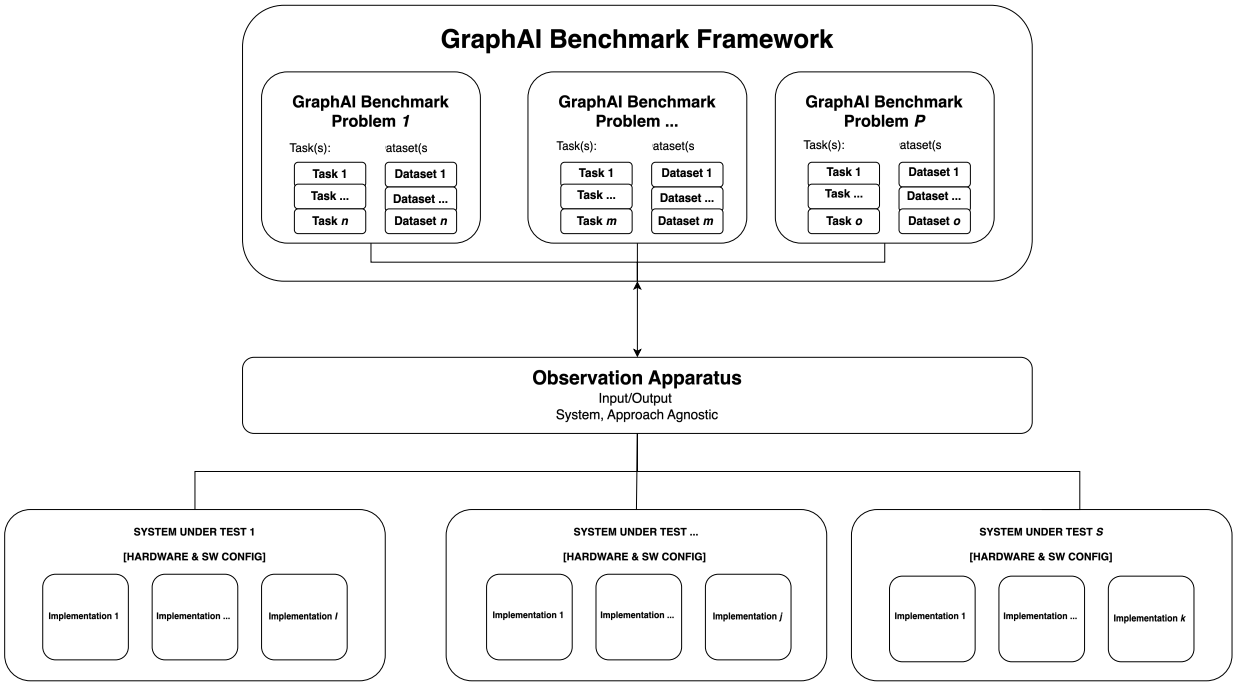


Figure 4.1: The components of the GraphAI benchmarking framework.

Figure 4.3 shows the high-level components of the GraphAI benchmark framework: the abstract *benchmark specification* in the top half of the Figure and the concrete (SUT) in the bottom half. These two components interact, and those interactions are what the benchmark measures using our *observation apparatus* that acts as an intermediary Application Programming Interface (API) between the benchmark specification and SUT.

4.2.1 Benchmark Specification

The benchmark specification is the main component of the GraphAI benchmark framework. The framework is a collection of problem-specific benchmarks, each of which has its own *datasets* drawn from a domain of interest, *tasks* drawn from a relevant problem space, and *metrics* that are tailored to reflect the specific evaluation criteria. More specifically:

Datasets are the challenges that a benchmark needs to 'solve.' Source domains provide many datasets that become input to a SUT, which can be real or synthetic. The SUT receives the dataset (the 'challenge') as input and auxiliary information like ground-truth answers for evaluation.

Tasks are the specific processes a benchmark needs to complete as part of the evaluation process. They are derived from the problem space and define a specific problem to be solved. A task is a blueprint for an implementation. Together, a dataset and a task comprise a *workload*.

Metrics are how we define, collect, and compute the observable interactions of a SUT and a workload. Metrics are defined abstractly as evaluation criteria, collected from the outputs of the SUT as measurements, and then compared to develop a relative performance model between systems. The most common metrics describe the efficiency, effectiveness, and cost of running a given workload on a SUT.

4.2.2 System Under Test

The SUT by the benchmark comprises of the *environment*, which consists of the *hardware* and underlying *software stack* as well as the *implementations*, which are the specific programs written to execute the workloads required by the benchmark specification. Figure 3.1 shows how

the exact benchmark specification can have many SUTs complete the same tasks on the same datasets to generate the same metrics for comparison.

Hardware is the environment component referring to all the physical components of the SUT, including host machines, hardware accelerators, or distributed systems. Hardware is the most commonly studied component of the SUT in benchmarks and workload analyses and is a popular target for acceleration. Identifying under what circumstances a hardware accelerator is justified is a valuable result of this benchmarking framework.

The **Software Stack** refers to all the software components that are not directly part of the implementation, including the operating system and enabling tools like a database where the focus of the evaluation is not on the database itself.

The **Implementation** is how the SUT approaches a given task. Implementations consist of the abstraction model, data structures, algorithms, and programming language used to solve a given GraphAI problem. One SUT can have many implementations.

The SUT is ultimately the target of the evaluation, and the benchmark specification defines what the SUT needs to do to complete the benchmarking process. The distinction between the specification and the SUT is clouded in kit-based benchmarks that provide reference implementations as baselines of performance or host-based benchmarks that expect all implementations to run on provided hardware.

4.2.3 Grey-Box Evaluation

Experimentation using benchmark suites involves modifying the benchmark specification or changing the SUT. Understanding the likely consequences of varying a benchmark component

Varied Component		Dependent Metric		
Dimension	Components	Effectiveness	Efficiency	Cost
Specification	Datasets	✓	✓	✓
	Tasks	✓	✓	✓
	Metrics			
System under Test	Hardware		✓	✓
	Software		✓	✓
	Implementation	✓	✓	✓

Table 4.1: Summary of dependencies between benchmark components and evaluation metrics.

is essential to our 'grey-box' design, and is part of how we use constraints to trigger bottleneck behavior in a SUT. Table 4.1 summarizes what metrics are expected to change when one benchmark component from Figure 4.3 is varied, assuming all others are fixed. For example, as the hardware component of the SUT is varied, assuming all other factors are fixed, we expect to see a change in the efficiency and cost metrics, but not in the effectiveness metrics.

4.3 GraphAI Benchmark Design

Our design of the GraphAI benchmarking framework draws jointly from the established methods of benchmark design, methods for the empirical study of algorithms [1,2] and the theory of constraints [141,161]. The prior work in benchmark design guides *what* components to include in the GraphAI benchmark, namely the *datasets, tasks, metrics* and *system under test*. The prior work in the empirical analysis of algorithms determines *how* we compose those components to measure the effectiveness and efficiency of a SUT for a specific workload. The prior work in constraint theory determines how we identify bottlenecks and design inputs to trigger bottleneck behavior in a SUT.

4.3.1 Empirical Benchmark Design

We elect to design our GraphAI benchmark empirically, which involves applying the scientific method to drive the selection of datasets, tasks, and metrics that are included in the benchmark. The empirical process provides a formal structure that enables transparent, reproducible benchmark development, which we describe in this section. The empirical benchmark design process is marked by the iterative development and testing of hypotheses that map from input datasets to expected outputs. The hypothesis-based mapping allows a benchmark user to infer the system's internal state based on the observed (input, output) pair.

Our approach draws from John Hooker's framework for the empirical analysis of algorithms, summarized in Figure 4.2. His method identifies four key components: the *problem*, the *approach*, the *phenomena*, and the *apparatus*. Before explaining the method, we define these four components. The *problem* here studies what we are trying to do (achieve some outcome in the domain and problem space). The *approach* here studies the dataset and task features we can use to stimulate the problem (identify datasets, tasks). The *phenomena* studies the interactions of an approach with a problem (identify suitable metrics and where to find them). The *apparatus* are the tools and techniques we use to investigate, measure, or otherwise study those interactions between the phenomena and the problem (get datasets, implement tasks, instrument system, monitor execution, etc.).

The process begins by **Identifying and Characterizing a Relevant Problem**, selecting a use case to study, and identifying the typical tasks, datasets, and challenges. **Identifying and Characterizing the Phenomena of Interest** then investigates the 'interesting things' we expect to observe and explains why measuring them helps us understand the underlying problem. After

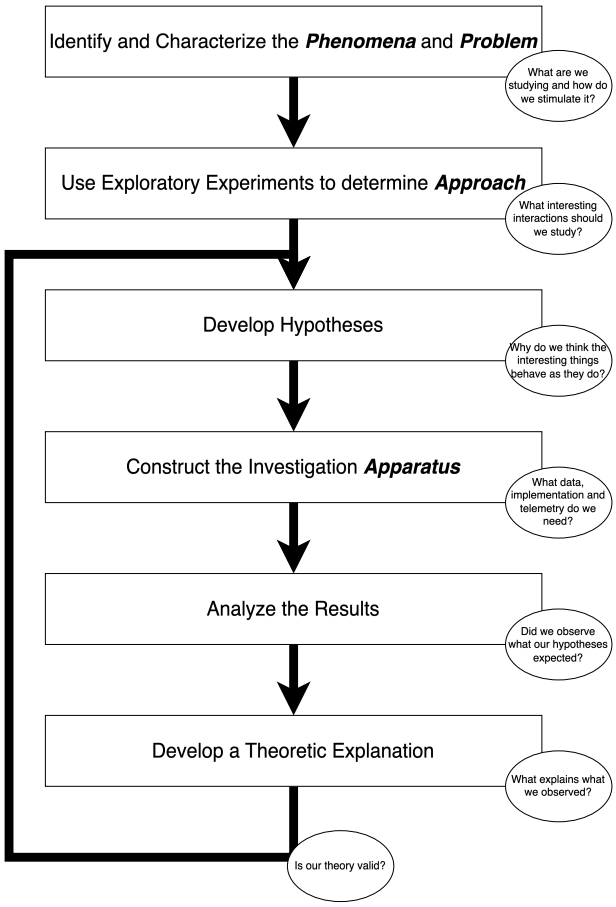


Figure 4.2: Hooker's methodology for an empirical science of algorithms [1, 2].

characterizing the problem and phenomena **Exploratory Experimentation** gathers the foundational understanding of the bottlenecks, friction points, curiosities, and insights about how to stimulate those bottlenecks. These insights are used to design meaningful experiments and valid **Hypotheses**. The hypotheses focus the analysis and inform data collection by specifying what phenomena we expect to observe given a problem and an approach under study. With the hypotheses established and the phenomena identified, the next step is to **Construct the Investigation Apparatus** that will coordinate interactions between the benchmark and the system, generate tasks and datasets, and collect the observable behaviors to produce the metrics of interest. Finally, the experimenter must **Analyze the Results, Develop a Theoretic Explanation and**

Iterate to move closer to understanding the foundational truths of the problem being studied.

4.3.2 Design using the Theory of Constraints

The design of the GraphAI framework fuses Hooker's empirical approach with Goldratt's constraint theory [141,161]. Constraint theory uses 'five focusing steps' to identify and remediate system bottlenecks. First, **identify** the bottleneck, then **exlploit** it by improving the system's efficiency surrounding the bottleneck. Next, **subordinating** the rest of the system to the bottleneck sets the system's throughput equal to that of the bottleneck to prevent a buildup of work-in-progress. Eventually, we aim to **elevate** the bottleneck by expanding its capacity until it is no longer the bottleneck before **iterating** to find the new constraint. We leverage this process in reverse order by designing tasks and datasets that promote potential bottlenecks to constrain the system's performance. It is then up to the SUT to identify how to use the acceleration techniques to exploit or elevate the constraints the adversarially shaped input data causes for the problem-centric tasks.

4.4 Our GraphAI Benchmark Framework

This section will present each of the steps in our proposed benchmark development methodology summarized in Figure 4.3, to develop a 'grey-box' approach to GraphAI benchmarking using an Input/Output model. It should be used as a guide for applying the methodology to develop a single GraphAI benchmark problem to be inserted into the framework. Figure 4.3 consists of six steps across the six black boxes. The grey circles are the focusing questions for each step, and the questions below the lines in the black boxes are the output prompts that define what the step

should be producing. The arrows show how each step contributes to the benchmark framework. Note that the GraphAI benchmark design is independent of the SUT, defining the observation apparatus as a single interaction point between the GraphAI benchmark framework and the SUT. We present the critical methodological considerations at each step to develop a single benchmark to be added to the framework and make some general observations about the requirement for and design of the observation apparatus. Chapter 5 demonstrates a worked example of how the methodology is implemented for a specific problem.

Identifying the Use Case. The first step in creating the benchmark is to identify the underlying domain for the benchmark and select several problem use cases. The use case may be driven by necessity or the target of exploratory research. The domain should be from the real world, but provided there are open problems and some representative datasets are available, most problems can be formalized into a use case.

Identifying and Characterizing a Relevant Problem. Characterizing the problem is about identifying what we want to test and identifying the tasks and datasets typical of the domain with a background study. Tasks should be defined as general goals and challenges rather than specific kernels. Formulating problem-centric tasks instead of abstracting them to a kernel encourages innovative problem-solving approaches. For example, in GraphAI , a problem definition might be *'Given the input X, identify who the immediate associates of a person with name Y, provide the output as a list of vertex IDs'* rather than kernels like *'BFS to node X'*, *'search for X in a GDB'*, etc.. The problem-centric approach still allows for comparing like-approaches (e.g., comparing the same BFS algorithm on two different SUTs) by freezing all other benchmark components and varying the SUT hardware.

Identifying and Characterizing the Phenomena of Interest. The goal of characterizing

the phenomena is to identify what we want to understand about the problems and what metrics we need to collect to understand them. The phenomena of interest is the trade-off between the efficiency, effectiveness, and cost of GraphAI systems. Thus, the metrics selected for the benchmark should characterize the trade-off between those aspects of performance. While not shown explicitly in Figure 4.3, the phenomena are generated by the interactions of the approaches, datasets, and tasks within a given problem and collected by the observation apparatus. For example, in most GraphAI tasks, we expect Extract Transform & Load Time (ETL Time), Ex Time, BLfootnoteThe time between when a vertex or edge is added to a graph and when it appears in the result of a workload [122] and *Cost* to be the chosen metrics. For each problem, the phenomena must be tailored so that an output's assessed 'goodness' is meaningful.

Conduct Exploratory Experimentation. Exploratory experimentation provides the initial clues about the bottlenecks, friction points, and curiosities worthy of further study. We are trying to understand how the phenomena relate to the problems and develop theories that we can explore formally later in the design process. Experimentation can be direct, using the problems and datasets identified in the prior steps to test some simple theories about expected performance. It can also be indirect, relying on a synthesis of prior experimental work to develop an understanding of the expected phenomena arising from a given approach to a particular problem. In GraphAI , reviews of workload analyses, the findings of prior benchmarks, and the designs of acceleration techniques offer valuable insight into what approaches will interact with what data to produce interesting phenomena.

Develop Hypotheses. Developing hypotheses is a scientific approach to quantifying the behavior we expect to observe in the system and designing circumstances to stimulate those behaviors. Hypotheses are developed in this methodology by using the knowledge gathered dur-

Constraint	**Dependent Metric**		
Component	Effectiveness	Efficiency	Cost
None (Baseline)	—	—	—
Time	↓	—	↓
Accuracy	—	↑	↓
Cost	↓	↓	—

Table 4.2: Summary of expected metric behaviors when a benchmark component is constrained.

Where: ↑ = Improve, — = Stable, ↓ = Degrade

ing exploratory experimentation. Hypotheses are effectively the mapping from input shapes to expected outputs in the context of some bottlenecks we expect to stimulate in the system. Consequently, it can be helpful to derive them by applying the three constraint levers of time (efficiency), resources (cost), and accuracy (effectiveness). These levers provide a mechanism to identify the trade-offs in GraphAI between efficiency, effectiveness, and cost by constraining one (or more) and observing changes in the phenomena under observation. Table 4.2 summarizes the behavior we expect to stimulate from a SUT by constraining time, accuracy, or cost across any GraphAI problem and defines an important interaction of constraint theory with the empirical approach to the study of algorithms, showing how manipulating the input or the constraints on efficiency, effectiveness and cost can shift the stress points within a SUT to provide a 'grey-box' view.

Construct the Investigation Apparatus. The investigation apparatus is constructed in three key steps. First is collecting or generating datasets that will stress each hypothesis to stimulate the phenomena we intend to observe. The second is defining the metrics of interest and developing a mechanism to collect the data needed to generate them. The third step is integrating the SUT into the apparatus. There will be some API configurations in an input/output paradigm, but for a hosted solution, it may involve test compilations, etc.. Specific to the GraphAI bench-

mark is a need to trade-off between effectiveness and efficiency that will require sampling the available answers over time and evaluating correctness as a distance from the ground-truth solution rather than a binary judgment of correctness.

Analyze the Results, Developing a Theoretic Explanation and Iterating. These three steps are combined because they are largely beyond the scope of the benchmark itself. The researcher is responsible for these, but the benchmark must enable the researcher to conduct their analysis. To that end, the benchmark should report the *efficiency, effectiveness*, and *cost* (performance metrics) of each workload on each SUT configuration. It should also report the details of the datasets, implementations, and SUT for posterity. Results should be viewable from the perspective of a SUT (exploring each aspect of performance) and from the perspective of each performance aspect (comparing across SUTs). The preliminary results can then be used to develop specific constraint-based experiments to be incorporated in future benchmark iterations as the hypotheses are refined and more input/output mappings are identified.

4.5 Summary: A 'Grey-Box' GraphAI Benchmark Framework

This chapter develops the GraphAI benchmark framework by identifying the common benchmark components and the need to add an 'observation apparatus' to mediate between benchmark problems and a SUT in an Input/Output paradigm. Synthesizing the empirical approach for the study of algorithms with constraint theory shows how our GraphAI benchmark problem development methodology empowers a benchmark developer to identify the problems and datasets that will trigger known and suspected bottlenecks in GraphAI algorithms and systems, creating a 'grey-box' method to investigate the internal state of SUTs without the inva-

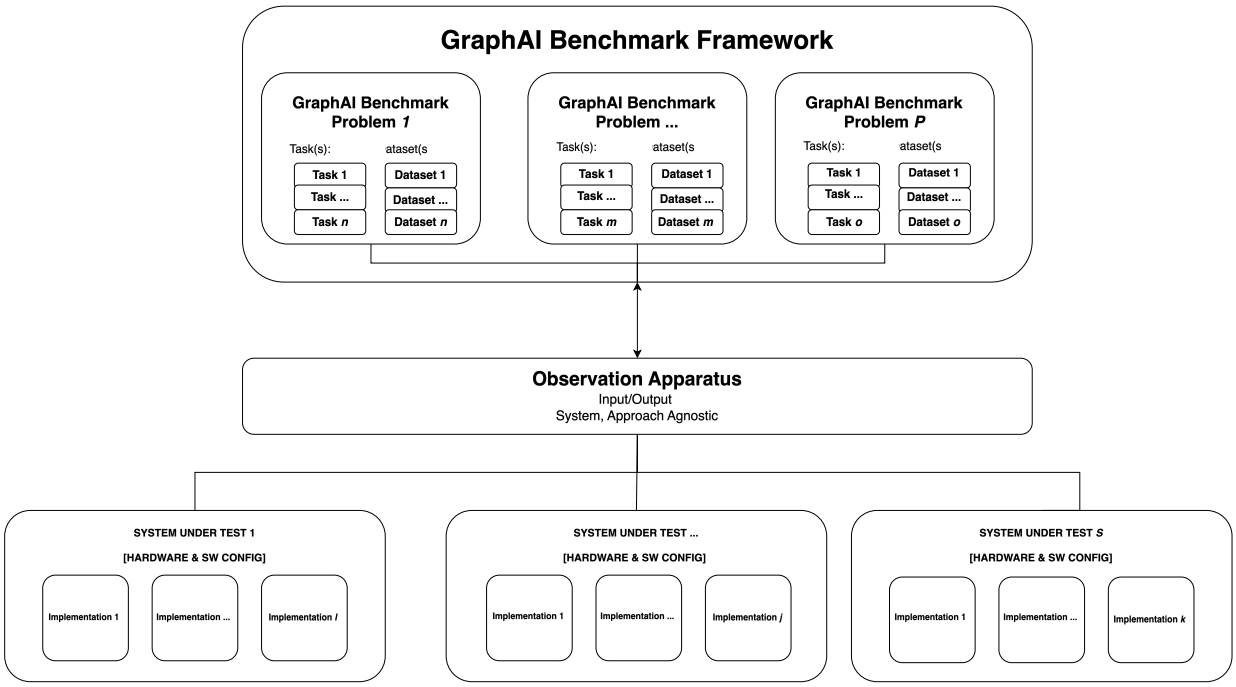


Figure 4.3: The components of the GraphAI benchmarking framework.

sive access required by traditional 'white-box' approaches. The next chapter shows how the GraphAI benchmark problem development methodology creates the *Population Segmentation Benchmark Problem* for the GraphAI benchmark framework and how the benchmark can evaluate systems' relative performance.

Chapter 5

The Population Segmentation Problem

This chapter describes the concrete steps to construct an exemplar benchmark on the population segmentation I&S domain use case. We follow the methodology outlined in Chapter 4 to develop the benchmark and demonstrate how it could be used by an engineer to identify how available hardware accelerators will impact GraphAI performance.

5.1 Identify and Characterize the Population Segmentation Problem

In this section, we describe the population segmentation problem defined in the I&S domain and identify the typical tasks and the characteristics of real-world datasets.

5.1.1 Definition

A *Population Segmentation Analysis* aims to divide a population into groups based on geography, demography, attitude toward an issue, or other thematic lines. These groups or 'segments' are a subset of the population with common attributes to which organizations often want to tailor their communications [55]. Beyond its I&S applications to Intelligence Preperation of the Battlespace (IPB), Information Operations (IO), and Civil-Military Coorperation (CIMIC), population segmentation is used heavily in marketing.

Traditionally, segmentation is done using data collected through surveys or observation. More recently, similar results have been achieved by mining large volumes of public social media

and communications data. For example, prior work has shown how email patterns can predict social subgroups [56] and phone call records reflect geographic and cultural clusters in the real world [57]. These communication networks can be represented as a graph. Each individual is a vertex, and each interaction (phone call, email, etc.) is an edge. Weights may reflect the volume or durations of interactions, and attributes for vertices and edges may be used to incorporate additional information.

5.1.2 Tasks

GraphAI tasks that can be used for population segmentation include *Graph Partitioning*, *Graph Clustering*, and more broadly *Community Detection (CD)*. Graph Partitioning takes a *top-down* approach and Graph Clustering a *bottom-up* approach to the task of optimally dividing a graph into K subgraphs such that each vertex belongs to exactly one partition, all partitions are disjoint, and some objective function is optimized. Both graph partitioning and graph clustering have been proven NP-complete. CD is a more general form of the problem that involves the discovery of latent *Communities*, or "subgraphs whose members are more tightly connected than the broader graph" [162]. CD is closely related to partitioning and clustering. However, it is distinct that communities are a more general structure that do not adhere to the requirements for independence that clusters and partitions enforce.

Based on the problem survey (section 2.1), available datasets, and expected tasks, we identify the initial collection of problems in Table 5.1, presented in a general form. These problems are reflective of real-world user needs, and many of them require combining kernel-level tasks to generate a solution to the problem. Their general specification allows users to identify the

Problem	Output Format
[1] Who is in the same group as `(x)`?	List of Vertex IDs
[2] Is `(x)` likely to be know `(y)`?	Tuple of (Y/N, Confidence)
[3] Count the vertices in the largest and smallest disjoint segments.	Tuple of (small_size,big_size,[small],[big])
[4] `(x)`, `(y)` and `(z)` are in communities `(a)`, `(b)` and `(c)`. Who else is in `(c)`?	List of Vertex IDs
[5] Who shares a segment with both `(a)` with property `(1)` and `(b)` with property `(2)`?	List of Vertex IDs
[6] Which segment has the most vertices with property `(3)`?	Tuple of (Group_name, [Vertex IDs])
[7] What are the segments, and who is in each?	List of Lists of Vertex IDs
[8] There are `(n)` segments, which segment does each vertex belong to?	List of Lists of Vertex IDs

Table 5.1: Example problem-centric tasks for the population segmentation benchmark.

approach that they believe will work best, given their systems' constraints. For example, task 1 *"Who is in the same group as (x)?"* requires that the vertex labeled 'x' be located, and then some process must identify to which group 'x' belongs. The problem-centric view means many approaches could be used to solve the problem. One approach might apply a CD algorithm like *Louvain* [3] or *Leiden* [163]. However, another could perform BFS for the node and return the nodes within a k-hop neighborhood as an approximate solution.

5.1.3 Datasets

Population segmentation focuses on human social networks, which are typically scale-free (see Section 2.3) and tend to exhibit well-defined segments. Segments can also exist in mesh graphs, but they typically are poorly defined or have boundaries that do not necessarily match the shape of the graph. For example, a road network may be divided into segments along county lines. However, those lines can be arbitrary in terms of the road density and the population living there.

The *community* definition reinforces that graph topologies are expected to consist of smaller, densely connected subgraphs within the containing graph. At the well-defined end of 'communities,' a graph will consist of cliques (completely connected subgraphs) connected to other cliques by single-edge 'bridges.' At the poorly defined end, a graph or multigraph can be fully connected with no topological community structure, and node attributes must be included to provide clues about community structure. The 'fuzzy' community is effectively a uniform random distribution, where the 'closeness' of any given community is no greater than that of the graph as a whole.

5.1.4 Outputs

Completing the problem definition step identifies the problems, common tasks, and datasets we use to inform the next steps.

Problems. Table 5.1 formally defines several problems in the population segmentation space that vary from single kernel-level tasks to complicated problems requiring intricate solutions.

Tasks. The literature's three most common approaches to population segmentation are Graph Partitioning, Graph Clustering, and Community Detection. When we take a problem-centric view and consider why these approaches exist, we develop a collection of tasks that are general enough to accommodate many approaches and are reflective of real-world needs.

Datasets. Typical datasets are scale-free and vary between having well-defined communities (cliques) and no clear community structure (fuzzy). We expect community detection approaches to be more effective on cliquey datasets than on fuzzy or topologically flat communities (e.g., fully connected graphs).

5.2 Identify and Characterize Phenomena of Interest

We identify the phenomena of interest from the results of the problem survey (Table 2.2) that indicate the prevalence of temporal constraints that force trading off effectiveness for faster partial or low-resolution solutions. This section describes how we characterize the phenomena of interest as the trade-off between effectiveness, efficiency, and cost.

5.2.1 Efficiency

Efficiency measurements are similar from problem to problem in GraphAI . We use the ubiquitous Ex Time but add support for ETL. Additionally, many approaches to population segmentation use iterative algorithms and so are well suited to providing interim solutions, making the measures of BL (time from data being sent until a result is generated) and Batch Rate (BR) (average time to produce an interim result) well suited for comparing static and dynamic approaches to the problem under the same framework.

5.2.2 Effectiveness

There are two ways to evaluate population segmentation's effectiveness: ground truth labels or structural characteristics. We prefer effectiveness evaluations using ground-truth label-based methods, and Normalized Mutual Information (NMI) [164] is the most commonly used metric to evaluate predicted segments against ground truth. NMI's scalability limitations drive alternate methods like Network-F1 (NF1) [165–167] but have not gained widespread traction. Without ground truth labels, the 'goodness' of a segment is calculated using a metric like *modularity* [168], *conductance* [169], or the *Constant Potts Model* [170]. These methods are limited

in their ability to provide a reliable evaluation, as they effectively model the probability that the community is more connected than a random graph. Further, optimizing the modularity metric is NP-Complete [18], and heuristic approaches degrade for small communities and skewed community distributions [162], which are typical of scale-free graphs [17, 171] that characterize the problem domain.

5.2.3 Cost

Cost is ideally measured as a function of power consumption or as an estimated 'cost per minute' based on the SUT's specifications. The GraphAI benchmark framework initially focuses on an Input/Output benchmark paradigm. Consequently, 'cost' is challenging to measure because the system's internal state (power consumption, memory usage, processor cycles etc.) is unknown. To better address cost in the future, we propose deploying a hosted benchmark instance to the ARLIS *Quantum Inspired Classical Computing* (QUICC) testbed. The testbed will provide measurements of power consumption by the controlling containers using *Kepler*[1]. Further, we aim to develop a daemon to deploy to a SUT to collect power consumption statistics using the *PowerAPI Project*[2], which would be an optional component of the benchmark limited to running on compatible physical Linux-based systems. Currently, cost is excluded from the Population Segmentation benchmark problem due to the lack of an accurate measurement avenue and the inherent drawbacks of predictive cost models.

[1] https://sustainable-computing.io/design/architecture/
[2] https://powerapi.org/

5.2.4 Outputs

In the phenomena identification step, we identify the interaction between effectiveness and efficiency as the phenomena of interest, namely whether it is possible to trade lower effectiveness for better efficiency. Effectiveness metrics of *NMI* (preferred) or *modularity* will be used as proxies for effectiveness. Metrics of ETL Time, Ex Time, and BL will be used as proxies for efficiency.

5.3 Conduct Exploratory Experimentation

For our exploratory analysis, we conduct a survey of prior work in GraphAI benchmarking, workload analysis, and acceleration for the population segmentation problem, grouped broadly by approach (Table 5.2). Almost all prior work falls within the 'community detection' approach discussed in Section 5.1, and so the language of this section reflects that focus. For each approach, we identify the reported worst-case complexity for the algorithm as the efficiency bottleneck and factors that degrade the quality of communities as the effectiveness bottlenecks.

Below, we characterize our observations of the stressors to community detection performance: datasets, algorithmic weaknesses, modularity optimization flaws, and dynamic graph processing challenges. During our exploration, we identified two techniques to improve performance: external information and approximate solutions.

Family	Algorithm	Efficiency Bottleneck	Effectiveness Botleneck
Graph Partitioning	Kernighan-Lin [172]	$\mathcal{O}(n^3)$	Dense; Small communities; Unknown C
	Extremal Mod. Optimization [173]	$\mathcal{O}(n^2 \log^2 n)$	Fuzzy
	Spectral Partitioning$_{base}$ [174]	$\mathcal{O}((m+n)n)$	Dense
	Spectral Partitioning$_{cl}$ [174]	$\mathcal{O}(n^3)$	Dense
	FEC [175]	$\mathcal{O}(cl(n+m))$	Fuzzy; Many communities; No Attributes
	Potts-Fuzzy [176]	$\mathcal{O}(m+q)$	$L_{in} \gg L_{out}$; Unknown C
Graph Clustering	Spectral Clustering [177]	$\mathcal{O}(n^3)$ [178]	Unknown C
	Girvan-Newman [17]	$\mathcal{O}(n^2 m)$	Fuzzy; Dense
	Hierarchical Agg. [179]	$\mathcal{O}(md \log n)$	Dense; Large C, Small C; Fuzzy
	Hierarchical Div. [168]	$\mathcal{O}((m+n)mn^2)$	Dense; Fuzzy
	Spectral [178]	$\mathcal{O}(nc^2 e)$	Skewed Community Distro; Unknown C
	Louvain [3]	$\mathcal{O}(n^2)$	Many Clique-Bridge Vertices
	Leiden [163]	$\mathcal{O}(n^2)$	Fuzzy
Graph Prediction	DNR [180]	$\mathcal{O}((nh+h^2)+(incd))$	Fuzzy; Unknown C
	Com-E [181]	$\mathcal{O}(n+m)$	Dense; Fuzzy
	M-NMF [182]	$\mathcal{O}(n(m^2 n))$	Dense; Fuzzy; Many C; Unknown C
Dynamic Graphs	Instant Optimal [167, 183]	$\mathcal{O}(T((n^2)+(c^2)))$	Fuzzy; Many C
	Temporal tradeoff [167, 184]	$\mathcal{O}(T(md \log n))$	Many changes per T
	Cross-Time [167, 185]	$\mathcal{O}(3^{\frac{d}{3}} 2^2 mnt)$	Dense

Table 5.2: Survey of computational approaches to population segmentation, showing efficiency and effectiveness bottlenecks.

Where: n is the number of vertices, m is the number of edges, c is the number of communities/clusters/partitions, k is a degree, t is the number of time slices, d is the degeneracy of the graph, h is the number of hidden layers, and i is the number of iterations. l is the number of steps in a random walk q is the number of permitted values in a potts-hamiltonian spin state e is the number of iterations for the K-Means algorithm to converge

5.3.1 Datasets Observations

Graph size. From the survey in Table 5.2, the worst-case complexity is quadratic in the number of vertices, particularly for the dominant algorithms like Louvain [3] and Leiden [163]. We observe that in many cases, efficiency decreases as $|E| \gg |V|$.

Graph shape. Further, we identify that the effectiveness of community detection approaches degrades when the communities are 'fuzzy' rather than 'cliquey' and when the mean degree of the graph is high.

5.3.2 Implementation Observations

Algorithmic weaknesses exist in community detection. Many approaches require the number of communities, clusters, or partitions to be known in advance, or they must run an algorithm like K-Means multiple times to discover the optimal number of communities ([172, 176–178, 180, 182]), significantly increasing the computational cost when the number of communities is large. Knowing the number of communities a priori does not reflect the discovery process expected in the real world [48]. Recursive-bisection partitioning approaches fail when there are three equally sized communities at a given recursion level; at least one will be mistakenly grouped with another and will usually be split across both. Greedy-agglomerative clustering approaches fail when two cliques are connected by nodes of a lower degree than the clique, pulling the 'bridge' nodes into the cluster they bridge to rather than their own. Most approaches cannot assign a vertex to multiple communities because of the underlying assumptions about clustering and partitioning, so overlapping communities rely on graph attributes for effective community detection on heterogeneous graph structures [186]. The most popular implementation (Louvain [3])

has been shown to produce disconnected communities when the shape of the input graph contains hierarchical community structures that contain cliques joined by bridge nodes [163].

Modularity optimization is not a reliable strategy. Modularity has several problems despite being the dominant metric in measuring community goodness. Modularity optimization is NP-Complete [18], so algorithms must take heuristic approaches to apply it. Modularity is also scale dependent, meaning it has a scaling factor where a community with a density of internal edges of the order of the $\sqrt{2|E|}$ times the edges in the graph will not be resolved by modularity optimization. That is, small communities will be missed by modularity optimization. In general, 'loosely connected' communities are mistakenly combined, especially when the community size distribution has long tails [162].

Dynamic processing relies on short batch latency. The inputs to dynamic graph processing algorithms are either a collection of related graphs describing the same problem at T time slices or may have a stream of updates to reflect when a vertex is added, deleted, or updates its properties. Consequently, the task is typically to identify when a community changes or which communities persist for extended periods (i.e., the co-traveler problem). Many dynamic processing approaches batch the updates and apply a static community detection approach at each iteration, then calculate the similarity in communities between each timestep, with more efficient implementations keeping the prior state in memory to reduce computation [167]. These batching approaches fail in real-time settings where the batch latency exceeds the length of each timestep T.

Community Detection is suitable for approximate and incremental approaches. Clustering, partitioning, and CD are typically implemented as iterative algorithms that converge to a heuristically optimal solution but can return some information at each iteration step and are

vulnerable to local maxima. Running an approximate algorithm may provide a less accurate result in a shorter timeframe, and iterative approaches that yield incremental improvements as time proceeds can take longer to converge than approaches that use a single-shot approach. Datasets with hierarchical communities can have correct solutions at different resolution levels, meaning that an iterative algorithm can yield communities at different resolutions over time.

Weights and attributes are critical for segmenting fuzzy and overlapping communities. Graph partitioning, clustering, and community detection are traditionally topology-only algorithms. However, access to some states, like the sentiment of an edge [175], or the weights associated with edges [187] yield superior segments. The same is true of attributes, and prior knowledge about group membership of some nodes dramatically improves the identification of overlapping and fuzzy communities [188].

5.3.3 Outputs

Completing the exploratory analysis through a literature review reveals the bottlenecks that impact population segmentation approaches, guiding hypothesis development in the next step.

Efficiency bottlenecks. We expect that increasing the number of vertices and the number of edges will degrade the efficiency of the approaches, given the worst-case complexity estimates in Table 5.2.

Effectiveness bottlenecks. The effectiveness of CD approaches is vulnerable to fuzzy and overlapping communities, denser graphs, and when 'bridges' exist between cliquey structures.

Friction points. We identify additional friction points, including the requirement for a priori knowledge of the number of communities by many approaches, the lack of support for

overlapping communities, and the lack of work in dynamic CD benchmarks.

Curiosities and opportunities. The salient opportunity is to explore the efficiency-effectiveness tradeoff by using an iterative approach to community detection that outputs interim results as they are generated and offers the opportunity to experiment with trading off efficiency and effectiveness. Another interesting constraint-based problem is in finding the optimal timestep T length for a dynamic CD approach to determine how long it should wait when receiving a stream before commencing a batch update.

5.4 Develop Hypotheses

Hypotheses link the problem and phenomena characterization with domain information gathered in exploratory experiments to specify the inputs and expected outputs for the 'grey-box' benchmark. They effectively formalize the mapping from input data to output characteristics and describe the nature of the bottleneck we expect to exist in a SUT if we observe that input/output pair in the experimental results. For each hypothesis, we consider what system characteristic or bottleneck we want to stimulate and build a hypothesis about how to shape the *input* to trigger an observable change in the *output* compared to some *baseline* of performance.

Inputs. The *size, shape* and *community structure* of the input graph are the three parameters that we can vary to elicit observable responses from the system.

Outputs. The *ETL Time, Ex Time, BL* and *NMI* of the solution are the observable components of the output that we can use to interpret the effect of the variation in inputs.

Baseline. All hypotheses are developed relative to a baseline configuration of a single socket Central Processing Unit (CPU), representing the graph as an adjacency list and implementing a

sequential single-shot algorithm on a small uniform random graph. The baseline configuration is selected as the simple, 'entry-level' configuration that most initial GraphAI prototyping will start from as proof-of-concept before transitioning to a more advanced configuration optimizing for efficiency.

Our 'grey-box' approach stimulates three areas: *hardware* to stress specific configurations, *representation* to stress different methods of representing and operating on graphs, and *implementation* to stress different algorithmic approaches. As Section 1.2 explains, there can be unexpected interactions as we combine hardware, representations, and approaches; detecting these interactions is one of the goals of hypotheses design.

5.4.1 Stimulating Hardware

The hardware configuration of a SUT directly impacts processing efficiency. While some changes will yield a linear improvement, like increasing available Random Access Memory (RAM), or increasing the memory bandwidth, other approaches like distributing computing [76], using multi-socket CPU systems [101], using Graphics Processing Unit (GPU) [90] or other specific hardware accelerators [145] can have more dramatic impacts on efficiency. From reviewing the literature on GraphAI benchmarking and workload analysis (see: Section 3.3) and the results of our exploratory analysis (see: Section 5.3), we know that the size of a graph (number of vertices), the density of the graph (number of edges), and shape of the graph can cause bottlenecks in different hardware configurations. To stimulate these bottlenecks, we vary two parameters in the input graph, the vertex count and graph shape, and define the following hypotheses about the expected impact on benchmark efficiency outputs:

H0: Distributed hardware approaches will have a longer ETL time relative to the same input and implementation on the baseline system.

H1: Distributed hardware approaches operating on scale-free input graphs will have a longer Ex Time than they do on mesh input graphs of comparable size.

H2: A distributed hardware implementation will have a longer Ex Time than a shared-memory parallel implementation for the same input data.

H3: GPU and other Graph Hardware Accelerator (GHA) hardware configurations will have a longer ETL time relative to the same input data and implementation on the baseline system.

H4: GPU and other GHA hardware configurations will fail as input graph sizes become larger than their on-chip memory.

H5: GPU will have longer ETL time for scale-free input graphs than for mesh input graphs of the same size.

H6: The effectiveness of each workload is independent of the hardware configuration used.

5.4.2 Stimulating Representations

Representational choices about storage, abstraction, primitive operations, and programming models can directly impact the efficiency of a SUT. For example, prior work reviewing linear algebraic graph processing discussed in Sections 2.4 and 5.3 has shown that the cost of converting between the canonical and linear algebraic representation can increase ETL time. Despite the linear algebraic efficiency gains, the Ex Time of linear algebraic approaches for CD is

slower than a system operating with the GAS processing model because of the transformation overheads [189]. The baseline for representations is an uncompressed adjacency list-based representation (i.e., CSC/CSR). To stimulate these bottlenecks, we vary the size and shape of the input graphs and define the following hypotheses about the expected impact of varying the input:

H7: Linear Algebraic representations used in CD will have a longer ETL time for the same input data than the baseline approach.

H8: Compressed representations will have a longer ETL and Ex Time than baseline representations of the input data.

H9: The effectiveness of each workload is independent of the representation used.

5.4.3 Stimulating Implementations

Choices about algorithm implementation used to solve the task on the input dataset can impact a solution's efficiency and effectiveness. Approximate solutions will accept a less effective result for a more efficient one, and iterative or incremental solutions will improve over time, yielding interim results that are less effective but much faster than waiting for the result of a single-shot implementation. Being able to handle static or dynamic graphs is also an essential distinction in evaluating implementations. The baseline for implementations is a sequential, single-shot algorithm for a static graph. To stimulate these bottlenecks, we vary the size and shape of the input graphs and define the following hypotheses about the expected impacts on the efficiency and effectiveness of varying the input:

H10: The BL of single-shot approaches will be equal to its ETL plus Ex Time for all inputs.

H11: The effectiveness of all implementations will be lower for mesh input graphs than scale-free input graphs of the same size.

H12: The effectiveness of an iterative or incremental implementation will improve monotonically over time as it converges to an optimal state.

H13: The effectiveness of an approximate implementation will be lower than a single-shot 'exact' approach on the same input data.

H14: Dynamic inputs will cause ETL time to approach Ex Time in incremental implementations.

5.4.4 Selecting Hypotheses

To effectively employ the hypotheses in constructing the benchmark apparatus, we need to select which hypotheses to test to meet the needs of the benchmark use case. When deciding between hardware configurations that will run the same implementation, we want to select the hypotheses that will provide insight into which hardware configurations will be most helpful in improving efficiency and preserving effectiveness. Sections 5.6 and 5.7 apply experimentation using the GraphAI benchmark to decide between different hardware configurations for the same implementations. Therefore, we focus on the hypotheses relevant to that problem for the remainder of this chapter, collecting and generating the datasets summarized in Figure 5.1 to stimulate hardware bottlenecks.

5.5 Construct the Investigation Apparatus

The apparatus is the physical instantiation of the problem benchmark. Figure 1.4 shows how the apparatus is derived from each process step. Three of the components (datasets, tasks, and metrics) are consistent with traditional benchmarking approaches (see Figure 3.1). We add the explicit definition of the observation apparatus. These four components are the concrete implementation of the benchmark. Combined, they have two key roles. First, they must interact with the SUT – problems and datasets go in, and solutions for evaluation come out. Second, they must monitor the phenomena emerging from the SUT throughout the process. Figure 5.2 is the envisaged workflow for the population segmentation problem benchmark, showing how the physical components of the observation apparatus interact between the benchmark and the SUT.

The population segmentation benchmark workflow is representative of the expected workflow for most problems in the GraphAI taxonomy because, as a GM task, it is in the family of GraphAI problems used most by the I&S community. It can be solved in multiple ways (including with GNNs), focuses on graph topology but can also use attribute information, admits approximate and iterative solutions, has a linear algebraic implementation, and has real-world temporal constraints that may benefit from trading effectiveness for efficiency. In the remainder of this section, we summarize the concrete components of the apparatus, showing how they were created and identifying expansion opportunities.

5.5.1 Observation Apparatus

The observation apparatus is a component we include in our benchmark design to explicitly represent the decisions about how the benchmark will interact with the SUT. We imple-

ment an Input/Output benchmark, augmented with constraint-based hypotheses testing. While the Input/Output approach prevents us from monitoring the system's internal state, it maximally accommodates researchers altering the hardware, software stack, and implementation. System variance reflects the benchmark's real-world implications; if someone can afford to buy and run expensive hardware to achieve better results, they are free to do so.

More specifically, we choose the Input/Output model to accommodate the wide variety in GraphAI acceleration approaches. For hardware-based acceleration, GHA [25,85,90,97,143,145, 147, 150, 152, 153, 190, 191] and GNN accelerators [36, 133, 155, 157, 191–199] are proliferating rapidly, and a future-proof benchmarking framework must accommodate as many variations as possible to remain relevant. Represenation-based acceleration techniques for CD have created multiple different implementations including canonical sequential [3], distributed [200], shared-memory [201] on CPU and GPU [90], as well as linear-algebraic approaches for CPU [189] and GPU [202]. Configuring instrumentation for all of these systems is infeasible, and a hosted benchmark would exclude too many of the approaches, leading us to select an enhanced Input/Output model for the benchmark.

5.5.2 Datasets

Datasets in the GraphAI population segmentation problem are expected to be large, scale-free, and heterogeneous attributed graphs. The hypotheses identify the requirement to scale the number of vertices, the density of the graphs, the distribution of degrees, and the distribution of communities, and they optionally include weights and attributes. The review of CD problems also shows that CD can be applied to dynamic graphs.

To stress across these parameters, we first look for datasets in the *Tiny, Small, Medium, Large* and *Huge* range. We define *Tiny* datasets as those that fit in the cache of a commercial system, < 60Mb[3]. *Small* can be processed in memory on a personal computer, no larger than ~ 16Gb, the *medium* should fit in the memory of a commercial system, no larger than ~ 1Tb, *large* should be fit within the memory of a commercial system (smaller than 5Tb), and *Huge* should be more extensive than what will fit in the memory of a single commercial system (at least ~ 5Tb).

Building on the review of dataset coverage in Table 5.3, we should aim for full coverage of scale-free, uniform-random, and mesh shapes. All datasets require ground-truth communities/clusters/partitions for NMI calculation. Some datasets should be weighted and attributed, and partial assignments of vertices to communities should be made available as an additional stress on memory and to allow approaches to exploit this additional information in their solutions. Dynamic graphs should be suitable for batch updating and continuous streaming to evaluate dynamic processing algorithms alongside static solutions. Finally, graphs should be available to stress effectiveness with varying community distributions, mean degree, 'fuzziness,' and overlapping and nested communities.

Our datasets are summarized in Table 5.3, with coverage of our target efficiency and effectiveness difficulty levels shown in Figure 5.1. We identify seven suitable datasets from the SLNDC repository ranging from tiny to medium, which are scale-free and cover the distribution of community sizes from small to large. Noting the lack of mesh and random graphs, we generate our own.

[3]Based on Intel Xeon Gold (`https://www.intel.com/content/www/us/en/products/details/processors/xeon/scalable/gold/products.html`)

5.5.2.1 Mesh Graph Generation

There were no publicly available mesh graphs with ground truth community labels. We develop a tool to generate directed, weighted, attributed graphs with hierarchical and overlapping communities from OSM road network data[4]. Our tool queries the OSM API for a given city, county, or state, extracts the road segments (ways), and creates a vertex where segment ends meet or multiple segments intersect. We construct the graph using Python's NetworkX [203] and assign each vertex the attributes of its latitude, longitude, and street name. We group the vertices by subordinate communities. For example, a state has community groups at the county and city levels. Where a road crosses a border, it is assigned to a particular group as a member of both communities. We use the tool to create six tiny and small datasets with very low density, wide diameters, and a broad distribution of community counts across multiple resolutions.

5.5.2.2 Random Graph Generation

While random graphs are not an adequate substitute for real-world graphs from the target domain, they are helpful supplemental tools for covering real-world data gaps and developing datasets to stress specific system components. As Figure 5.1 shows, we lack real-world coverage of the spectrum of graph shapes between mesh and scale-free. Imitative generators are not yet mature enough to generate the medium and large datasets we require, and suitable statistical summaries of graphs are not available to seed imitative generators. Specific community-detection generators like LFR [9], AcMARK [33], and GenCAT [10] can generate graphs with ground-truth communities following mathematical distributions, and we use them to fill in the gaps shown in

[4] tool available at `https://github.com/osullik/umd_thesis/blob/main/md_roads/osmg.py`

Figure 5.1.

Source	Dataset	Size	Vertices	Edges	Communities	Med Deg	Topo	D	A	W	T	Domain/Model
Real-World	Karate Club [56, 138]	500b	34	78	2	50	SF					
(SNAP)	Cisco Networks$_{g_{21}}$ [204]	2Mb	317	3025	23			✓			✓	Computer Network
	Cisco Networks$_{g_{22}}$ [204]	34.9Mb	33,241	702,361	10			✓			✓	Computer Network
	Email Eu Core [205]	200Kb	1005	25571	42		SF	✓				Emails
	Orkut [206]	1.77Gb	3,072,441	117,185,083	6,288,363		SF					Social Media
	Friendster [206]	32.36Gb	65,608,366	1,806,067,135	957,154		SF					Social Media
	Wiki-topcats [205]	46.6Mb	1,791,489	28,511,807	17,364		SF					Wiki Hyperlinks
Real-World	OSM-M	240Kb	2063	2572	4; 2	2	M	✓	✓	✓		Roads
(Mine)	OSM-DC	15.2Mb	147,599	163,156	27; 10	2	M	✓	✓	✓		Roads
	OSM-DE	63.5Mb	637,463	680,858	63; 17	2	M	✓	✓	✓		Roads
	OSM-RI	65.2Mb	671,727	711,157	63; 4	2	M	✓	✓	✓		Roads
	OSM-MD	441.6Mb	4,513,407	4,745,260	402; 108	2	M	✓	✓	✓		Roads
	OSM-WA	665.3Mb	6,690,579	6,996,063	430; 108	2	M	✓	✓	✓		Roads
Synthetic												

Table 5.3: Summary of datasets for the population segmentation problem showing size, summary statistics and domain.

D is Directed, **A** is Attributed, **W** is weighted, and **T** is Temporal, **SF** is Scale-Free and **M** is Mesh. **Size** is the sum of the uncompressed adjacency list and attribute file. All datasets have a ground-truth community. Where different community resolutions exist, both are listed (e.g., city county) for the OSM graphs. Cross-community nodes are permitted.

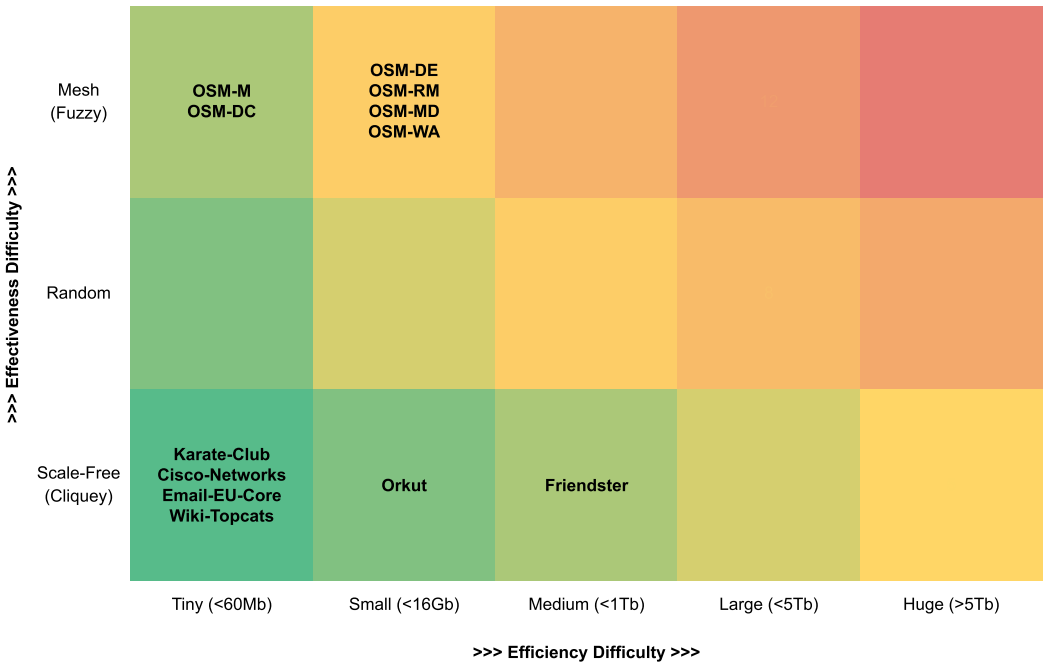


Figure 5.1: The coverage of static CD datasets showing the expected difficulty of effectiveness versus efficiency. Red colors are more 'difficult' interactions.

5.5.3 Tasks

We draw on the problems identified in Table 5.1 for our specific tasks. To enhance the benchmark, we intend for the tasks to be publicly and generically specified. That means the complete list of problem-centric task templates will be available to benchmark users in advance to prepare and test systems. However, the specific values will only be provided at experiment time to ensure fairness. For example, the public generic definition might be `'What other nodes are in the same group as vertex (x)?'`, but when the benchmark is actually run, the specific problem of `''What other nodes are in the same group as the vertex with name=='Mr_Black' ''?` Figure 5.2 shows the back-and-forth process between the SUT and the benchmark as steps $1-4$, which is how all tasks are provided

to the SUT by the Benchmark Server (BS).

5.5.4 Metrics

The Input/Output paradigm limits telemetry collection to observing what data is passed into a system, what data comes out, and how much time elapses between input and output events. Figure 5.2 shows in grey the polling period in which interim results are periodically collected along with timing information. ETL and processing are shown linearly but could be interleaved if a SUT takes an incremental approach. During the polling period, the BS will periodically poll the SUT for its current answer and log the time and content of the response, identifying partial or approximate answers where they are available before the algorithm officially 'completes.' At each process step, the BS logs the time taken to that point to provide the data for generating the efficiency metrics *Transfer Time (Tx Time), ETL Time, Ex Time, BL* and *BR* during the final evlauation phase. The effectiveness metric for all problems requiring comparing predicted communities to the the ground truth is *NMI*.

5.5.5 Limitations

Several limitations exist in the current design of the population segmentation problem benchmark. First, data transfer is highly sensitive to network bandwidth and latency, disadvantaging remote systems. Given that the benchmark uses static datasets, we minimize the chances that systems can 'cheat' by implementing randomized problem parameters, hash checks, and graph data summaries to verify that the benchmark process is being followed.

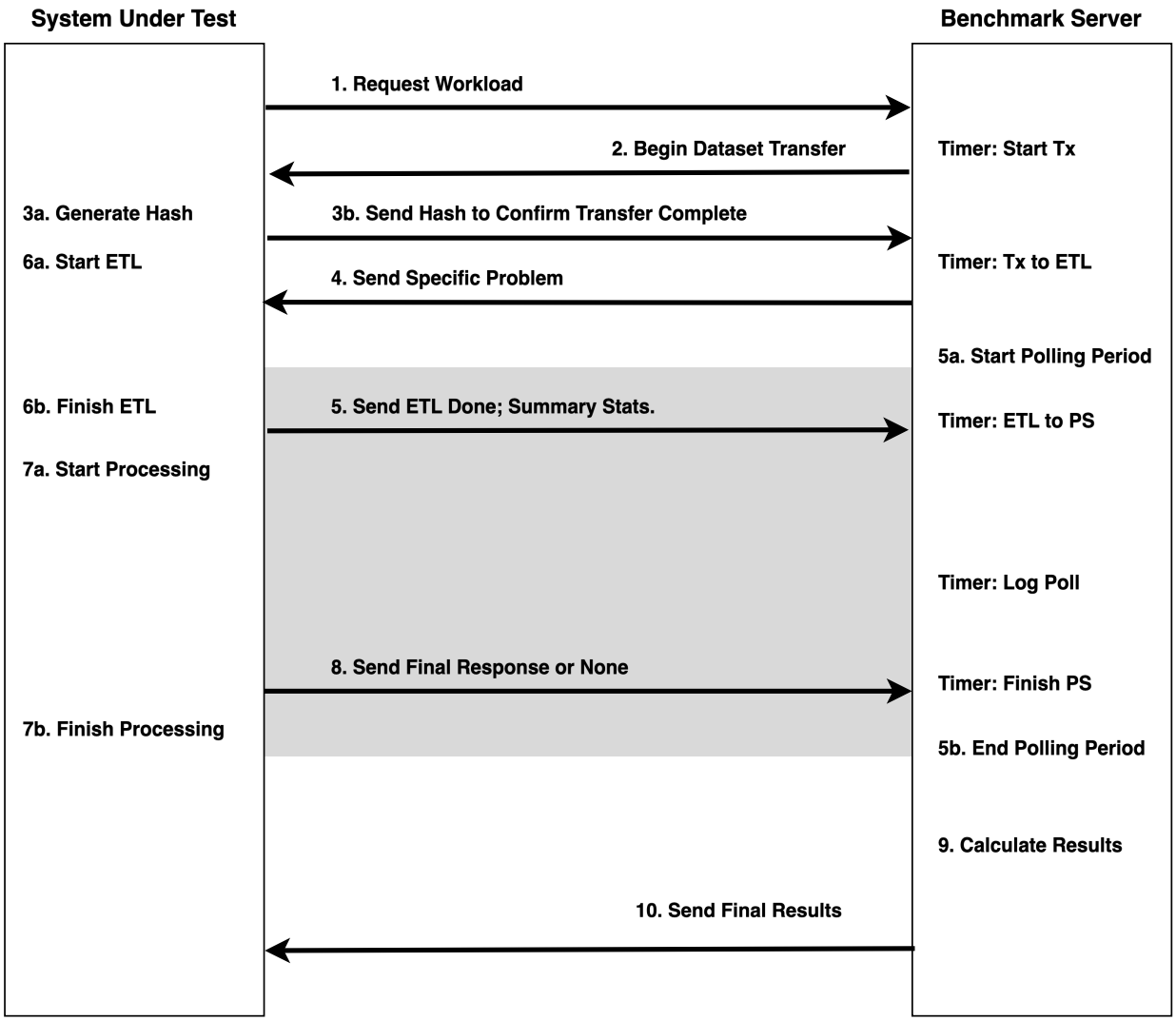


Figure 5.2: The observation apparatus for the population segmentation benchmark.

5.5.6 Outputs

The apparatus construction step yields the dataset observation apparatus, and GraphAI problem instance used for testing. Most importantly, it creates the datasets varied along the size and shape parameters that will be used to stimulate the behaviors to produce outputs consistent with our hypotheses from Section 5.4.

5.6 Analyze Results, Develop Theoretic Explanation and Iterate

This results section and the following discussion demonstrate how an engineer would use the population segmentation benchmark problem as a component of the GraphAI benchmark framework to evaluate several shortlisted system configurations for procurement. The engineer is aware of analyst requirements for their GraphAI tasks to complete within a limited timeframe and a willingness of analysts to sacrifice the optimality of the solution for a faster result. First, we describe the specifics of the experiment setup, and second, we describe the results.

5.6.1 Experiment Setup

We implement the GraphAI population segmentation benchmark in Python 3.11.5 and make the environment details available[5]. The implementation follows the design from Figure 5.2, except that the benchmark server is located on the SUT and that the polling is done passively by checking the timestamps of an interim-results folder after the experiment. To run the benchmark, the researcher specifies the problem and dataset.

We provide experiment specifications for the experiments shown in Figure 5.3, and listed in Table 5.4 as `json` files[6] that the BS uses to execute the experiments. The process is entirely automated to maximize usability, from requesting a benchmark problem until the server provides the data and figures in the return report.

We test the three SUTs. The *Baseline* system is a personal laptop computer, representing the existing capability of the Engineer's team[7]. The *CPU* system is a single-socket of a

[5] `https://github.com/osullik/umd_thesis/tree/main`

[6] available at: `https://github.com/osullik/umd_thesis/tree/main/gaib/experiment_definitions`

[7] 2023 Apple Macbook Pro; 12 Core M2 Max Processor, 32GB Hynix LPDDR5 RAM; MacOS 14.1-Darwin-23.4.0

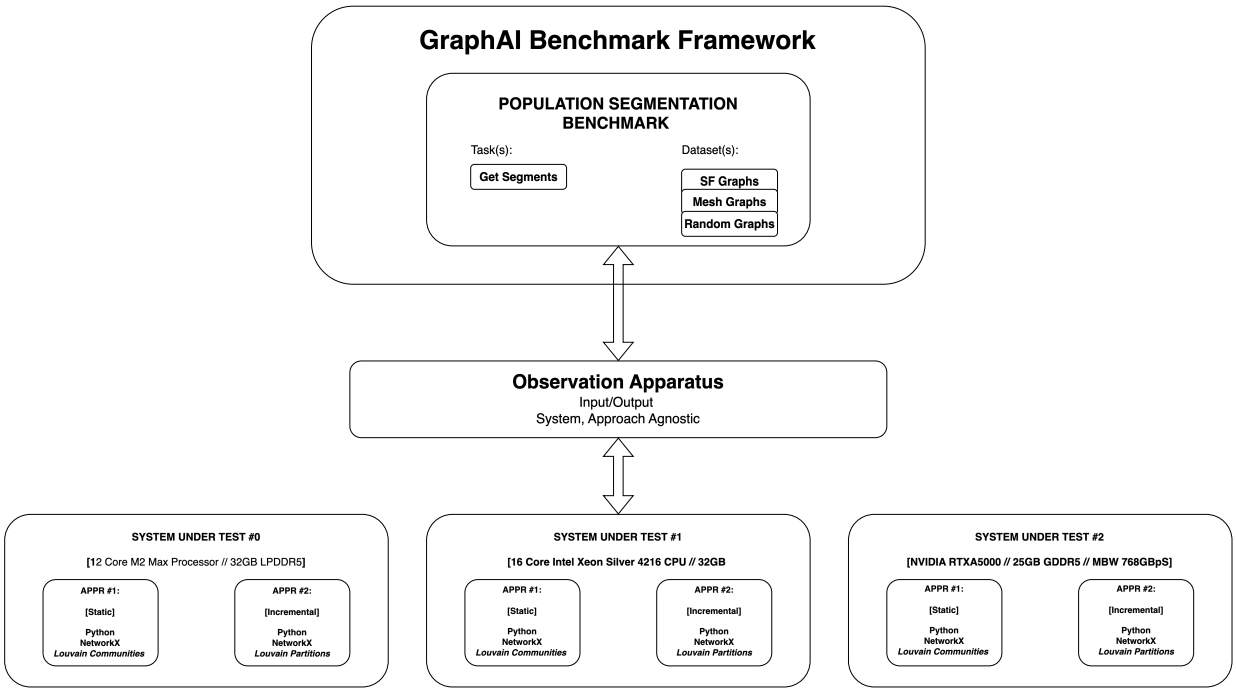


Figure 5.3: The experiments compare three SUT, each with a static and iterative implementation of the Louvain Algorithm [3].

HPC cluster[8] and the GPU system is a single GPU hosted on a HPC cluster[9]. We limit our choice of implementations to the NetworkX library to standardize the representation and ETL processes between implementations. The static approach uses the NetworkX *Louvain Communities*[10] implementation and the incremental use the NetworkX *Louvain Partitions*[11] library, which iteratively yields communities at decreasing granularity.

[8] 16 Core Intel Xeon Silver 4216 3.2GHz, 32GB RAM; RedHat Enterprise Linux 8.9

[9] As per CPU system, with addition of 1× NVIDIA RTXA5000 (8192 Cores, 24GB GDDR6, 768DB/s memory bandwidth), Compute Uniform Device Architecture (CUDA) v12.4

[10] `https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.louvain.louvain_communities.html`

[11] `https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.louvain.louvain_partitions.html`

#	Dataset	Problem	Return Format	Approach
1	OSM-M (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
2	OSM-M (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
3	OSM-M (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
4	OSM-M (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
5	OSM-DC (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
6	OSM-DC (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
7	OSM-DC (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
8	OSM-DC (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
9	OSM-DE (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
10	OSM-DE (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
11	OSM-DE (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
12	OSM-DE (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
13	OSM-RI (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
14	OSM-RI (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
15	OSM-RI (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
16	OSM-RI (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
17	OSM-MD (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
18	OSM-MD (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
19	OSM-MD (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
20	OSM-MD (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
21	OSM-WA (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
22	OSM-WA (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
23	OSM-WA (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
24	OSM-WA (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
25	Email-EU-Core	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
26	Email-EU-Core	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
27	Wiki-Topcats	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
28	Wiki-Topcats	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental

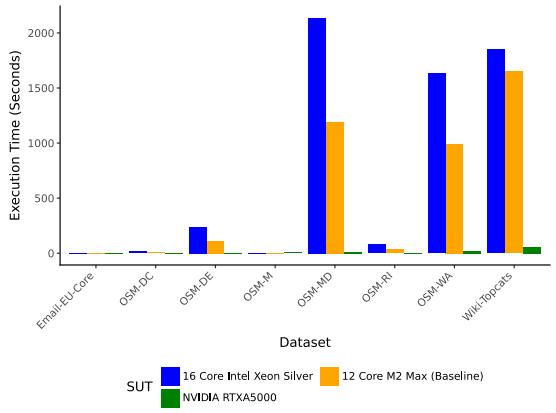
Table 5.4: Summary of experiments for the population segmentation benchmark. Each SUT completes the same tasks on the same data.

5.6.2 Experiment Results

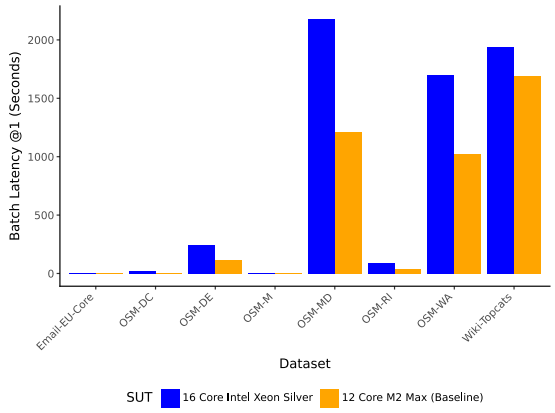
We tie the evaluation closely to the Hypotheses from Section 5.4 and group the results by experiments based on hardware stimulation and implementation stimulation hypotheses. Hypotheses are used as a guide for investigation rather than a formal testing mechanism, so we abstain from statistical analysis of results at this time but intend to automate evaluation against the hypotheses as part of future work.

5.6.2.1 Hardware Stimulation Results

Figure 5.4 shows how the GPU SUT completes the GraphAI population segmentation benchmark problems up to 172× faster than the baseline SUT using the static implementation.



(a) GPU speeds up Execution Time by up to 172× compared to baseline performance.



(b) Batch Latency for first result is ∼total execution time. The GPU has no incremental implementation.

Figure 5.4: The GPU implementation is drastically faster than the Baseline and Xeon SUT for the same tasks.

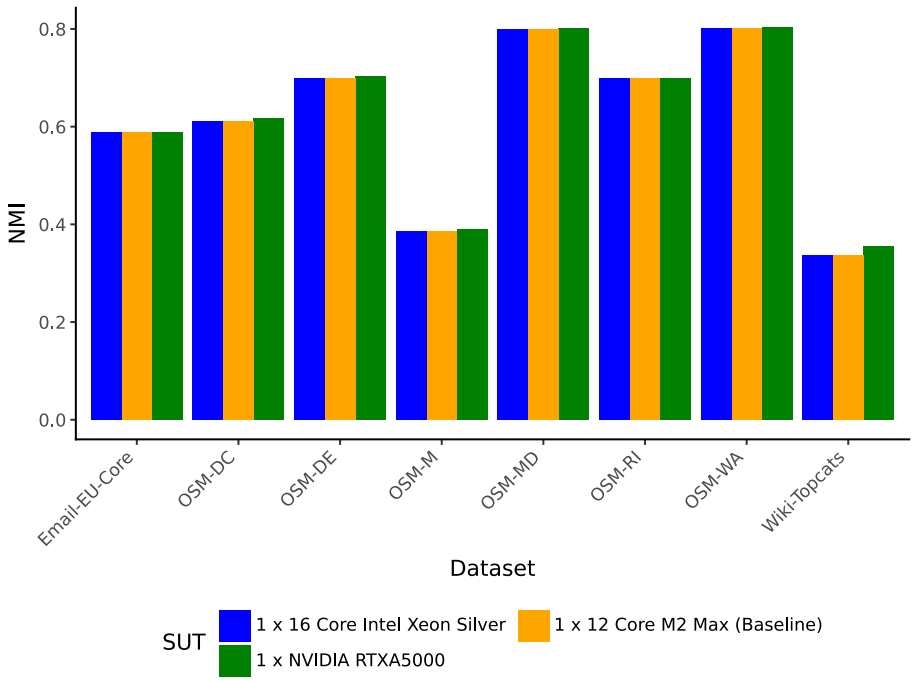


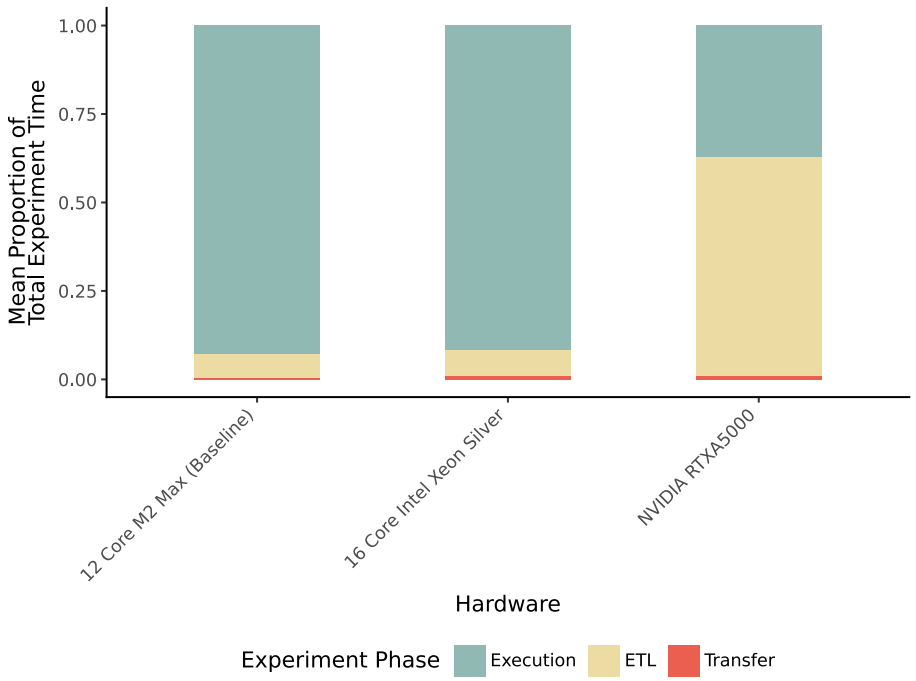
Figure 5.5: The GPU SUT experiences a dip in NMI for Scale-Free datasets while effectiveness remains constant across both CPU-based SUTs.

The GPU SUT does not support the iterative implementation of the algorithm. However, the batch latency for the first batch of results from the baseline and CPU SUTs shows that the NetworkX implementation computes the results iteratively but does not yield them to the user until convergence.

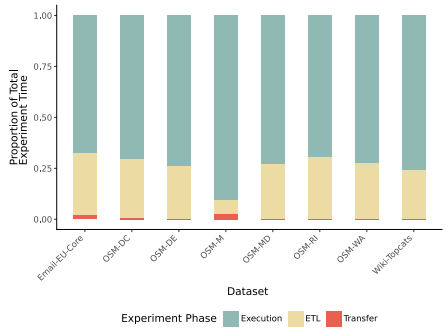
5.6.2.2 Implementation Stimulation Results

Figure 5.5 shows that all approaches have comparable effectiveness for static processing approaches, with the effectiveness varying relative to the input dataset and contradicting the hypothesis that scale-free input datasets would have a higher NMI than mesh input datasets.

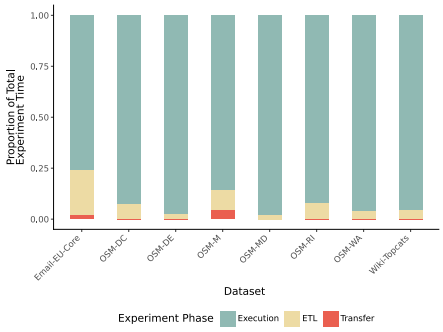
Figure 5.7 shows that the choice of hardware impacts the execution of the iterative algorithm, with no implementation for the GPU and erratic non-monotonic improvement identified for the baseline SUT.



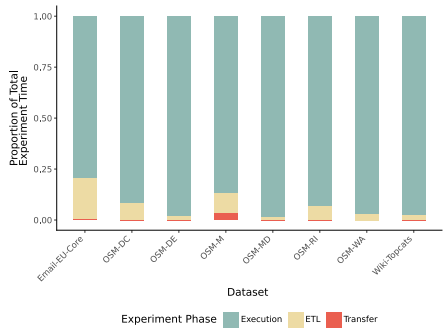
(a) Comparing SUTs across all systems shows GPUs spend a greater proportion of their time on ETL than CPU-based approaches.



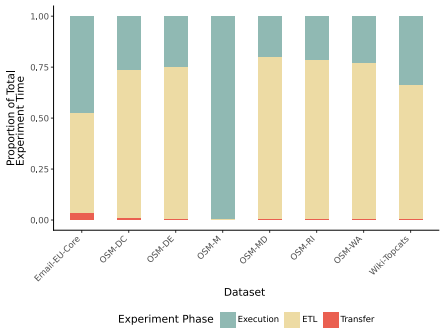
(b) Average time per phase across all datasets.



(d) Time per phase across all datasets for the CPU SUT.

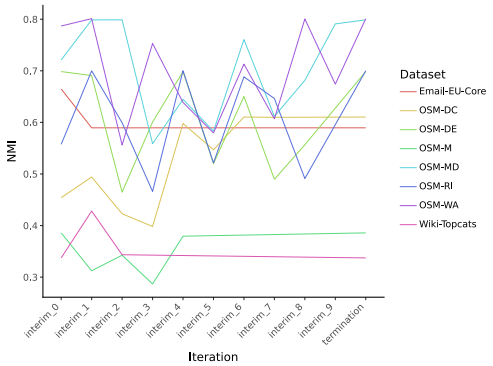


(c) Time per phase across all datasets for the Baseline SUT.

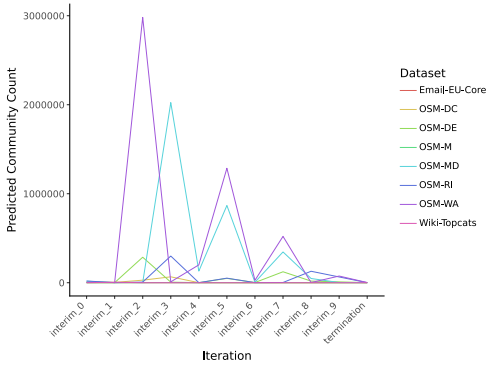


(e) Time per phase across all datasets for the GPU SUT

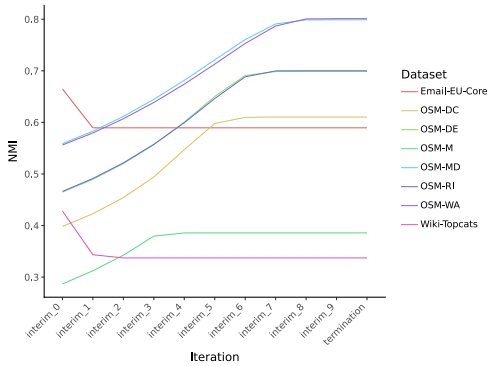
Figure 5.6: Porportion of a total experiment shows GPU spend more time on ETL relative to total Ex Time.



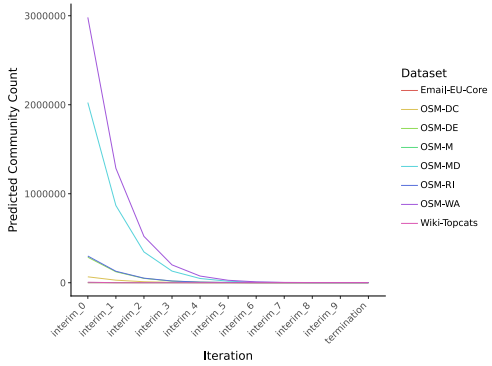
(a) Baseline NMI improves non-monotonically.



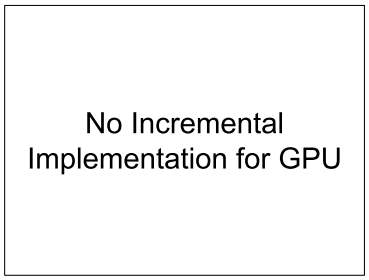
(b) Baseline community count shrinks non-monotonically.



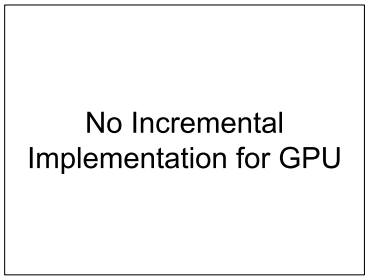
(c) Xeon SUT NMI improves monotonically.



(d) Xeon SUT community count decreases monotonically.



(e) The GPU SUT does not support iterative implementation.



(f) The GPU SUT does not support iterative implementation.

Figure 5.7: The underlying hardware impacts the performance of iterative algorithms, with only the Xeon CPU SUT showing monotonic improvement that allows trading effectiveness for efficiency.

5.7 Discussion

We make the following observations based on the results described in Section 5.6 and the context of the hypotheses stated in Section 5.4.

The NetworkX implementaion on the GPU SUT is much faster across all data shapes. Execution time increases proportional to the size of the graph, approximately quadratically to the number of vertices across all implementations. Our datasets are mostly mesh graphs where the number of $|E| \rightarrow |V|$. The OSM-WA dataset is the most extensive mesh graph, yet the OSM-MD mesh graph takes the longest to complete. Both have comparable edge densities, but the somewhat irregular shape of the state of Maryland likely yields a larger diameter, suggesting that the shape of the input graph has as much influence as the size. More experimentation is required with graphs of intermediate shapes between OSM-WA and OSM-MD to confirm this assessment. While mesh graphs do not represent typical real-world community structures, it shows that the assertion that the Louvain algorithm and its derivatives are 'near linear in practice [3, 163] is misleading.

Execution time dominates, but ETL time is non-trivial for small graphs and GPUs. Benchmarks typically focus on Ex Time because the execution phase is frequently the processing bottleneck, and the ETL costs in static graph processing are assumed to be amortized over the life of the graph. The results in Figure 5.6 show that for small graphs on CPU based systems, ETL can take up to 20% of total experiment time, and for GPU based systems, it can approach 75% on larger graphs. While the GPU results in part, emphasize the processing phase's speed, it also highlights the overhead incurred with transforming data and copying it to an external accelerator. We expect additional testing on larger datasets will show that GPU approaches will fail as the size

of the dataset exceeds the amount of memory available on the GPU. Further, as we implement support for dynamic graphs involving many small ETL activities over time, we expect ETL to take more significant proportions of the total experiment runtime, which will exacerbate the ETL overheads incurred by external accelerators as each timestep now has an ETL step and the cost is not hidden as effectively by the improvements in Ex Time.

The Louvain Partitions implementation does not yield incremental solutions. Reviewing the BL (Figure b) shows that the NetworkX implementation of Louvain Partitions does not yield an answer after each partition. Instead, it completes the algorithm and returns a generator containing all the partitions. To effectively measure the 'benefit' of an incremental solution through batch latency, an incremental algorithm must return results in real-time or significantly less than the total runtime of the algorithm. Without returning results in real-time, they provide no real ability for a user to decide on thresholds for early-stopping in their system and remove the option to trade effectiveness for efficiency as an optimization tactic.

The Louvain Partition implementation behaves inconsistently between SUT. One assumption about the benefit of an incremental or approximate approach to balancing effectiveness and efficiency is that it should improve monotonically. That is, the longer it runs, the better the solution gets. Figure 5.7 shows erratic variation in the size of communities and the NMI at each step for the baseline SUT, suggesting that we cannot make predictable trade-offs of efficiency and effectiveness on the baseline system. The GPU SUT does not support the iterative Louvain Partitions implementation. However, the CPU SUT exhibits the expected monotonic behavior, allowing users to trade-off between effectiveness and efficiency if the implementation is adjusted to output results in-stride, as an alternative approach to work within time constraints if a GPU is too costly.

The population segmentation benchmark supports hardware selection. Our experiments show how the engineer can determine that the current baseline implementation has some issues with the effectiveness that warrant moving quickly to a different solution and that the GPU will provide a significant speed-up on datasets at the tiny and small sizes. It also shows that the NetworkX Louvain Partitions implementation must be updated to trade effectiveness for efficiency if they wish to pursue this route rather than using a GPU. Knowing the empirical gains expected from procuring the hardware, the engineer can make an informed, data-driven decision about whether the cost of that hardware is justified for the use case. For a fully-informed view, the engineer should repeat the benchmarking process with the other GraphAI benchmark problems important to their team to understand the trade-offs that using a GPU based approach may require of GraphAI tasks.

The benchmark development methodology creates a useful tool for evaluating GraphAI systems. The population segmentation benchmark shows how the GraphAI benchmark development methodology can create a helpful component benchmark for the GraphAI benchmark framework. It shows how selecting the tasks and datasets allows us to manipulate the inputs to trigger observable behaviors that reveal something about the bottlenecks in the hardware, representation, and implementation acceleration techniques. As the benchmark continues to be used, we will develop more hypotheses to map the input graphs to expected outputs, given the presence of some combination of hardware, representation, and implementation acceleration techniques.

Chapter 6

Conclusion

6.1 Future Work

The GraphAI benchmark framework is in its infancy. It has many avenues for future work, including simple implementation and engineering challenges, the development of a complementary maturity model, support for dynamic inputs, cost measurement, and further development of constraint-based experimentation.

Engineering challenges. Many engineering and data collection problems must be addressed to build the benchmark system, including expanding the dataset coverage using synthetic data generators, adding a linear algebraic approach to CD, and testing the benchmark on a distributed SUT, a shared-memory parallel SUT and hardware-accelerated SUT.

A Benchmarking Maturity Model. After reviewing many benchmark approaches, we have identified that there appears to be an implicit maturity model in benchmark development. For a given field, early benchmarks focus on proving functionality. Over time, the focus shifts to improving the effectiveness relative to other solutions and, finally, to making solutions more efficient. Poorly constructed maturity models resulting from a related study and compiled without a formal methodology have gained negative attention recently [207, 208]. So, the formal development of a maturity model for benchmarking was scoped out of this thesis. However, it will be helpful for future work in investigating the correlation between benchmark availability and field maturity.

Dynamic Graph Processing. The current GraphAI benchmark can evaluate static and dynamic problems together but does not have the datasets or the transfer mechanism to feed datasets dynamically to the SUT. An important avenue of future work is to extend the GraphAI benchmark framework observation apparatus to support dynamic graphs, which we will implement with a 'push' based system configured in the same fashion as the current polling system.

Automated Hypothesis Evaluation. Our design process involves the development of hypotheses to guide the design of the investigation apparatus and collection of data. Creating an alert mechanism to notify a benchmark user when a hypothesis is violated is a valuable augmentation to the current approach. It will boost the utility of the benchmark and move it further away from being a tool for 'racing systems' to a platform for meaningful research.

Non-Invasive Cost Measurement. The major limitation of the GraphAI benchmark framework is its inability to measure cost accurately. Prior approaches focus on estimation using a cost model or selecting a proxy metric like power consumption as the basis of cost estimates. We must develop a method to measure cost from the Input/Output interface to add the third crucial performance metric that should be considered alongside efficiency and effectiveness. Alternatively, developing a method to characterize a remote system so that the benchmark framework can estimate costs for the system under test may be feasible by providing a micro benchmarking task that a SUT runs to provide the seed value for a predictive cost model. However, a micro benchmarking approach will break down when using bespoke hardware accelerators that may not support the benchmark as written or whose engineering breaks the assumptions of a cost model.

Constraint-Based Experimentation. Real-world GraphAI systems must operate under efficiency, effectiveness, and cost constraints. The experiments in Section 5.6 show how efficiency and effectiveness can be easily tracked and limited to simulate constrained environments. The

GraphAI benchmark system can be easily extended for exploratory experimentation to investigate how GraphAI will perform under specific constraints beyond what has been explored by the benchmark in terms of evaluating hardware, representation, and implementation variations. Extending the experiments will evolve the GraphAI benchmark framework into a platform for simulating more complicated 'what-if' scenarios, potentially simulating edge-computing, low Size Weight and Power (SWaP) thresholds, and other business or policy constraints.

6.2 Conclusion

This thesis has developed a 'grey-box' GraphAI benchmarking framework that fuses the empirical evaluation of algorithms with constraint theory to answer *if, how* and *when* to speed up GraphAI processing given real-world efficiency, effectiveness, and cost constraints. By exploring the landscape of GraphAI and identifying key challenges, we have formulated a novel approach to benchmarking that moves beyond traditional 'black-box' and 'white-box' methods.

Our findings highlight the hierarchy of challenges that afflict GraphAI , stemming from the foundational limitations of graph processing and impeding the fair evaluation of systems across the breadth of our GraphAI taxonomy. We show how a problem-centric view of benchmarking is more flexible than a kernel-centric view and that it supports static and dynamic approaches in a single framework while encouraging innovative problem-solving. Most importantly, we show how the 'grey-box' approach can use hypotheses to map input characteristics to expected outputs, giving benchmark designers the tools they need to detect the existence of bottlenecks in a System Under Test (SUT) without invasive system-level access.

We have laid the groundwork for a new approach to GraphAI problem-solving by advocat-

ing for a more holistic evaluation framework that recognizes a GraphAI user may need to balance efficiency, effectiveness, and cost when deploying systems to complete GraphAI tasks. We show how our approach would directly support an engineer designing a system architecture for solving GraphAI problems, providing a repeatable and extensible process to add new GraphAI problems as they become relevant to an organization's use case.

This thesis contributes literature reviews, methodological advancements, technical artifacts, and experimental findings to advance the field of GraphAI benchmarking. First, it synthesizes prior work by conducting four systematic literature reviews to develop the GraphAI taxonomy, the Intelligence and Security (I&S) use-case survey, the GraphAI benchmarking analysis, and the Community Detection (CD) bottleneck analysis. Second, it contributes the GraphAI benchmarking framework and the methodology for GraphAI benchmark problem development, which is a novel fusion of traditional benchmarking, the empirical analysis of algorithms, and constraint theory into a 'grey-box' approach. Third, our technical contributions include a data generator and the benchmark observation apparatus. The data generator creates directed, weighted, attributed mesh graphs with ground truth communities at multiple resolution levels using real-world map data drawn from the Open StreetMaps (OSM) database to fill an availability gap in real-world mesh graphs for evaluating CD. The observation apparatus is an Application Programming Interface (API) between a SUT and a Benchmark Server (BS) that implements a 'polling period' to sample answers over time and compare static to dynamic techniques in the context of trading efficiency for effectiveness. Finally, we contribute the specification for the GraphAI population segmentation benchmark and show how it can guide architectural decisions when designing and building GraphAI systems.

Looking ahead, there are several avenues for future research. Further refinement of the

benchmark framework, expansion to include additional GraphAI problems, and exploration of alternative evaluation metrics represent promising directions for future inquiry.

In summary, this thesis offers valuable insights into the challenges and opportunities of benchmarking GraphAI systems. We hope to catalyze continued progress in this rapidly evolving field by promoting transparency, reproducibility, and problem-centric evaluation.

Bibliography

[1] John N Hooker. Needed: An empirical science of algorithms. *Operations research*, 42(2):201–212, 1994.

[2] John N Hooker. Testing heuristics: We have it all wrong. *Journal of heuristics*, 1:33–42, 1995.

[3] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment*, 2008(10):P10008, 2008.

[4] Paddy Smith. What is graphai. *The Artificial Intelligence Magazine*, 2021.

[5] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. Graph neural networks: A review of methods and applications. *AI open*, 1:57–81, 2020.

[6] Feng Xia, Ke Sun, Shuo Yu, Abdul Aziz, Liangtian Wan, Shirui Pan, and Huan Liu. Graph learning: A survey. *IEEE Transactions on Artificial Intelligence*, 2(2):109–127, 2021.

[7] Xuhao Chen. Graph ai: Graph mining, graph machine learning, and graph neural networks. *MIT CSAIL*, 2021.

[8] Jóakim V Kistowski, Jeremy A Arnold, Karl Huppler, Klaus-Dieter Lange, John L Henning, and Paul Cao. How to build a benchmark. In *Proceedings of the 6th ACM/SPEC international conference on performance engineering*, pages 333–336, 2015.

[9] Andrea Lancichinetti, Santo Fortunato, and Filippo Radicchi. Benchmark graphs for testing community detection algorithms. *Physical review E*, 78(4):046110, 2008.

[10] Seiji Maekawa, Yuya Sasaki, George Fletcher, and Makoto Onizuka. Gencat: Generating attributed graphs with controlled relationships between classes, attributes, and topology. *Information Systems*, 115:102195, 2023.

[11] Xuhao Chen, Roshan Dathathri, Gurbinder Gill, and Keshav Pingali. Pangolin: An efficient and flexible graph mining system on cpu and gpu. *Proceedings of the VLDB Endowment*, 13(8):1190–1205, 2020.

[12] Kathrin Blagec, Adriano Barbosa-Silva, Simon Ott, and Matthias Samwald. A curated, ontology-based, large-scale knowledge graph of artificial intelligence tasks and benchmarks. *Scientific Data*, 9(1):322, 2022.

[13] Chitu Okoli. A guide to conducting a standalone systematic literature review. *Communications of the Association for Information Systems*, 37, 2015.

[14] Stuart Russell and Peter Norvig. Artificial intelligence: a modern approach, 4th us ed. *University of California, Berkeley*, 2021.

[15] Li Zeng, Yan Jiang, Weixin Lu, and Lei Zou. Deep analysis on subgraph isomorphism. *arXiv preprint arXiv:2012.06802*, 2020.

[16] Kent O'Sullivan, Nicole R Schneider, and Hanan Samet. Compass: Cardinal orientation maniulation and pattern aware spatial search. In Li Hao, Gabriele Cavallaro, Dora Blanco Heras, Dalton Lunga, Martin Werner, and Andreas Züfle, editors, *GeoSearch'23: Proceedings of the 2nd ACM SIGSPATIAL International Workshop on Searching and Mining Large Collections of Geospatial Data*, 2023.

[17] Michelle Girvan and Mark EJ Newman. Community structure in social and biological networks. *Proceedings of the national academy of sciences*, 99(12):7821–7826, 2002.

[18] Ulrik Brandes, Daniel Delling, Marco Gaertler, Robert Görke, Martin Hoefer, Zoran Nikoloski, and Dorothea Wagner. Maximizing modularity is hard. *arXiv preprint physics/0608255*, 2006.

[19] Lawrence Page. The pagerank citation ranking: Bringing order to the web. technical report. *Stanford Digital Library Technologies Project, 1998*, 1998.

[20] Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of mathematical sociology*, 25(2):163–177, 2001.

[21] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. `http://snap.stanford.edu/data`, jun 2014.

[22] Luc De Raedt, Kristian Kersting, Sriraam Natarajan, and David Poole. Statistical relational artificial intelligence: Logic, probability, and computation. *Synthesis lectures on artificial intelligence and machine learning*, 10(2):1–189, 2016.

[23] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and S Yu Philip. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE transactions on neural networks and learning systems*, 33(2):494–514, 2021.

[24] Alexander Maedche and Steffen Staab. Ontology learning. In *Handbook on ontologies*, pages 173–190. Springer, 2004.

[25] Xiaojun Chen, Shengbin Jia, and Yang Xiang. A review: Knowledge reasoning over knowledge graph. *Expert Systems with Applications*, 141:112948, 2020.

[26] Antoine Bosselut, Hannah Rashkin, Maarten Sap, Chaitanya Malaviya, Asli Celikyilmaz, and Yejin Choi. Comet: Commonsense transformers for automatic knowledge graph construction. *arXiv preprint arXiv:1906.05317*, 2019.

[27] Aleksandr Perevalov, Xi Yan, Liubov Kovriguina, Longquan Jiang, Andreas Both, and Ricardo Usbeck. Knowledge graph question answering leaderboard: A community resource to prevent a replication crisis. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 2998–3007, 2022.

[28] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016.

[29] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 701–710, 2014.

[30] Yuanqi Du, Shiyu Wang, Xiaojie Guo, Hengning Cao, Shujie Hu, Junji Jiang, Aishwarya Varala, Abhinav Angirekula, and Liang Zhao. Graphgt: Machine learning datasets for graph generation and transformation. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.

[31] Angela Bonifati, Irena Holubová, Arnau Prat-Pérez, and Sherif Sakr. Graph generators: State of the art and open challenges. *ACM computing surveys (CSUR)*, 53(2):1–30, 2020.

[32] Anton Tsitsulin, Benedek Rozemberczki, John Palowitch, and Bryan Perozzi. Synthetic graph generation to benchmark graph learning. *arXiv preprint arXiv:2204.01376*, 2022.

[33] Seiji Maekawa, Jianpeng Zhang, George Fletcher, and Makoto Onizuka. General generator for attributed graphs with community structure. In *Proceeding of the ECML/PKDD graph embedding and mining workshop*, pages 1–5, 2019.

[34] Kiarash Zahirnia, Yaochen Hu, Mark Coates, and Oliver Schulte. Neural graph generation from graph statistics. *Advances in Neural Information Processing Systems*, 36, 2024.

[35] Yukuo Cen, Zhenyu Hou, Yan Wang, Qibin Chen, Yizhen Luo, Zhongming Yu, Hengrui Zhang, Xingcheng Yao, Aohan Zeng, Shiguang Guo, et al. Cogdl: A comprehensive library for graph deep learning. In *Proceedings of the ACM Web Conference 2023*, pages 747–758, 2023.

[36] Trinayan Baruah, Kaustubh Shivdikar, Shi Dong, Yifan Sun, Saiful A Mojumder, Kihoon Jung, José L. Abellán, Yash Ukidave, Ajay Joshi, John Kim, and David Kaeli. Gnnmark: A benchmark suite to characterize graph neural network training on gpus. In *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 13–23, 2021.

[37] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems*, 32(1):4–24, 2020.

[38] Hao Yuan, Jiliang Tang, Xia Hu, and Shuiwang Ji. Xgnn: Towards model-level explanations of graph neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 430–438, 2020.

[39] Bastian Pfeifer, Anna Saranti, and Andreas Holzinger. Gnn-subnet: disease subnetwork detection with explainable graph neural networks. *Bioinformatics*, 38(Supplement_2):ii120–ii126, 2022.

[40] Doug Lenat and Gary Marcus. Getting from generative ai to trustworthy ai: What llms might learn from cyc. *arXiv preprint arXiv:2308.04445*, 2023.

[41] Wenhao Yu. Retrieval-augmented generation across heterogeneous knowledge. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Student Research Workshop*, pages 52–58, 2022.

[42] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, and Bin Cui. Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473*, 2024.

[43] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, pages 1–20, 2024.

[44] Dan Hendrycks, Nicholas Carlini, John Schulman, and Jacob Steinhardt. Unsolved problems in ml safety. *arXiv preprint arXiv:2109.13916*, 2021.

[45] Ilaria Tiddi, Freddy Lécué, and Pascal Hitzler. *Knowledge graphs for explainable artificial intelligence: Foundations, applications and challenges*. IOS Press, 2020.

[46] Ilaria Tiddi and Stefan Schlobach. Knowledge graphs as tools for explainable machine learning: A survey. *Artificial Intelligence*, 302:103627, 2022.

[47] Amit Sheth, Kaushik Roy, and Manas Gaur. Neurosymbolic artificial intelligence (why, what, and how). *IEEE Intelligent Systems*, 38(3):56–62, 2023.

[48] William Regli, Peter Loats, Laurel Miller-Sims, Ben Johnson, Jacob Bunker, Jared Ott, and Christine Herlihy. Operationalizing emerging hardware for ai applications: A survey of transition opportunities and datasets. Technical report, 2022.

[49] Jennifer J Xu and Hsinchun Chen. Fighting organized crimes: using shortest-path algorithms to identify associations in criminal networks. *Decision Support Systems*, 38(3):473–487, 2004.

[50] Abdullah Sawas, Abdullah Abuolaim, Mahmoud Afifi, and Manos Papagelis. A versatile computational framework for group pattern mining of pedestrian trajectories. *GeoInformatica*, 23:501–531, 2019.

[51] Andreas Tritsarolis, George-Stylianos Theodoropoulos, and Yannis Theodoridis. Online discovery of co-movement patterns in mobility data. *International Journal of Geographical Information Science*, 35(4):819–845, 2021.

[52] Lu-An Tang, Yu Zheng, Jing Yuan, Jiawei Han, Alice Leung, Wen-Chih Peng, and Thomas La Porta. A framework of traveling companion discovery on trajectory data streams. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 5(1):1–34, 2014.

[53] Xiongbin Wang, Chen Liu, and Meiling Zhu. Instant traveling companion discovery based on traffic-monitoring streaming data. In *2016 13th Web Information Systems and Applications Conference (WISA)*, pages 89–94. IEEE, 2016.

[54] Kent O'Sullivan, Nicole R Schneider, Aleeza Rasheed, and Hanan Samet. Gestalt: Geospatially enhanced search with terrain augmented location targeting. In Li Hao, Gabriele Cavallaro, Dora Blanco Heras, Dalton Lunga, Martin Werner, and Andreas Züfle, editors, *GeoSearch'23: Proceedings of the 2nd ACM SIGSPATIAL International Workshop on Searching and Mining Large Collections of Geospatial Data*, 2023.

[55] Department of the Army. *ADP 3-13 Information*. Department of Defense, 2023.

[56] Wayne W Zachary. An information flow model for conflict and fission in small groups. *Journal of anthropological research*, 33(4):452–473, 1977.

[57] Stanislav Sobolevsky, Riccardo Campari, Alexander Belyi, and Carlo Ratti. General optimization technique for high-quality community detection in complex networks. *Physical Review E*, 90(1):012811, 2014.

[58] Shima D Keene et al. Operationalizing counter threat finance strategies. *The Letort Papers*, 2014.

[59] Anastasios Balaskas and Virginia NL Franqueira. Analytical tools for blockchain: Review, taxonomy and open challenges. In *2018 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, pages 1–8. IEEE, 2018.

[60] Daniel Goldsmith, Kim Grauer, and Yonah Shmalo. Analyzing hack subnetworks in the bitcoin transaction graph. *Applied Network Science*, 5(1):1–20, 2020.

[61] Krzysztof Michalak and Jerzy Korczak. Graph mining approach to suspicious transaction detection. In *2011 Federated conference on computer science and information systems (FedCSIS)*, pages 69–75. IEEE, 2011.

[62] Reza Soltani, Uyen Trang Nguyen, Yang Yang, Mohammad Faghani, Alaa Yagoub, and Aijun An. A new algorithm for money laundering detection based on structural similarity. In *2016 IEEE 7th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, pages 1–7, 2016.

[63] TRADOC Intelligence Support Activity-Threats Fort Leavenworth KS. *A Military Guide to Terrorism in the Twenty-First Century*. Department of Defense, 2007.

[64] Eric M Hutchins, Michael J Cloppert, Rohan M Amin, et al. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. *Leading Issues in Information Warfare & Security Research*, 1(1):80, 2011.

[65] Lukáš Sadlek, Pavel Čeleda, and Daniel Tovarňák. Identification of attack paths using kill chain and attack graphs. In *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, pages 1–6. IEEE, 2022.

[66] Kent O'Sullivan and Benjamin Turnbull. The cyber simulation terrain: Towards an open source cyber effects simulation ontology. In *Proceedings of the 11th Annual Australian Information Warfare Conference*. Security Research Institute, Edith Cowan University, 2015.

[67] Department of the Army. *ADP 2-0 Intelligence*. Department of Defense, 2019.

[68] Pedro Szekely, Craig A Knoblock, Jason Slepicka, Andrew Philpot, Amandeep Singh, Chengye Yin, Dipsy Kapoor, Prem Natarajan, Daniel Marcu, Kevin Knight, et al. Building and using a knowledge graph to combat human trafficking. In *The Semantic Web-ISWC 2015: 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015, Proceedings, Part II 14*, pages 205–221. Springer, 2015.

[69] Yulei Wu, Hong-Ning Dai, and Haina Tang. Graph neural networks for anomaly detection in industrial internet of things. *IEEE Internet of Things Journal*, 9(12):9214–9231, 2021.

[70] Lingfei Ren, Ruimin Hu, Dengshi Li, Yang Liu, Junhang Wu, Yilong Zang, and Wenyi Hu. Dynamic graph neural network-based fraud detectors against collaborative fraudsters. *Knowledge-Based Systems*, 278:110888, 2023.

[71] Mark S Granovetter. The strength of weak ties. *American journal of sociology*, 78(6):1360–1380, 1973.

[72] Daniel Carmody, Martina Mazzarello, Paolo Santi, Trevor Harris, Sune Lehmann, Timur Abbiasov, Robin Dunbar, and Carlo Ratti. The effect of co-location on human communication networks. *Nature Computational Science*, 2(8):494–503, 2022.

[73] Micaela Lucia Bangerter, Giuseppe Fenza, Mariacristina Gallo, Vincenzo Loia, Alessandra Petrone, and Alberto Volpe. Terrorist organization identification using link prediction over heterogeneous gnn. *Human-centric Computing and Information Sciences. 12: 1-13 (www. hcisjournal. com)*, 2022.

[74] a Ahmadi, Hoang H Nguyen, Zijian Zhang, Dmytro Bozhkov, Daniel Kudenko, Maria Jofre, Francesco Calderoni, Noa Cohen, and Yosef Solewicz. Inductive and transductive link prediction for criminal network analysis. *Journal of Computational Science*, page 102063, 2023.

[75] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *science*, 286(5439):509–512, 1999.

[76] Scott Beamer, Krste Asanovic, and David Patterson. Locality exists in graph processing: Workload characterization on an ivy bridge server. In *2015 IEEE International Symposium on Workload Characterization*, pages 56–65. IEEE, 2015.

[77] Paul Erdős, Alfréd Rényi, et al. On the evolution of random graphs. *Publ. math. inst. hung. acad. sci*, 5(1):17–60, 1960.

[78] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. R-mat: A recursive model for graph mining. In *Proceedings of the 2004 SIAM International Conference on Data Mining*, pages 442–446. SIAM, 2004.

[79] Jure Leskovec, Deepayan Chakrabarti, Jon Kleinberg, Christos Faloutsos, and Zoubin Ghahramani. Kronecker graphs: an approach to modeling networks. *Journal of Machine Learning Research*, 11(2), 2010.

[80] Aydin Buluç, Jeremy T Fineman, Matteo Frigo, John R Gilbert, and Charles E Leiserson. Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks. In *Proceedings of the twenty-first annual symposium on Parallelism in algorithms and architectures*, pages 233–244, 2009.

[81] Julian Shun, Laxman Dhulipala, and Guy E Blelloch. Smaller and faster: Parallel processing of compressed graphs with ligra+. In *2015 Data Compression Conference*, pages 403–412. IEEE, 2015.

[82] Robert Ryan McCune, Tim Weninger, and Greg Madey. Thinking like a vertex: A survey of vertex-centric frameworks for large-scale distributed graph processing. *ACM Computing Surveys (CSUR)*, 48(2):1–39, 2015.

[83] Grzegorz Malewicz, Matthew H Austern, Aart JC Bik, James C Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: a system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 135–146, 2010.

[84] Sherif Sakr, Faisal Moeen Orakzai, Ibrahim Abdelaziz, and Zuhair Khayyat. *Large-scale graph processing using Apache Giraph*. Springer, 2016.

[85] Hang Liu and H Howie Huang. {SIMD-X}: Programming and processing of graph algorithms on {GPUs}. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 411–428, 2019.

[86] Amitabha Roy, Ivo Mihailovic, and Willy Zwaenepoel. X-stream: Edge-centric graph processing using streaming partitions. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 472–488, 2013.

[87] Joseph E Gonzalez, Reynold S Xin, Ankur Dave, Daniel Crankshaw, Michael J Franklin, and Ion Stoica. {GraphX}: Graph processing in a distributed dataflow framework. In *11th USENIX symposium on operating systems design and implementation (OSDI 14)*, pages 599–613, 2014.

[88] Miguel E Coimbra, Alexandre P Francisco, and Luís Veiga. An analysis of the graph processing landscape. *journal of Big Data*, 8(1):55, 2021.

[89] Vasiliki Kalavri, Vladimir Vlassov, and Seif Haridi. High-level programming abstractions for distributed graph processing. *IEEE Transactions on Knowledge and Data Engineering*, 30(2):305–324, 2017.

[90] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D Owens. Gunrock: A high-performance graph processing library on the gpu. In *Proceedings of the 21st ACM SIGPLAN symposium on principles and practice of parallel programming*, pages 1–12, 2016.

[91] Laxman Dhulipala, Guy Blelloch, and Julian Shun. Julienne: A framework for parallel graph algorithms using work-efficient bucketing. In *Proceedings of the 29th ACM Symposium on Parallelism in Algorithms and Architectures*, pages 293–304, 2017.

[92] Aydın Buluç and John R Gilbert. The combinatorial blas: Design, implementation, and applications. *The International Journal of High Performance Computing Applications*, 25(4):496–509, 2011.

[93] Ariful Azad, Oguz Selvitopi, Md Taufique Hussain, John R Gilbert, and Aydın Buluç. Combinatorial blas 2.0: Scaling combinatorial algorithms on distributed-memory systems. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):989–1001, 2021.

[94] Tim Mattson, David Bader, Jon Berry, Aydin Buluc, Jack Dongarra, Christos Faloutsos, John Feo, John Gilbert, Joseph Gonzalez, Bruce Hendrickson, et al. Standards for graph algorithm primitives. In *2013 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–2. IEEE, 2013.

[95] Jeremy Kepner, David Bader, Aydın Buluç, John Gilbert, Timothy Mattson, and Henning Meyerhenke. Graphs, matrices, and the graphblas: Seven good reasons. *Procedia Computer Science*, 51:2453–2462, 2015.

[96] Jeremy Kepner, Peter Aaltonen, David Bader, Aydin Buluç, Franz Franchetti, John Gilbert, Dylan Hutchison, Manoj Kumar, Andrew Lumsdaine, Henning Meyerhenke, et al. Mathematical foundations of the graphblas. In *2016 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–9. IEEE, 2016.

[97] Narayanan Sundaram, Nadathur Satish, Md Mostofa Ali Patwary, Subramanya R Dulloor, Michael J Anderson, Satya Gautam Vadlamudi, Dipankar Das, and Pradeep Dubey. Graphmat: High performance graph analytics made productive. *Proceedings of the VLDB Endowment*, 8(11), 2015.

[98] Ariful Azad, Mohsen Mahmoudi Aznaveh, Scott Beamer, Mark Blanco, Jinhao Chen, Luke D'Alessandro, Roshan Dathathri, Tim Davis, Kevin Deweese, Jesun Firoz, et al. Evaluation of graph analytics frameworks using the gap benchmark suite. In *2020 IEEE International Symposium on Workload Characterization (IISWC)*, pages 216–227. IEEE, 2020.

[99] Nadathur Satish, Narayanan Sundaram, Md Mostofa Ali Patwary, Jiwon Seo, Jongsoo Park, M Amber Hassaan, Shubho Sengupta, Zhaoming Yin, and Pradeep Dubey. Navigating the maze of graph analytics frameworks using massive graph datasets. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, pages 979–990, 2014.

[100] Julian Shun and Guy E Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*, pages 135–146, 2013.

[101] Laxman Dhulipala, Guy E Blelloch, and Julian Shun. Theoretically efficient parallel graph algorithms can be fast and scalable. *ACM Transactions on Parallel Computing (TOPC)*, 8(1):1–70, 2021.

[102] Aleksa Vukotic, Nicki Watt, Tareq Abedrabbo, Dominic Fox, and Jonas Partner. *Neo4j in action*, volume 22. Manning Shelter Island, 2015.

[103] John L. and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition, 2019.

[104] I-Hsin Chung, Changhoan Kim, Hui-Fang Wen, and Guojing Cong. Application data prefetching on the ibm blue gene/q supercomputer. In *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–8. IEEE, 2012.

[105] Aydin Buluç, Erika Duriakova, Armando Fox, John R Gilbert, Shoaib Kamil, Adam Lugowski, Leonid Oliker, and Samuel Williams. High-productivity and high-performance analysis of filtered semantic graphs. In *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*, pages 237–248. IEEE, 2013.

[106] Abanti Basak, Shuangchen Li, Xing Hu, Sang Min Oh, Xinfeng Xie, Li Zhao, Xiaowei Jiang, and Yuan Xie. Analysis and optimization of the memory hierarchy for graph processing workloads. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 373–386. IEEE, 2019.

[107] Jack J Dongarra, Piotr Luszczek, and Antoine Petitet. The linpack benchmark: past, present and future. *Concurrency and Computation: practice and experience*, 15(9):803–820, 2003.

[108] Holger H Hoos and Thomas Stützle. Satlib: An online resource for research on sat. *Sat*, 2000:283–292, 2000.

[109] Richard C Murphy, Kyle B Wheeler, Brian W Barrett, and James A Ang. Introducing the graph 500. *Cray Users Group (CUG)*, 19:45–74, 2010.

[110] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014.

[111] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6904–6913, 2017.

[112] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2024.

[113] Adriano Barbosa-Silva, Simon Ott, Kathrin Blagec, Jan Brauner, and Matthias Samwald. Mapping global dynamics of benchmark creation and saturation in artificial intelligence. *arXiv preprint arXiv:2203.04592*, 2022.

[114] Rafael H Saavedra and Alan J Smith. Analysis of benchmark characteristics and benchmark performance prediction. *ACM Transactions on Computer Systems (TOCS)*, 14(4):344–384, 1996.

[115] Aaron Stump, Geoff Sutcliffe, and Cesare Tinelli. Starexec: A cross-community infrastructure for logic solving. In *International joint conference on automated reasoning*, pages 367–373. Springer, 2014.

[116] Sewon Min, Jordan Boyd-Graber, Chris Alberti, Danqi Chen, Eunsol Choi, Michael Collins, Kelvin Guu, Hannaneh Hajishirzi, Kenton Lee, Jennimaria Palomaki, Colin Raffel, Adam Roberts, Tom Kwiatkowski, Patrick Lewis, Yuxiang Wu, Heinrich Küttler, Linqing Liu, Pasquale Minervini, Pontus Stenetorp, Sebastian Riedel, Sohee Yang, Minjoon Seo, Gautier Izacard, Fabio Petroni, Lucas Hosseini, Nicola De Cao, Edouard Grave, Ikuya Yamada, Sonse Shimaoka, Masatoshi Suzuki, Shumpei Miyawaki, Shun Sato, Ryo Takahashi, Jun Suzuki, Martin Fajcik, Martin Docekal, Karel Ondrej, Pavel Smrz, Hao Cheng, Yelong Shen, Xiaodong Liu, Pengcheng He, Weizhu Chen, Jianfeng Gao, Barlas Oguz, Xilun Chen, Vladimir Karpukhin, Stan Peshterliev, Dmytro Okhonko, Michael Schlichtkrull, Sonal Gupta, Yashar Mehdad, and Wen-tau Yih. Neurips 2020 efficientqa competition: Systems, analyses and lessons learned. In Hugo Jair Escalante and Katja Hofmann, editors, *Proceedings of the NeurIPS 2020 Competition and Demonstration Track*, volume 133 of *Proceedings of Machine Learning Research*, pages 86–111. PMLR, 06–12 Dec 2021.

[117] Julian Shun, Guy E Blelloch, Jeremy T Fineman, Phillip B Gibbons, Aapo Kyrola, Harsha Vardhan Simhadri, and Kanat Tangwongsan. Brief announcement: The problem based benchmark suite. In *Proceedings of the twenty-fourth annual ACM symposium on Parallelism in algorithms and architectures*, pages 68–70, 2012.

[118] Jure Leskovec and Rok Sosič. Snap: A general-purpose network analysis and graph-mining library. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 8(1):1–20, 2016.

[119] Mihai Capotă, Tim Hegeman, Alexandru Iosup, Arnau Prat-Pérez, Orri Erling, and Peter Boncz. Graphalytics: A big data benchmark for graph-processing platforms. In *Proceedings of the GRADES'15*, pages 1–6. 2015.

[120] Alexandru Iosup, Tim Hegeman, Wing Lung Ngai, Stijn Heldens, Arnau Prat-Pérez, Thomas Manhardto, Hassan Chafio, Mihai Capotă, Narayanan Sundaram, Michael An-

derson, et al. Ldbc graphalytics: A benchmark for large-scale graph analysis on parallel and distributed platforms. *Proceedings of the VLDB Endowment*, 9(13):1317–1328, 2016.

[121] Scott Beamer, Krste Asanović, and David Patterson. The gap benchmark suite. *arXiv preprint arXiv:1508.03619*, 2017.

[122] Abanti Basak, Jilan Lin, Ryan Lorica, Xinfeng Xie, Zeshan Chishti, Alaa Alameldeen, and Yuan Xie. Saga-bench: Software and hardware characterization of streaming graph analytics workloads. In *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 12–23. IEEE, 2020.

[123] Laxman Dhulipala, Jessica Shi, Tom Tseng, Guy E Blelloch, and Julian Shun. The graph based benchmark suite (gbbs). In *Proceedings of the 3rd Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, pages 1–8, 2020.

[124] Daniel Anderson, Guy E Blelloch, Laxman Dhulipala, Magdalen Dobson, and Yihan Sun. The problem-based benchmark suite (pbbs), v2. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 445–447, 2022.

[125] Alexandru Iosup, Ahmed Musaafir, Alexandru Uta, Arnau Prat Pérez, Gábor Szárnyas, Hassan Chafi, Ilie Gabriel Tănase, Lifeng Nai, Michael Anderson, Mihai Capotă, et al. The ldbc graphalytics benchmark version 1.0.5. *arXiv preprint arXiv:2011.15028*, 2023.

[126] Ying Zhang, Pham Minh Duc, Oscar Corcho, and Jean-Paul Calbimonte. Srbench: a streaming rdf/sparql benchmark. In *The Semantic Web–ISWC 2012: 11th International Semantic Web Conference, Boston, MA, USA, November 11-15, 2012, Proceedings, Part I 11*, pages 641–657. Springer, 2012.

[127] Renzo Angles, Arnau Prat-Pérez, David Dominguez-Sal, and Josep-Lluis Larriba-Pey. Benchmarking database systems for social network applications. In *First International Workshop on Graph Data Management Experiences and Systems*, pages 1–7, 2013.

[128] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. The ldbc social network benchmark: Interactive workload. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 619–630, 2015.

[129] McKnight Consulting (Sponsored by Stardog). Trillion edge knowledge graph. Technical report, Jun 2021. Accessed 04 Sep 2023, Sponsored by Stardog.

[130] Ante Javor. Memgraph vs. neo4j: A performance comparison. blog, Apr 2022. Accessed 04 Sep 23.

[131] Ante Javor. Introduction to benchgraph and its architecture. blog, Apr 2023. Accessed 04 Sep 23.

[132] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33:22118–22133, 2020.

[133] Junwei Wu, Jingwei Sun, Hao Sun, and Guangzhong Sun. Performance analysis of graph neural network frameworks. In *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 118–127. IEEE, 2021.

[134] Kezhao Huang, Jidong Zhai, Zhen Zheng, Youngmin Yi, and Xipeng Shen. Understanding and bridging the gaps in current gnn performance optimizations. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 119–132, 2021.

[135] Vijay Prakash Dwivedi, Ladislav Rampášek, Michael Galkin, Ali Parviz, Guy Wolf, Anh Tuan Luu, and Dominique Beaini. Long range graph benchmark. *Advances in Neural Information Processing Systems*, 35:22326–22340, 2022.

[136] Vijay Prakash Dwivedi, Chaitanya K Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Benchmarking graph neural networks. *Journal of Machine Learning Research*, 24(43):1–48, 2023.

[137] Zhiyao Zhou, Sheng Zhou, Bochao Mao, Xuanyi Zhou, Jiawei Chen, Qiaoyu Tan, Daochen Zha, Yan Feng, Chun Chen, and Can Wang. Opengsl: A comprehensive benchmark for graph structure learning. *Advances in Neural Information Processing Systems*, 36, 2024.

[138] Ryan A. Rossi and Nesreen K. Ahmed. The network data repository with interactive graph analytics and visualization. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, 2015.

[139] David A Bader, John Feo, John Gilbert, Jeremy Kepner, David Koester, Eugene Loh, Kamesh Madduri, Bill Mann, Theresa Meuse, and Eric Robinson. Hpc scalable graph analysis benchmark. *Citeseer. Citeseer*, 2009:1–10, 2009.

[140] Intel. A hierarchical identify verify exploit (hive) project: Programmable and integrated unified memory architecture (piuma) workload analysis document. Technical report, Intel Corporation, 2020.

[141] Eliyahu M Goldratt and Jeff Cox. *The goal: a process of ongoing improvement*. Routledge, third revised edition edition, 2004.

[142] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. A scalable processing-in-memory accelerator for parallel graph processing. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, pages 105–117, 2015.

[143] Zhisong Fu, Michael Personick, and Bryan Thompson. Mapgraph: A high level api for fast development of high performance graph analytics on gpus. In *Proceedings of workshop on GRAph data management experiences and systems*, pages 1–6, 2014.

[144] Aydin Buluç and John R Gilbert. Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments. *SIAM Journal on Scientific Computing*, 34(4):C170–C191, 2012.

[145] Sriram Aananthakrishnan, Nesreen K Ahmed, Vincent Cave, Marcelo Cintra, Yigit Demir, Kristof Du Bois, Stijn Eyerman, Joshua B Fryman, Ivan Ganev, Wim Heirman, et al. Piuma: programmable integrated unified memory architecture. *arXiv preprint arXiv:2010.06277*, 2020.

[146] Yunming Zhang, Vladimir Kiriansky, Charith Mendis, Saman Amarasinghe, and Matei Zaharia. Making caches work for graph analytics. In *2017 IEEE International Conference on Big Data (Big Data)*, pages 293–302. IEEE, 2017.

[147] William S Song, Vitaliy Gleyzer, Alexei Lomakin, and Jeremy Kepner. Novel graph processor architecture, prototype system, and results. In *2016 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–7. IEEE, 2016.

[148] Svilen Kanev, Juan Pablo Darago, Kim Hazelwood, Parthasarathy Ranganathan, Tipp Moseley, Gu-Yeon Wei, and David Brooks. Profiling a warehouse-scale computer. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, pages 158–169, 2015.

[149] Tayo Oguntebi and Kunle Olukotun. Graphops: A dataflow library for graph analytics acceleration. In *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 111–117, 2016.

[150] Tae Jun Ham, Lisa Wu, Narayanan Sundaram, Nadathur Satish, and Margaret Martonosi. Graphicionado: A high-performance and energy-efficient accelerator for graph analytics. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–13. IEEE, 2016.

[151] Anurag Mukkara, Nathan Beckmann, Maleen Abeydeera, Xiaosong Ma, and Daniel Sanchez. Exploiting locality in graph analytics through hardware-accelerated traversal scheduling. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–14. IEEE, 2018.

[152] Timothy Dysart, Peter Kogge, Martin Deneroff, Eric Bovell, Preston Briggs, Jay Brockman, Kenneth Jacobsen, Yujen Juan, Shannon Kuntz, Richard Lethin, et al. Highly scalable near memory processing with migrating threads on the emu system architecture. In *2016 6th Workshop on Irregular Applications: Architecture and Algorithms (IA3)*, pages 2–9. IEEE, 2016.

[153] Vidushi Dadu, Sihao Liu, and Tony Nowatzki. Polygraph: Exposing the value of flexibility for graph processing accelerators. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 595–608. IEEE, 2021.

[154] Muhammet Mustafa Ozdal, Serif Yesil, Taemin Kim, Andrey Ayupov, John Greth, Steven Burns, and Ozcan Ozturk. Graph analytics accelerators for cognitive systems. *IEEE Micro*, 37(1):42–51, 2017.

[155] Mingyu Yan, Lei Deng, Xing Hu, Ling Liang, Yujing Feng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. Hygcn: A gcn accelerator with hybrid architecture. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 15–29. IEEE, 2020.

[156] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. *A Modern Primer on Processing in Memory*, pages 171–243. Springer Nature Singapore, Singapore, 2023.

[157] Kevin Kiningham, Philip Levis, and Christopher Ré. Grip: A graph neural network accelerator architecture. *IEEE Transactions on Computers*, 72(4):914–925, 2022.

[158] Rong Chen, Jiaxin Shi, Yanzhe Chen, Binyu Zang, Haibing Guan, and Haibo Chen. Powerlyra: Differentiated graph computation and partitioning on skewed graphs. *ACM Transactions on Parallel Computing (TOPC)*, 5(3):1–39, 2019.

[159] Luiz CR Carpinetti and Alexandre M De Melo. What to benchmark? a systematic approach and cases. *Benchmarking: An International Journal*, 9(3):244–255, 2002.

[160] Karl Huppler. The art of building a good benchmark. In *Technology conference on performance evaluation and benchmarking*, pages 18–30. Springer, 2009.

[161] Eliyahu M Goldratt. *Theory of constraints*. North River Croton-on-Hudson, 1990.

[162] Santo Fortunato and Marc Barthelemy. Resolution limit in community detection. *Proceedings of the national academy of sciences*, 104(1):36–41, 2007.

[163] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific reports*, 9(1):5233, 2019.

[164] Alexander Strehl and Joydeep Ghosh. Cluster ensembles—a knowledge reuse framework for combining multiple partitions. *Journal of machine learning research*, 3(Dec):583–617, 2002.

[165] Giulio Rossetti, Luca Pappalardo, and Salvatore Rinzivillo. A novel approach to evaluate community detection algorithms on ground truth. In *Complex Networks VII: Proceedings of the 7th Workshop on Complex Networks CompleNet 2016*, pages 133–144. Springer, 2016.

[166] Giulio Rossetti. Rdyn: graph benchmark handling community dynamics. *Journal of Complex Networks*, 5(6):893–912, 2017.

[167] Giulio Rossetti and Rémy Cazabet. Community discovery in dynamic networks: a survey. *ACM computing surveys (CSUR)*, 51(2):1–37, 2018.

[168] Mark EJ Newman and Michelle Girvan. Finding and evaluating community structure in networks. *Physical review E*, 69(2):026113, 2004.

[169] Jure Leskovec, Kevin J Lang, and Michael Mahoney. Empirical comparison of algorithms for network community detection. In *Proceedings of the 19th international conference on World wide web*, pages 631–640, 2010.

[170] Vincent A Traag, Paul Van Dooren, and Yurii Nesterov. Narrow scope for resolution-limit-free community detection. *Physical Review E*, 84(1):016114, 2011.

[171] Albert-László Barabási, Réka Albert, and Hawoong Jeong. Mean-field theory for scale-free random networks. *Physica A: Statistical Mechanics and its Applications*, 272(1-2):173–187, 1999.

[172] Brian W Kernighan and Shen Lin. An efficient heuristic procedure for partitioning graphs. *The Bell system technical journal*, 49(2):291–307, 1970.

[173] Jordi Duch and Alex Arenas. Community detection in complex networks using extremal optimization. *Physical review E*, 72(2):027104, 2005.

[174] Mark EJ Newman. Modularity and community structure in networks. *Proceedings of the national academy of sciences*, 103(23):8577–8582, 2006.

[175] Bo Yang, William Cheung, and Jiming Liu. Community mining from signed social networks. *IEEE transactions on knowledge and data engineering*, 19(10):1333–1348, 2007.

[176] Jörg Reichardt and Stefan Bornholdt. Detecting fuzzy community structures in complex networks with a potts model. *Physical review letters*, 93(21):218701, 2004.

[177] Andrew Ng, Michael Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. *Advances in neural information processing systems*, 14, 2001.

[178] Scott White and Padhraic Smyth. A spectral clustering approach to finding communities in graphs. In *Proceedings of the 2005 SIAM international conference on data mining*, pages 274–285. SIAM, 2005.

[179] Aaron Clauset, Mark EJ Newman, and Cristopher Moore. Finding community structure in very large networks. *Physical review E*, 70(6):066111, 2004.

[180] Liang Yang, Xiaochun Cao, Dongxiao He, Chuan Wang, Xiao Wang, and Weixiong Zhang. Modularity based community detection with deep learning. In *IJCAI*, volume 16, pages 2252–2258, 2016.

[181] Sandro Cavallari, Vincent W Zheng, Hongyun Cai, Kevin Chen-Chuan Chang, and Erik Cambria. Learning community embedding with community detection and node embedding on graphs. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, pages 377–386, 2017.

[182] Xiao Wang, Peng Cui, Jing Wang, Jian Pei, Wenwu Zhu, and Shiqiang Yang. Community preserving network embedding. In *Proceedings of the AAAI conference on artificial intelligence*, volume 31, 2017.

[183] Nagehan İlhan and Şule Gündüz Öğüdücü. Predicting community evolution based on time series modeling. In *Proceedings of the 2015 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining 2015*, pages 1509–1516, 2015.

[184] Shweta Bansal, Sanjukta Bhowmick, and Prashant Paymal. Fast community detection for dynamic complex networks. In *Complex Networks: Second International Workshop, CompleNet 2010, Rio de Janeiro, Brazil, October 13-15, 2010, Revised Selected Papers*, pages 196–207. Springer, 2011.

[185] Anne-Sophie Himmel, Hendrik Molter, Rolf Niedermeier, and Manuel Sorge. Enumerating maximal cliques in temporal graphs. In *2016 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, pages 337–344. IEEE, 2016.

[186] Jörg Reichardt and Stefan Bornholdt. Statistical mechanics of community detection. *Physical review E*, 74(1):016110, 2006.

[187] Martin Rosvall and Carl T Bergstrom. Maps of random walks on complex networks reveal community structure. *Proceedings of the national academy of sciences*, 105(4):1118–1123, 2008.

[188] Zhong-Yuan Zhang. Community structure detection in complex networks with partial background information. *Europhysics Letters*, 101(4):48005, 2013.

[189] Tze Meng Low, Daniele G Spampinato, Scott McMillan, and Michel Pelletier. Linear algebraic louvain method in python. In *2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 223–226. IEEE, 2020.

[190] Yuduo Wu, Yangzihao Wang, Yuechao Pan, Carl Yang, and John D Owens. Performance characterization of high-level programming models for gpu graph analytics. In *2015 IEEE International Symposium on Workload Characterization*, pages 66–75. IEEE, 2015.

[191] Shichang Zhang, Atefeh Sohrabizadeh, Cheng Wan, Zijie Huang, Ziniu Hu, Yewen Wang, Jason Cong, Yizhou Sun, et al. A survey on graph neural network acceleration: Algorithms, systems, and customized hardware. *arXiv preprint arXiv:2306.14052*, 2023.

[192] Yi-Chien Lin and Viktor Prasanna. Hyscale-gnn: A scalable hybrid gnn training system on single-node heterogeneous architecture. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 557–567. IEEE, 2023.

[193] Bingyi Zhang and Viktor Prasanna. Dynasparse: Accelerating gnn inference through dynamic sparsity exploitation. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 233–244. IEEE, 2023.

[194] Yi-Chien Lin, Bingyi Zhang, and Viktor Prasanna. Hitgnn: High-throughput gnn training framework on cpu+ multi-fpga heterogeneous platform. *IEEE Transactions on Parallel and Distributed Systems*, 2024.

[195] Zeyu Zhu, Fanrong Li, Gang Li, Zejian Liu, Zitao Mo, Qinghao Hu, Xiaoyao Liang, and Jian Cheng. Mega: A memory-efficient gnn accelerator exploiting degree-aware mixed-precision quantization, 2023.

[196] Kai Zhong, Shulin Zeng, Wentao Hou, Guohao Dai, Zhenhua Zhu, Xuecang Zhang, Shihai Xiao, Huazhong Yang, and Yu Wang. Cognn: An algorithm-hardware co-design approach to accelerate gnn inference with mini-batch sampling. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.

[197] Lizhi Zhang, Kai Lu, Zhiquan Lai, Yongquan Fu, Yu Tang, and Dongsheng Li. Accelerating gnn training by adapting large graphs to distributed heterogeneous architectures. *IEEE Transactions on Computers*, 2023.

[198] Jiaxian Chen, Zhaoyu Zhong, Kaoyi Sun, Chenlin Ma, Rui Mao, and Yi Wang. Lift: Exploiting hybrid stacked memory for energy-efficient processing of graph convolutional networks. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2023.

[199] Stefan Abi-Karam and Cong Hao. Gnnbuilder: An automated framework for generic graph neural network accelerator generation, simulation, and optimization. In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*, pages 212–218. IEEE, 2023.

[200] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, Hao Lu, Daniel Chavarrià-Miranda, Arif Khan, and Assefaw Gebremedhin. Distributed louvain algorithm for graph community detection. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 885–895, 2018.

[201] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, and Ananth Kalyanarainan. Scaling and quality of modularity optimization methods for graph clustering. In *2019 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–6. IEEE, 2019.

[202] Tim Mattson, Timothy A Davis, Manoj Kumar, Aydin Buluc, Scott McMillan, José Moreira, and Carl Yang. Lagraph: A community effort to collect graph algorithms built on top of the graphblas. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 276–284. IEEE, 2019.

[203] Aric Hagberg, Pieter Swart, and Daniel S Chult. Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2008.

[204] Omid Madani, Sai Ankith Averineni, and Shashidhar Gandham. A dataset of networks of computing hosts. In *Proceedings of the 2022 ACM on International Workshop on Security and Privacy Analytics*, pages 100–104, 2022.

[205] Hao Yin, Austin R Benson, Jure Leskovec, and David F Gleich. Local higher-order graph clustering. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 555–564, 2017.

[206] Jaewon Yang and Jure Leskovec. Defining and evaluating network communities based on ground-truth. In *Proceedings of the ACM SIGKDD Workshop on Mining Data Semantics*, pages 1–8, 2012.

[207] Gerrit Lahrmann, Frederik Marx, Tobias Mettler, Robert Winter, and Felix Wortmann. Inductive design of maturity models: applying the rasch algorithm for design science research. In *Service-Oriented Perspectives in Design Science Research: 6th International Conference, DESRIST 2011, Milwaukee, WI, USA, May 5-6, 2011. Proceedings 6*, pages 176–191. Springer, 2011.

[208] Raghad Baker Sadiq, Nurhizam Safie, Abdul Hadi Abd Rahman, and Shidrokh Goudarzi. Artificial intelligence maturity model: a systematic literature review. *PeerJ Computer Science*, 7:e661, 2021.